

Универзитет у Београду
Факултет организационих наука

Завршни рад

Вељко Благојевић 2023/3031

Београд, 2026. године

Универзитет у Београду

Факултет организационих наука

Информациони системи и технологије

Информациони системи

Завршни рад

**Развој веб апликације применом Microfrontend
архитектуре**

Вељко Благојевић 2023/3031

Београд, 2026. године

Сагласност чланова комисије за преглед, јавну одбрану и оцену завршног рада	
Комисија која је прегледала завршни рад	
<i>Студента:</i>	Вељко Благојевић
<i>Број индекса:</i>	2023/3031
<i>Под насловом</i>	Развој веб апликације применом Microfrontend архитектуре
<i>The Title of Master Thesis</i>	Web application development using microfrontend architecture
У следећем саставу	
Ментор:	др Слађан Бабарогић, редовни професор
Први члан:	др Иван Луковић, редовни професор
Други члан:	др Бојан Јовановић, ванредни професор
дала је електронску сагласност за јавну одбрану завршног рада.	

Изјава о академској честитости

Име и презиме: Вељко Благојевић

Број индекса: 2023/3031

Студијски програм: Информациони системи и технологије

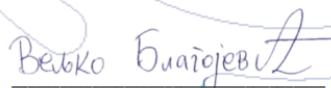
Модул: Информациони системи

Аутор завршног рада под насловом	Развој веб апликације применом Microfrontend архитектуре
Чија је израда одобрена на Седници Већа студијских програма мастер академских студија одржаној:	25.9.2024.

Потписивањем изјављујем:

- Да је рад искључиво резултат мог сопственог истраживачког рада;
- Да сам рад и мишљења других аутора које сам користио у овом раду назначио или цитирао у складу са Упутством;
- Да су сви радови и мишљења других аутора наведени у списку литературе/референци који су саставни део овог рада и писани су у складу са Упутством;
- Да сам довио све дозволе за коришћење ауторског дела који се у потпуносци/целости уносе у предати рад и да сам то јасно навео;
- Да сам свестан да је плагијат коришћење туђих радова у било ком облику (као цитата, парафраза, слика, табела, дијаграма, дизајна, планова, фотографија, филма, музике, формула, веб сајтова, компјутерских програма и сл.) без навођења аутора или представљање туђих ауторских дела као мојих, кажњиво по закону (Закон о ауторским и сродним правима, Службени гласник Републике Србије, бр. 104/2009, 99/2011, 119/2012), као и других закона и одговарајућих аката Универзитета у Београду и Факултета организационих наука;
- Да сам свестан да плагијат укључује и представљање, употребу и дистрибуирање рада предавача или других студената као сопствених;
- Да сам свестан последица које код доказаног плагијата могу проузроковати на предати завшни мастер рад и мој статус;
- Да је електронска верзија завршног рада идентична штампаном примерку и пристајем на његово објављивање под условима прописаним актима Универзитета и Факултета.

Београд, 01.09.2026. године


Потпис студента

РАДНА БИОГРАФИЈА

Вељко Благојевић је рођен 02.08.2000. године у Панчеву где и завршава средњу електротехничку школу 2019. године. Након чега похађа Факултет организационих наука и дипломира 2023. године на смеру Информациони системи и технологије. Од 2022. креће на праксу у компанији BlackRock где и данас ради као програмер.

Резиме

Циљ мастер рада јесте разумевање и вредновање примене микрофронтенд архитектуре на клијентском слоју веб апликације: шта се преласком са фронтенд монолита добија, шта се губи и под којим условима је та цена оправдана. Домен истраживања је клијентски слој веб апликације MicroMedic, намењене заказивању и вођењу лекарских прегледа, чији је кориснички интерфејс подељен на функционално раздвојива подручја одговорности лекара и пацијента.

Рад је заснован на методи истраживања кроз пројектовање (енгл. *design science research*): знање се стиче изградњом артефакта - микрофронтенд архитектуре реализоване кроз више независних делова у четири различита радна оквира, и његово вредновање по унапред утврђеним архитектонским карактеристикама. Вредновање је довело до четири налаза: разноликост радних оквира у једном интерфејсу није сама по себи анти-патерн него последица одсуства механизма управљања; конзистентност изгледа међу деловима писаним у различитим оквирима постиже се конструкцијом кроз заједнички систем дизајна; избор механизма комуникације међу деловима следи из њиховог тополошког односа на екрану; и микрофронтенд архитектура успоставља стварне границе одговорности, али не и безбедносне границе на страни клијента, које остају искључиво обавеза серверског слоја.

Главно ограничење рада је цена коју микрофронтенд архитектура наплаћује у изворном коду: умножен број пројеката и договарање о заједничким зависностима. Вредност рада је у томе што су налази изведени из система изграђеног у целини, а не хипотетичке анализе.

Кључне речи: Microfrontend, микросервиси, софтверска архитектура, клијентски слој корисничког интерфејса

The goal of this master's thesis is to understand and evaluate the application of microfrontend architecture on the client side of a web application: what is gained by transitioning from a frontend monolith, what is lost, and under what conditions the cost is justified. The research domain is the client side of the MicroMedic web application, designed for scheduling and managing medical appointments, whose user interface is divided into functionally distinct areas of responsibility for doctors and patients.

The work is based on the design science research method: knowledge is acquired by building an artifact - a microfrontend architecture implemented through multiple independent parts in four different frameworks — and evaluating it according to predetermined architectural characteristics. The evaluation led to four findings: the diversity of frameworks within a single interface is not inherently an anti-pattern but a consequence of the absence of management mechanisms; visual consistency among parts written in different frameworks is achieved through construction via a shared design system; the choice of communication mechanisms between parts follows from their topological relationship on the screen; and the microfrontend architecture establishes real boundaries of responsibility but not client-side security boundaries, which remain an exclusive obligation of the server layer.

The main limitation of the work is the cost that microfrontend architecture incurs in the source code - a multiplied number of projects and coordination of shared dependencies. The value of the work lies in the fact that the findings are derived from a fully built system rather than hypothetical analysis.

Keywords: Microfrontend, microservices, software architecture, client-side user interface

Листа акронима

Акроним	Значење
XML	Extensible Markup Language
AJAX	Asynchronous JavaScript and XML
API	Application Programming Interface
SPA	Single Page Application
UI	User Interface
HTML	HyperText Markup Language
DOM	Document Object Model
CSS	Cascading Style Sheets
DDD	Domain-driven design
CORS	Cross-Origin Resource Sharing
DNS	Domain Name System
SSI	Server Side Includes
TTFB	Time to First Byte
ESI	Edge Side Includes
CDN	Content Delivery Network
UTF	Unicode Transformation Format
FON	Fakultet organizacionih nauka
ECMA	European Computer Manufacturers Association
ES	ECMAScript
JSON	JavaScript Object Notation
kB	kilobyte

OAuth	Open Authorization
JWT	JSON Web Token
BFF	Backend for Frontend
GraphQL	Graph Query Language
REST	Representational State Transfer
E2E	End to End
CSP	Content Security Policy
XSS	Cross-site scripting
HSTS	HTTP Strict Transport Security
HTTPS	Hypertext Transfer Protocol Secure
OWASP	Open Worldwide Application Security Project
МКБ-10	Међународна класификација болести, десета ревизија
ICD-10	International Classification of Diseases, 10th Revision
PDF	Portable Document Format
MM	MicroMedic

Листа табела

Табела 2-1 Архитектонске карактеристике	5
Табела 10-1 Учесници и њихове надлежности у домену	54
Табела 10-2 Функционални захтеви	57
Табела 10-3 Архитектонске карактеристике	58
Табела 10-4 Поддомени, ограничени контексти и врста границе	60
Табела 11-1 Од ограниченог контекста до фрагмента	62
Табела 11-2 Механизам према топологији, и шта се њиме предаје	64
Табела 11-3 Оквир за одлучивање испуњен за MicroMedic.....	65
Табела 12-1 Топологија микрофронтендова у изведеном систему	71
Табела 12-2 Изведени механизми комуникације и место сваког у артефакту.....	77
Табела 12-3 Адаптација по потрошачу и недостатак који надомешта	87

Листа слика

Слика 2-1 Трослојна софтверска архитектура	7
Слика 2-2 Микросервисна софтверска архитектура.....	8
Слика 2-3 Традиционални модел веб апликације (лево) у поређењу са Ајах моделом (десно) (Garrett, 2005).....	8
Слика 2-4 Патерн синхроне интеракције традиционалне веб апликације (горе) у поређењу са патерном асинхроне комуникације Ајах модела (доле) (Garrett, 2005).....	9
Слика 3-1 Хоризонтални наспрам вертикалног типа поделе (Mezzalira, Micro-frontends in context, 2020).....	14
Слика 3-2 Пример хоризонталне поделе на мултимедијалном систему аутомобила	15
Слика 3-3 Пример вертикалне поделе на мултимедијалном систему аутомобила	15
Слика 4-1 Почетна страна фиктивне апликације ФОН-а	16
Слика 4-2 Страница за преузимање фиктивне апликације ФОН-а	17
Слика 4-3 Преусмеравање страница путем а href тага	17
Слика 4-4 Упоредба три типа композиције (Mezzalira, Micro-frontends in context, 2020)	18
Слика 4-5 Коришћење iframe тага у HTML документу.....	19
Слика 4-6 Приказ iframe компоненте на страници	20
Слика 4-7 DOM стабло унутар iframe тага	20
Слика 4-8 Прилагођено дугме унутар DOM "под сенком"	23
Слика 4-9 Апликација обмотач повлачи микрофронтендове на основу захтева (Chen, 2021).26	
Слика 4-10 Рутирање на једном и два нивоа (Chen, 2021).....	27
Слика 5-1 Комуникација родитеља и фрагмента	29
Слика 5-2 Комуникација фрагмената уз помоћ догађаја (Mezzalira, Micro-frontends in context, 2020)	31
Слика 5-3 Broadcast Channel	32
Слика 5-4 Claim Check патерн (Mosyan, 2023).....	33
Слика 5-5 Веб складиште и колачићи.....	34
Слика 5-6 API капија у микросервисној архитектури (Ozkaya, 2021)	35

Слика 5-7 Миграција монолита уз помоћ обрасца смокве давитељке (Villavicencio, 2023)	36
Слика 5-8 Поређење API капије и BFF обрасца (Das, 2024).....	36
Слика 5-9 REST API наспрам GraphQL API (Mukhadin, 2026).....	36
Слика 7-1 Blue-Green испорука (Maruthi, 2022).....	43
Слика 7-2 Canary испорука (Rastogi, 2022).....	43
Слика 7-3 Пирамида тестова (Fowler, 2012).....	45
Слика 7-4 Промена домена микрофронтенда при локалном развоју.....	45
Слика 9-1 Ситна подела микрофронтендова (Rappl, Top 10 Micro Frontend Anti-Patterns, 2024)	50
Слика 9-2 Чврста спрегнутост микрофронтендова (Rappl, Top 10 Micro Frontend Anti-Patterns, 2024)	50
Слика 9-3 Неконзистентност дистрибуираних података (Rappl, Top 10 Micro Frontend Anti- Patterns, 2024)	51
Слика 9-4 Велики број различитих радних оквира (Rappl, Top 10 Micro Frontend Anti-Patterns, 2024)	52
Слика 9-5 Више микрофронтендова позива исти API.....	53
Слика 10-1 Дијаграм активности радног тога лекара.....	55
Слика 11-1 Проширени модел објекти везе MicroMedic	67
Слика 12-1 Листа пакета на клијентском слоју.....	68
Слика 12-2 Иницијално повлачење зависности и auth пакета.....	72
Слика 12-3 Страница за пријаву	73
Слика 12-4 Страница за регистрацију	73
Слика 12-5 Хоризонтална подела examination и icd10 фрагмента	74
Слика 12-6 Приказ nav и notifications фрагмента након пријаве.....	75
Слика 12-7 Учитавање calendar, nav, header и footer скрипти тек након пријаве корисника ...	76
Слика 12-8 Одабир дијагнозе у icd10 фрагменту окида догађај у магистрали који examination фрагмент ослушкује и уписује у формулар.....	79
Слика 12-9 Параметар из адресе као предајни податак при транзицији и комуникацији calendar и examination фрагмента.....	80

Слика 12-10 Веб складиште са токеном сесије	83
Слика 12-11 Списак компоненти унутар система дизајна	84

Садржај

1	Увод.....	1
2	Софтверске архитектуре	5
2.1	Архитектонске карактеристике и компромиси	5
2.2	Архитектонски стилови.....	6
2.3	Од вишестраничних ка једностраничним апликацијама.....	8
2.4	Фронтенд монолит као проблем.....	9
2.5	Модуларност на фронтенду	10
3	Микрофронтенд архитектура	11
3.1	Предности, недостаци и критеријуми примене	12
3.2	Дефинисање ограниченог контекста.....	13
3.3	Простор типова микрофронтенд архитектуре.....	14
3.4	Тимске границе и Конвејев закон.....	15
4	Композиција, интеграција и рутирање	16
4.1	Транзиција путем линкова и рутирање на страни сервера	16
4.2	Композиција на страни сервера	18
4.3	Композиција на страни клијента	19
4.4	Module Federation и дељење зависности.....	24
4.5	Апликација обмотач и рутирање на страни клијента.....	26
5	Комуникација и управљање стањем	29
5.1	Три смера: родитељ→фрагмент, фрагмент→родитељ, фрагмент↔фрагмент	29
5.2	Магистрала догађаја: прилагођени догађаји и <i>Broadcast Channel API</i>	30
5.3	Пренос преко адресе	33
5.4	Глобални контекст и аутентификација	33
5.5	Локално складиште и колачићи.....	34
5.6	Комуникација са серверским слојем.....	35
5.7	Репликација података међу тимовима	37
6	Кориснички интерфејс и систем дизајна.....	39

6.1	Систем дизајна наспрам библиотеке компонената.....	39
6.2	Изузетак од начела о дељењу	39
6.3	Критеријуми за дељење.....	40
7	Испорука, аутоматизација и тестирање.....	42
7.1	Моно-репозиторијум, поли-репозиторијум и стратегија рада с гранама	42
7.2	Верзионисање, повратак на претходну верзију и прекидачи функционалности.....	42
7.3	Миграција са монолитног фронтенда	44
7.4	Стратегија тестирања и функције прикладности.....	44
7.5	Локални развој и изолација фрагмената	45
7.6	Развојно искуство.....	46
8	Безбедност и изолација микрофронтендова.....	47
8.1	Стандарди веб платформе као граница.....	47
8.2	Границе скриптова.....	47
8.3	Складиштење приступног токена.....	48
8.4	Дељене зависности.....	48
9	Анти-патерни микрофронтенд архитектуре	49
9.1	Погрешна гранулација: микрофронтенд или компонента	49
9.2	Дељење стања и двосмерна зависност	50
9.3	Анархија: вишеструки оквири без механизма управљања.....	51
9.4	Недостатак слоја против корупције и преурањена апстракција.....	52
9.5	Вишеструки позиви ка бекенду	53
10	Анализа проблема истраживања	54
10.1	Домен, корисници и радни ток лекара.....	54
10.2	Функционални захтеви	57
10.3	Нефункционални захтеви	58
10.4	Идентификација поддомена и ограничених контекста	59
10.5	Ограничења монолитног фронтенда у посматраном домену	60
10.6	Захтеви поверљивости и трага приступа медицинским подацима	61

11	Архитектура решења	62
11.1	Примена оквира за одлучивање: дефинисање граница фрагмената	62
11.2	Композиција	62
11.3	Рутирање	63
11.4	Комуникација: типизирана магистрала догађаја	63
11.5	Оквир за одлучивање	64
11.6	Серверски слој	65
12	Имплементација клијентског слоја	68
12.1	Структура моно-репозиторијума и радни простори	68
12.2	Апликација обмотач: рутирање, функције активације, капија аутентификације	69
12.3	Топологија микрофронтендова	71
12.4	Вертикална подела: пакет auth	72
12.5	Хоризонтална подела: examination уз icd10	74
12.6	Увек монтирани фрагменти: nav и notifications	75
12.7	Комуникација између фрагмената: изведени механизми	77
12.8	Систем дизајна: једна изведба, четири окружења	83
12.9	Дељено стање и заједнички приступ сервисима	87
12.10	Вишеструки оквири као намерни оглед	88
12.11	Остварене безбедносне мере	88
13	Закључак	93
	Литература	96

1 Увод

Архитектура, као дисциплина која обухвата уметност и науку пројектовања и изградње, има дубоке корене у историји људске цивилизације. Од најранијих времена, људи су користили архитектуру као средство за изражавање својих културних вредности, технолошких достигнућа и друштвених потреба. Кроз векове, архитектура је еволуирала, прилагођавајући се променама у друштву, технологији и естетици, док је истовремено обликовала начин на који живимо, радимо и комуницирамо са нашим окружењем. (Collins, Gowans, Ackerman, & Scruton, 2026)

Реч „архитектура“ потиче од старогрчке речи „*arkhitēktōn*“, што у преводу означава „главни градитељ“ и означава оног који води градњу, а не оног који је изводи. У том значењу архитектура нису технички детаљи градње, него низ одлука о томе шта се гради и по којим приоритетима: одлука које се доносе пре него што градња почне и које је утолико теже променити што је градња дуже одмакла.

Те одлуке никад нису биле само техничке. Египатске пирамиде и грчки храмови обављају функцију за коју би биле довољне много једноставније конструкције, а њихов облик и мере сведоче о приоритетима: трајности, симетрији, видљивости - који су надживели непосредну намену градње (Münster, 2024). Управо такви приоритети, а не функција, одговарају ономе што се у софтверској архитектури назива *архитектонским карактеристикама*: својствима система које се не могу прочитати из списка његових функција и о којима се стога одлучује компромисом, а не избором (Richards & Ford, 2025).

Развој корисничког интерфејса веб апликација прошао је кроз промену сличног реда величине. Увођењем асинхроне размене података са сервером, описане као *Ajax* (енгл. *Asynchronous JavaScript and XML*) (Garrett, 2005), страница је престала да буде документ који се при свакој интеракцији поново учитава и постала апликација која се извршава у прегледачу (енгл. *web browser*). Радни оквири (енгл. *frameworks*) настали у наредним годинама учинили су такав начин развоја подразумеваним, а једностраничну апликацију (енгл. *Single-page application*) стандардним обликом клијентског слоја (Mikowski & Powell, 2013).

Тај облик, међутим, носи последицу која се испољава тек са растом. Једностранична апликација је по правилу један пројекат, једна градња (енгл. *build*) и једна испорука (енгл. *delivery*): свака промена, колико год локална, захтева поновно превођење и објављивање целине, а број људи који на њој могу истовремено радити ограничен је бројем људи који се могу договорити о једној верзији сваке екстерне библиотеке и о једном тренутку испоруке.

Са растом организације та последица престаје да буде својство једне апликације и постаје неравнотежа између слојева. Серверски слој разложен је на микросервисице (енгл. *microservices*) које независни тимови испоручују одвојено, а тим који свој сервис може да објави неколико пута дневно чека на заједничку испоруку фронтенда да би тај сервис постао видљив кориснику. Фронтенд монолит стога није проблем због своје величине, него зато што је уско грло тока испоруке који је свуда осим у њему разложен (Geers, 2020).

Микрофронтенд архитектура (енгл. *Micro frontend*) настала је као одговор на ту неравнотежу: кориснички интерфејс дели се на независно развијане, испоручиване и извршаване делове, граница међу њима није техничка него пословна, а место његовог састављања у целини јесте прегледач (Geers, 2020) (Mezzalana, Building Micro-Frontends, 2025). Подела доноси аутономију тимова и независну испоруку, али и цену - прегледач при састављању мора да реши проблем рутирања, комуникацију између делова, конзистентност изгледа, количину преузетог кода и границе поверења међу деловима, а ниједан од тих проблема у монолитној апликацији није постојао, јер их је радни оквир решавао имплицитно, у позадини без потребе за напором од стране програмера.

Најоспораванија последица те поделе јесте могућност да делови интерфејса буду имплементирани у различитим радним оквирима. Литература је према томе изразито уздржана и такву употребу најчешће описује као анти-патерн, наводећи увећано преузимање кода, неусаглашен изглед и отежано одржавање (Mezzalana, Building Micro-Frontends, 2025) (Rufus, 2023). Уздржаност је, међутим, усмерена на одсуство механизма управљања, а не на разноликост саму по себи, а разлика између те две тврдње не решава се начелним расуђивањем, него анализом над изграђеним системом.

Тема овог рада припада области архитектуре информационих система, а домен истраживања је клијентски слој веб апликација са више функционално раздвојивих подручја. Предмет рада је микрофронтенд архитектура као начин организације тог слоја, а циљ је да се на конкретном, у целини изграђеном систему испита шта се преласком са монолитног фронтенда на микрофронтенд архитектуру добија, шта се губи и под којим условима употреба више радних оквира остаје контролисана одлука, а не анти-патерн. Систем на коме се испитивање изводи јесте „MicroMedic“, информациони систем за заказивање и евиденцију лекарских прегледа, одабран зато што његов домен садржи јасно раздвојиве поддомене, радни ток који на једном екрану захтева више облика поделе интерфејса и начина комуникације.

Рад је заснован на методи истраживања кроз пројектовање (енгл. *design science research*), у којој се знање стиче изградњом артефакта и његовим вредновањем по унапред утврђеним

критеријумима (Vaishnavi & Kuechler, 2015). Артефакт је поменути систем, чији је клијентски слој реализован као скуп независно испоручивих микрофронтендова свесно имплементираних у више различитих радних оквира, док је серверски слој намерно остављен монолитним да би променљива која се испитује била ограничена на клијентски слој.

Друго поглавље уводи појам софтверске архитектуре, архитектонске карактеристике као критеријум по коме се архитектуре упоређују, монолитне и дистрибуиране стилове, и развој од вишестраничних до једностраничних апликација, до фронтенд монолита као проблема.

Треће поглавље излаже микрофронтенд архитектуру: дефиницију и принципе, предности и недостатке, дефинисање граница помоћу домена, простор њених типова, и оквир за одлучивање који се у наставку рада користи као аналитички инструмент.

Четврто поглавље обрађује композицију, интеграцију и рутирање, од транзиције путем линкова, преко композиције на страни сервера и клијента, до механизма *Module Federation*, и утицај изабране технике на учитавање ресурса.

Пето поглавље разматра комуникацију и управљање стањем: три смера комуникације, магистралу догађаја, глобални контекст и аутентификацију, и комуникацију са серверским слојем.

Шесто поглавље посвећено је корисничком интерфејсу (енгл. *user interface*) и систему дизајна (енгл. *design system*), како одржати конзистентност изгледа када интерфејс израђује више међусобно независних тимова.

Седмо поглавље описује испоруку, аутоматизацију и тестирање: непрекидну интеграцију, однос моно-репозиторијума и поли-репозиторијума, верзионисање, метрике испоруке и слојеве тестне стратегије.

Осмо поглавље обрађује безбедност и изолацију микрофронтендова: стандарде веб платформе, питање зашто изолација на страни клијента није решива, аутентификацију и складиштење токена.

Девето поглавље систематизује анти-патерне микрофронтенд архитектуре, изложене после свих механизма које критикују, и међу њима издваја анархију, употребу више радних оквира без механизма управљања, као случај који је непосредно предмет овог рада.

Десето поглавље анализира проблем истраживања на конкретном домену: радни ток лекара при прегледу пацијента, функционалне захтеве, нефункционалне захтеве изражене као

архитектонске карактеристике, идентификоване поддомене, и захтеве поверљивости и трага приступа медицинским подацима.

Једанаесто поглавље приказује архитектуру решења „MicroMedic“, објашњава одлуке којима је пројектован клијентски слој и описује серверски слој.

Дванаесто поглавље детаљно описује имплементацију клијентског слоја: апликацију обмотач (енгл. *shell application*), топологију микрофронтендова, оба примењена облика поделе интерфејса, дељено стање, систем дизајна са посредничким пакетима, употребу више радних оквира као контролисани оглед, и остварене и неостварене безбедносне мере.

Тринаесто поглавље резимира до чега је рад дошао кроз четири налаза, наводи доприносе рада и правце даљих истраживања.

2 Софтверске архитектуре

Софтверска архитектура обухвата структуру система, његове делове и односе међу њима, заједно са одлукама и начелима по којима је та структура изведена (Clements, et al., 2010).

Пројектовање структуре обухвата две делатности: разумевање пословног домена, који одређује шта систем ради, и утврђивање својстава која систем мора да поседује да би био успешан, а која одређују како и под којим условима то ради (Richards & Ford, 2025). Прва описује понашање система, друга његове способности. Та својства називају се архитектонским карактеристикама, а именовани начини да се одређени њихов скуп подржи, архитектонским стиливима.

Организационо начело обе делатности је модуларност, груписање сродног кода у делове који се разумевају и мењају одвојено, при чему је то раздвајање логичко, а не обавезно физичко (Richards & Ford, 2025). Скуп класа може бити уредно подељен по одговорностима, а да се и даље испоручује као једна целина.

2.1 Архитектонске карактеристике и компромиси

Својства која систем мора да поседује, а која се не могу прочитати из списка његових функција, називају се *нефункционалним захтевима, атрибутима квалитета* или *архитектонским карактеристикама*. (Richards & Ford, 2025) први термин одбацују зато што вредносно омаловажава оно што описује, а други зато што квалитет упућује на накнадну оцену готовог система, а не на предмет пројектовања; стога се овде користи трећи.

Да би захтев био архитектонска карактеристика, мора да задовољи три услова: да не припада пословном домену, да утиче на структуру решења и да буде критичан за успех система (Richards & Ford, 2025). Трећи услов је ограничавајући, зато што свака подржана карактеристика уводи сложеност: циљ је изабрати најмањи број њих.

Карактеристике се разврставају на оперативне, структурне и унакрсне. Део њих је изричит и стоји у захтевима, а део имплицитан: доступност, поузданост и безбедност подразумевају се у готово свакој апликацији, па их архитекта изводи из познавања домена.

Табела 2-1 Архитектонске карактеристике

Карактеристика	Група	Одређење
----------------	-------	----------

Перформансе	оперативна	Време одзива и трајање почетног учитавања при очекиваном оптерећењу.
Скалабилност	оперативна	Способност система да задржи одзив при расту броја корисника или захтева.
Доступност	оперативна	Део времена у коме је систем употребљив.
Модуларност	структурна	Степен разложености система на делове високе кохезије и мале спрегнутости.
Испоручивост	структурна	Учестаност, лакоћа и ризик испоруке једне измене у продукцију.
Тестабилност	структурна	Лакоћа потпуне провере система аутоматизованим тестовима.
Проширивост	структурна	Лакоћа додавања нове функционалности без измене постојећих делова.
Безбедност	унакрсна	Отпорност система на неовлашћен приступ и на злонамерну употребу.
Поверљивост	унакрсна	Заштита података о личности од приступа којем нису намењени.
Ауторизација	унакрсна	Провера да учесник има право на радњу коју захтева.
Приступачност	унакрсна	Употребљивост интерфејса помоћним технологијама.
Агилност	сложена	Сложена карактеристика: испоручивост, модуларност и тестабилност заједно.

Однос карактеристике и структуре показује поређење безбедности и скалабилности: безбедност се у монолитном систему може обезбедити пројектовањем, шифровањем, хеширањем и дисциплином у писању кода, док скалабилност преко одређене границе никаквим пројектовањем није достижна, него захтева прелазак на дистрибуирани стил (Richards & Ford, 2025). Прва је, дакле, карактеристика која структуру може, а друга која структуру мора да промени.

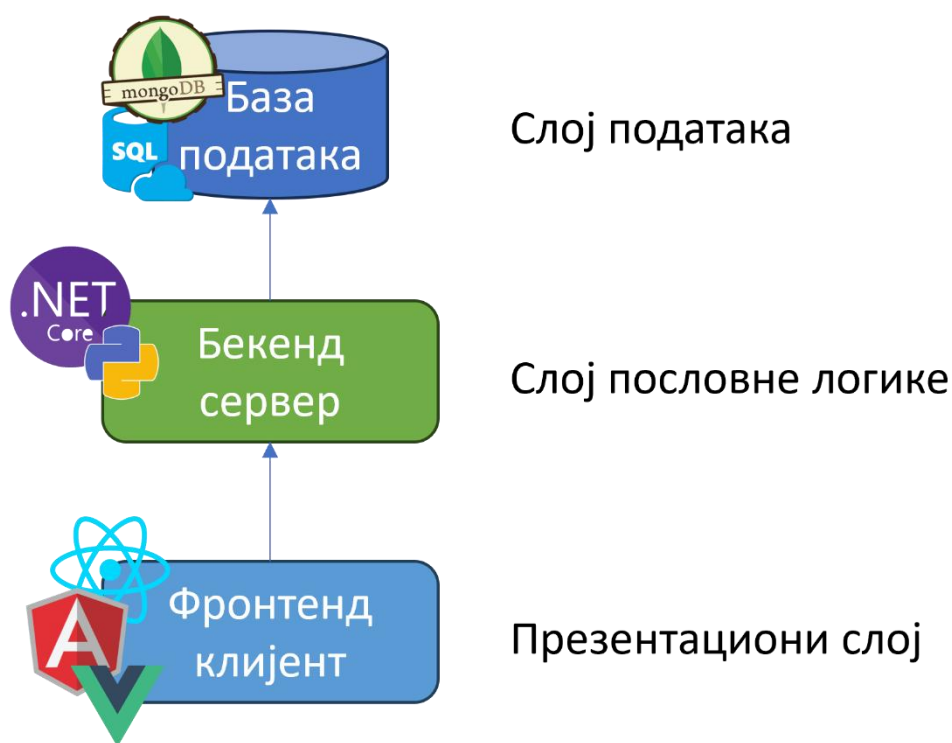
Ниједна карактеристика не долази без цене, а карактеристике делују и једна на другу, појачана безбедност готово увек оптерећује перформансе. Отуда начело да не треба тражити најбољу архитектуру, него најмање лошу (Richards & Ford, 2025), и отуда се архитектуре упоређују по карактеристикама, једној по једној.

2.2 Архитектонски стилови

Архитектонски стил је именован скуп ограничења над структуром система, којим се одређене карактеристике подржавају, а друге жртвују. Силови се разврставају у две породице: *монолитне*, у којима се цео систем испоручује као једна јединица, и *дистрибуиране*, у којима су делови самостално испоручиви (Richards & Ford, 2025).

Монолитна архитектура заснива се на једној кодној бази (енгл. *codebase*) која обухвата све функционалности (Mishra, Kunde, & Nambiar, 2018). Иста особина носи и њене предности: једноставност развоја, тестирања и испоруке, и њене изазове: промена било где захтева испоруку целине, а систем се скалира само умножавањем у целости.

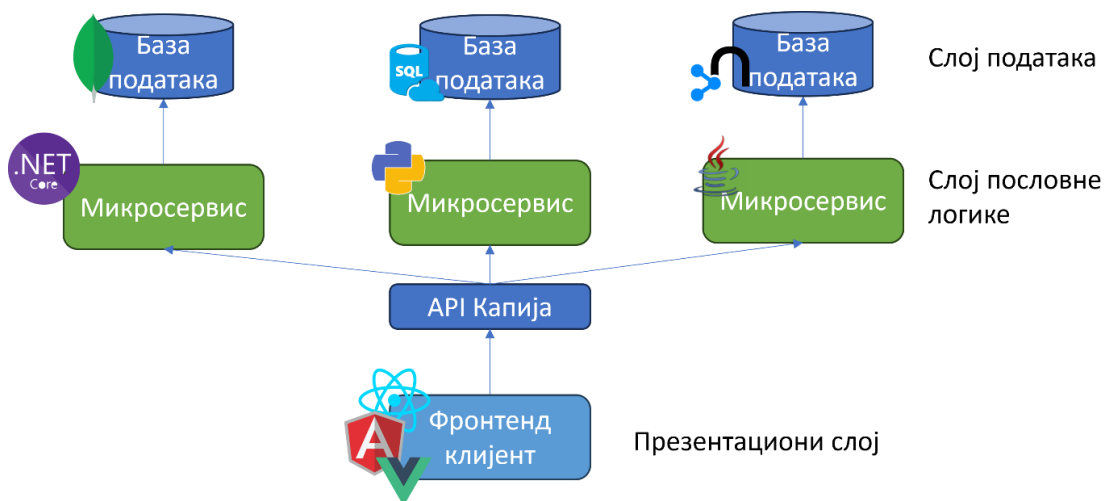
Унутар монолитне породице разликују се два начина поделе, који се називају **техничком** и **доменском**. Слојевита архитектура је технички подељена: систем се дели хоризонтално, по техничкој улози делова, најчешће на кориснички интерфејс, пословну логику, приступ подацима и складиштење (Влајић, 2020). Слојеви су при том затворени, тако да промена у једном не досеже до осталих докле год се уговор међу њима не мења, а то представља својство које се назива *изолацијом слојева*. Пошто је подела техничка, један пословни домен пресеца све слојеве, што у организацији подељеној по слојевима значи да у изради једне функционалности учествује више тимова. Модуларни монолит је доменски подељен: исти систем дели се вертикално, на модуле који одговарају пословним доменима и повезани су добро дефинисаним интерфејсима, најчешће над једном базом података (Gandhi, Richards, & Ford, 2024). Таква подела олакшава паралелан рад тимова организованих по домену, али се флексибилност испоруке практично не мења, модули се и даље испоручују као једна јединица, па и модуларни монолит остаје монолит у оном својству које је за овај рад најважније.



Слика 2-1 Трослојна софтверска архитектура

Дистрибуирани стилови ту границу премештају: систем се дели на јединице које се испоручују одвојено, а сарађују преко стандардизованих уговора, због чега свака може да користи технологију прилагођену свом задатку. У микросервисној архитектури сервиси су моделовани око пословних домена, а начела на којима почива јесу независна испорука, пословни домен као основа поделе и засебан домен података по сервису, чиме изолованост престаје да буде својство само апликативног слоја (Newman, 2021). Цена је, међутим, иста у свим

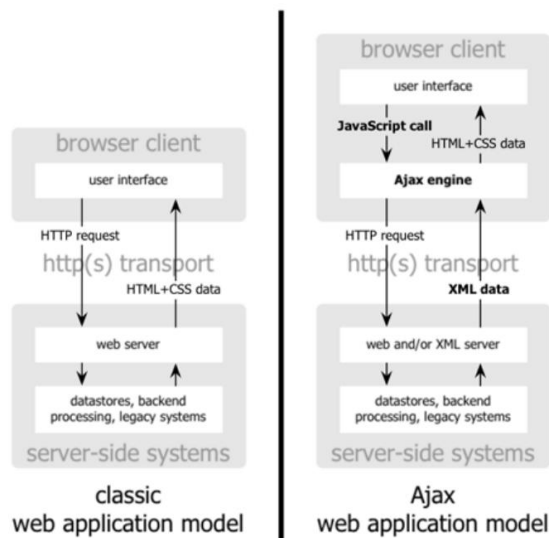
дистрибуираним стилovima: мрежа постаје тачка отказа, конзистентност слаби, а сложеност се премешта из кодне базе у испоруку и надгледање.



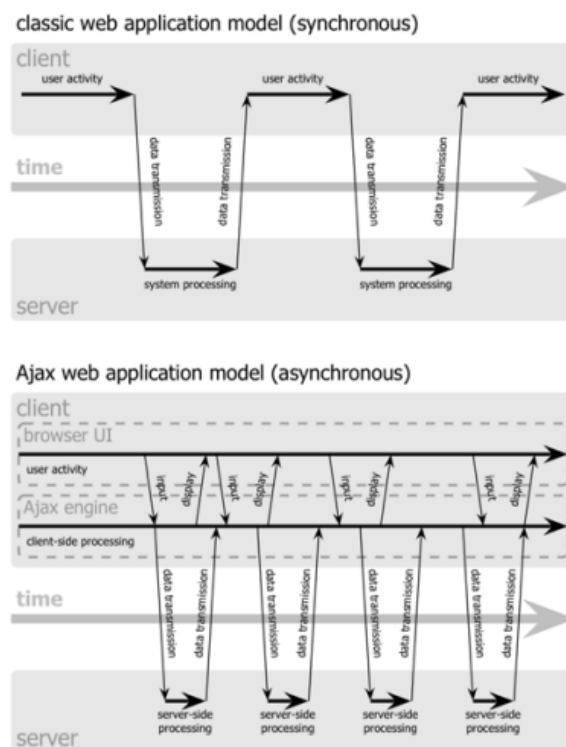
Слика 2-2 Микросервисна софтверска архитектура

2.3 Од вишестраничних ка једностраничним апликацијама

У вишестраничним апликацијама документ саставља сервер, а свака интеракција која мења приказ јесте нов захтев и поновно учитавање целе странице. Развојни образац описан као *Ajax* (енгл. *Asynchronous JavaScript and XML*) прекинуо је ту везу: посреднички механизам у прегледачу асинхроно размењује податке са сервером, па корисничка интеракција више не чека мрежни циклус (Garrett, 2005).



Слика 2-3 Традиционални модел веб апликације (лево) у поређењу са Ајах моделом (десно) (Garrett, 2005)



Слика 2-4 Патерн синхроне интеракције традиционалне веб апликације (горе) у поређењу са патерном асинхроне комуникације Ајах модела (доле) (Garrett, 2005)

Из тог обрасца настале су **једностраничне апликације**: читав фронтенд преузима се унапред, у виду *JavaScript* датотека, а потом се мењају само делови приказа, док апликација у позадини преузима податке са приступних тачака сервера (Monteiro, 2014) (Yadav, 2025). Клијент тако преузима одговорности које је раније имао сервер, рутирање, управљање стањем и састављање приказа, чиме подела на слојеве остаје, али се граница међу њима помера, а клијент постаје „дебео“ (Mikowski & Powell, 2013).

Добијено је корисничко искуство блиско оном код десктоп апликација, али не без цене: почетно учитавање је дуже, индексирање садржаја претраживачима захтева додатне поступке, и, што је за овај рад најважније, целина клијентског слоја постаје један пројекат, једна градња и једна испорука.

2.4 Фронтенд монолит као проблем

Последица описана на крају претходног одељка не зависи од стила у коме је остатак система изведен. Монолитна архитектура, подела на фронтенд (енгл. *frontend*) и бекенд (енгл. *backend*), и микросервисна архитектура имају једно заједничко својство: у свима је фронтенд монолитан, то јест долази из једне кодне базе на којој смислено може да ради само један тим.

Из тога следе два ограничења. Прво је организационо: ако су тимови подељени по слојевима, у изради једне функционалности учествује више њих, а свака граница међу њима уводи

договор, специфицирање и чекање, смањивање тог чекања је основни циљ микрофронтенд архитектуре. Друго је техничко: једна кодна база значи једну верзију радног оквира и једну одлуку о надоградњи за цео интерфејс, па се нова технологија не може увести и проверити у ограниченом делу апликације без плана миграције за целину (Geers, 2020).

2.5 Модуларност на фронтенду

Модуларност се мери кохезијом, која показује колико делови једног модула припадају заједно, и спрегнутошћу (енгл. *coupling*), која показује колико модул зависи од других; спрегнутост је при том бројива у два смера, бројем модула који зависе од датог и бројем модула од којих он зависи (енгл. *afferent* и *efferent coupling*) (Richards & Ford, 2025). На клијентском слоју јединица тог груписања је **компонента**: целина која обједињује структуру приказа, стил и понашање, а окружењу излаже сужен интерфејс, својства која прима и догађаје које објављује.

Потреба да компоненте оду и на платформски ниво, ван радног оквира, тиме долази до стварања стандарда **Веб компоненти** (енгл. *Web Components*), састављен од три технологије (Ottaviani Aragão, 2024): *елемената по мери* (енгл. *custom elements*), који уводе нове ознаке у HTML (енгл. *HyperText Markup Language*) заједно са методама животног циклуса; *HTML шаблона*, фрагмената документа који се не приказују док не буду употребљени; и *DOM-а „под сенком“* (енгл. *Shadow Document Object Model*), одвојеног DOM дрвета чија су структура и стилови затворени за остатак документа, док догађаји и даље прелазе његову границу (Glazkov, 2011).

3 Микрофронтенд архитектура

Одељак 2.4 показао је да фронтенд остаје неразложен и онда када је остатак система већ дистрибуиран. Микрофронтенд архитектура настаје тако што се доменска подела (2.2), већ примењена на сервер кроз микросервисе, проширује и на презентациони слој: интерфејс се дели на самостално развијане, тестиране и испоручиве делове, а не само логика испод њега (Mezzalira, Building Micro-Frontends, 2025).

Микрофронтенд је, отуда, независно развијена, тестирана и испоручена кришка корисничког интерфејса која представља један пословни поддомен, а с другим микрофронтендовима комбинује се тек у извршавању, у прегледачу, у целовиту апликацију (Mezzalira, Building Micro-Frontends, 2025). Апликација која те делове обједињује назива се **обмотачем** (енгл. *application shell*): она сама не носи пословну логику, него дистрибуира делове, обезбеђује им заједнички простор за читавање и, најчешће, управља аутентификацијом и заједничким процесима.

Разликују се два облика у којима микрофронтенд улази у екран: као **страница**, када је цео поглед заузет једним микрофронтендом, или као **фрагмент**, самостални део који се уклапа у туђи поглед. Овај рад у наставку користи оба израза: страница за облик који одговара вертикалној подели (3.3), фрагмент за облик који одговара хоризонталној.

Седам начела микросервисне архитектуре, примењују се и на фронтенд:

- **Моделовање око пословних домена**, граница микрофронтенда прати домен, а не техничку улогу.
- **Култура аутоматизације**, брз циклус повратне информације претходи усвајању архитектуре.
- **Сакривање имплементационих детаља**, уговор (енгл. *Application Programming Interface - API*) одваја интерфејс од имплементације.
- **Децентрализација**, тим бира технологију и темпо рада за сопствени домен.
- **Независно испоручивање**, сваки микрофронтенд се објављује без чекања на остале.
- **Изолација грешака**, квар једног дела не обара целину, него се сакрива или замењује.
- **Висока уочљивост**, грешка мора моћи да се прати кроз ланац независних делова.

Разлика између микрофронтенда и компоненте није у величини, него у критеријуму поделе: компонента решава технички проблем поновне употребе и излаже интерфејс који контејнер (енгл. *host*) познаје у детаље, док микрофронтенд у потпуности поседује пословни домен и излаже сведен скуп података потребних да схвати контекст.

Микрофронтенд архитектура уводи и изазове које компонентни модел није имао: дељено стање, рутирање, надгледање перформанси и конзистентност изгледа. Отуда и опрез: архитектура није универзално решење, него оправдана само када итеративан развој дугог века трајања, велики развојни тим, вишекорисничка (енгл. *multi-tenant*) апликација или поступна замена постојећег система то захтевају (Mezzalana, Building Micro-Frontends, 2025). Формулација која важи и за овај избор, а не само за избор архитектонског стила: не постоји најбоља архитектура, него најмање лоша комбинација компромиса.

3.1 Предности, недостаци и критеријуми примене

Аутономија коју микрофронтенд архитектура доноси није бесплатна и издвајају се четири последице.

Прва је **сувишност**: сваки тим одржава сопствени процес превођења (енгл. *transpiling*) и испоруке, па се исти проблем понекад решава више пута, критична грешка у библиотеци коју користи више тимова не поправља се на једном месту, него код сваког тима засебно.

Друга је **конзистентност**: када је једном тиму потребан податак који поседује други, репликација уводи кашњење, па подаци краткотрајно могу бити неусаглашени између делова истог погледа, компромис који даје робусност уместо гарантоване конзистентности.

Трећа је **хетерогеност технологија**: слобода избора радног оквира отежава интерну мобилност програмера између тимова и размену праксе.

Четврта је **више фронтенд кода**: изолација која чини сваки фрагмент самосталним нужно уводи извесну сувишност *JavaScript* и *CSS* (енгл. *Cascading Style Sheets*) кода, мада та количина не расте линеарно с бројем тимова.

Ове четири последице одређују и када архитектура има смисла. Своди се на праг тимске величине: правило „тим за две пице“ (енгл. *Two-Pizza Team Rule*), тим који две велике пице не могу да прехране, отприлике десетак људи, постаје кандидат за поделу. Испод тог прага комуникације унутар једног тима није проблем који треба решавати архитектуром, а трошак поставке, заједничка правила о именским просторима (енгл. *namespace*), размена знања међу тимовима, премашује корист. (Geers, 2020)

Микрофронтенд архитектура, осим тога, најбоље функционише на вебу: изворно отвореној платформи на којој се делови независно преузимају и мењају, за разлику од изворних мобилних апликација (енгл. *native mobile applications*), које се објављују као једна целина. Није добар избор ни када домен често и суштински мења облик (пивот пословног модела), јер тада реорганизација граница микрофронтендова уноси више трења него што архитектура штеди. Одсуство комуникационог проблема у малом тиму и честа промена домена су, дакле, два одвојена разлога за исти закључак, да микрофронтенд архитектура овде не одговара.

3.2 Дефинисање ограниченог контекста

Пре него што се одлучи како се микрофронтендови састављају, потребно је одлучити шта један микрофронтенд *јесте*, где почиње и где се завршава пословни домен који носи. Из тог разлога је корисно применити развој заснован на домену (енгл. *domain-driven design, DDD*), иако *DDD* изворно није фронтенд техника.

DDD разликује три врсте поддомена: **кључне** (нпр. за платформу за стриминг (енгл. *streaming*) видеа, каталог садржаја), **подржавајуће** (нпр. систем гласања или оцењивања) и **генеричке** (нпр. пријава и регистрација), према томе колико директно доносе вредност, разлика важна зато што кључни поддомен заслужује искуснији тим и брзу повратну информацију, а генерички се често решава готовим решењем.

Изнад поддомена стоји **ограничени контекст** (енгл. *bounded context*): логичка граница која сакрива имплементацију и излаже само уговор преко којег се домен користи. У новом систему поддомен и ограничени контекст најчешће се поклапају, чиме се вертикална подела (3.3) непосредно мапира на један ограничени контекст по тиму.

Правило препознавања добро постављене границе исто је правило из поглавља 3, изражено обрнуто: ако микрофронтенд захтева више од токена сесије и контекстног идентификатора да би функционисао, граница вероватно није добро постављена, а не да контејнер треба да му пружи више података. Друго правило је временско: границу треба одложити до последњег тренутка развоја када постоји довољно података о томе како корисници стварно користе систем, а не унапред претпоставити поделу (Mezzalana, Building Micro-Frontends, 2025).

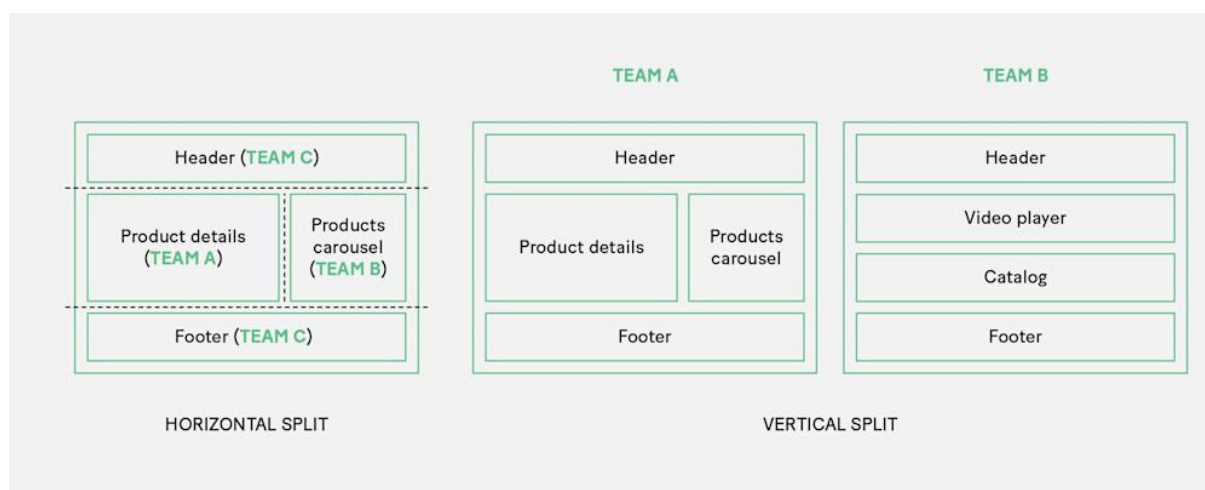
Овај одељак дефинише инструмент; његова примена на конкретан домен, идентификација поддомена и ограничених контекста, предмет је одељка 10.4.

3.3 Простор типова микрофронтенд архитектуре

Прва оса је **време састављања**: статично, у тренутку превођења, или динамично, у тренутку извршавања. Статичан приступ даје бржу апликацију по цени да свака промена микрофронтенда захтева поновно превођење целине; динамичан приступ (пример: *Module Federation*) дозвољава независну испоруку по цени веће крхкости и сложенијег управљања верзијама.

Друга оса је **место састављања**: на серверу или на клијенту. Састављање на серверу даје кориснику готов документ без цене коју плаћа прегледач, али захтева сложену инфраструктуру за скалирање; састављање на клијенту (пример: *single-spa*) даје највећу флексибилност избора технологије, по цени времена потребног прегледачу да састави делове.

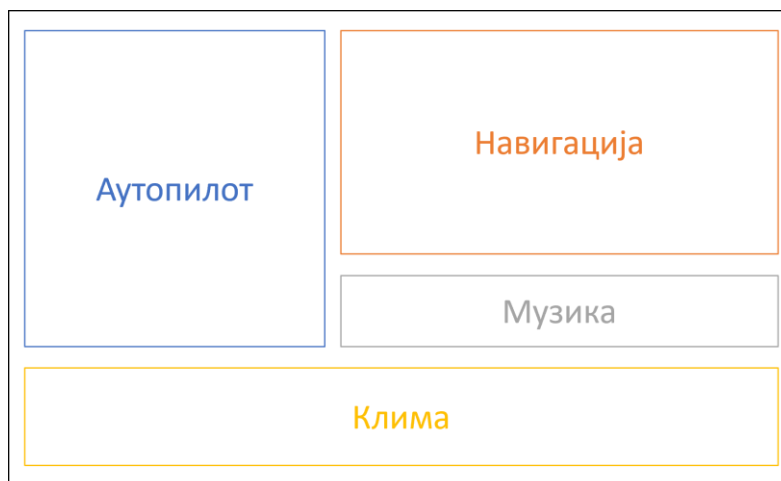
Трећа оса је **тип поделе**: хоризонтална, на екрану се појављује више микрофронтендова, вертикална, један микрофронтенд по екрану. (Mezzalira, Micro-frontends in context, 2020)



Слика 3-1 Хоризонтални наспрам вертикалног типа поделе (Mezzalira, Micro-frontends in context, 2020)

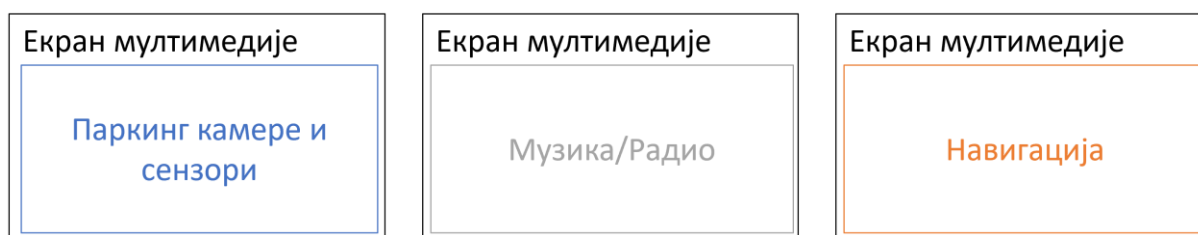
Хоризонтална подела представља стил у којем се на истом погледу/екрану приказују више различитих микрофронтенд апликација. Овим приступом је омогућено да више програмерских тимова утичу на исти екран изменама својих респективних апликација. Овим је и додата флексибилност, узимајући у обзир да се један микрофронтенд може појавити и на више екрана.

Пример хоризонталне поделе је дат кроз мултимедијални интерфејс аутомобила где на почетном екрану можете видети већину функција које систем пружа, свака на свом делу екрана.



Слика 3-2 Пример хоризонталне поделе на мултимедијалном систему аутомобила

Вертикална подела је подела у којој корисник на екрану може да види само један микрофронтенд. Овим је омогућено да један поглед има само једну апликацију, што је практично при подели на пословне домене (унутар *Domain Driven Design* - *DDD*). Пример је исто дат кроз слику мултимедијалног система аутомобила, али овај пут код старијег система где се притиском на дугме на инструмент табли може у једном тренутку отворити само једна функционалност.



Слика 3-3 Пример вертикалне поделе на мултимедијалном систему аутомобила

3.4 Тимске границе и Конвејев закон

Микрофронтенд архитектура претпоставља тимове које треба разграничити, а Конвејев закон (енгл. *Conway's law*) каже да ће структура система копирати структуру комуникације организације која га пројектује (Conway, 1968). За вертикалну поделу то значи: границе тимова треба да прате границе апликација, а не обрнуто.

Три поступка помажу да се граница пронађе: развој заснован на домену (3.2), фокус на кориснику (фазе пута корисника, „шта корисник тражи да уради“) и преглед постојеће структуре екрана. Ниједан није коначан па се саветује да се граница помера тек кад подаци о понашању корисника то потврде, не унапред.

4 Композиција, интеграција и рутирање

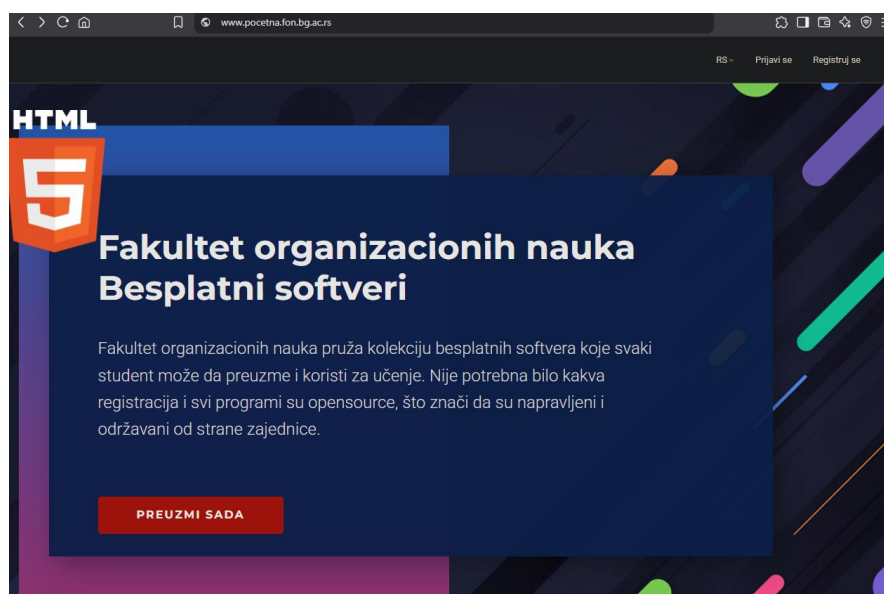
Треће поглавље одговорило је на питање шта микрофронтенд јесте; ово поглавље одговара на друга два питања: како се делови састављају у један екран и како се корисник води између њих. Разликују се три појма која се у пракси мешају, **технику транзиције** (како корисник прелази с једне странице на другу), **технику композиције** (како се више микрофронтендова појављује на истом екрану истовремено) и **архитектуру високог нивоа** (комбинацију прве две с одлуком о месту на коме се приказ гради).

4.1 Транзиција путем линкова и рутирање на страни сервера

Најпростији облик интеграције јесте хиперлинк (енгл. *hyperlink*), а његову вредност мери дефиниција спрегнутости коју (Geers, 2020) уводи за односе међу тимовима: спрегнутост је количина знања коју један тим мора да има о другом. Код линка та количина сведена је на једну адресу, тим који линк поставља не зна ни у ком је радном оквиру циљна апликација написана, ни како она доставља своје датотеке, а прегледач између двеју апликација не успоставља никакву везу, па је техничка изолација потпуна без ичијег труда.

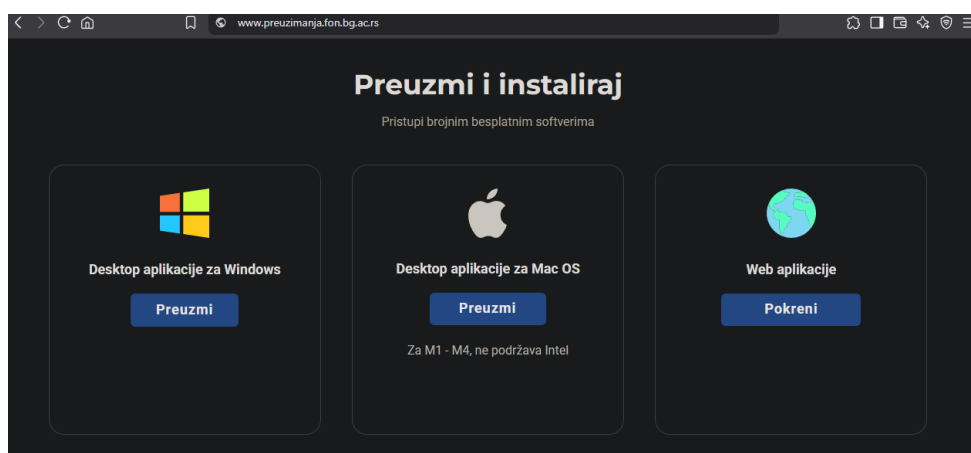
Цена те независности јесте прихваћена сувишност: свака апликација доставља сопствене стилове, чиме се избегава централна датотека која би оба тима спрегла (Geers, 2020). Адреса је при том уговор, а параметрима из адресе (енгл. *query parameters*) постаје богатији и крхкији, јер сваку његову промену морају испоштовати обе стране.

На примеру фиктивног система можемо видети да је прва апликација испоручена на домену *www.pocetna.fon.bg.ac.rs*.



Слика 4-1 Почетна страна фиктивне апликације ФОН-а

Уколико корисник кликне на дугме са ознаком „PREUZMI SADA“, интернет прегледач ће га одвести на следећу страницу, која је под другим доменом (на пример фиктивни домен *www.preuzimanja.fon.bg.ac.rs*). Свака од ових апликација засебно довлачи све потребне материјале, датотеке и учитава их потпуно независно. Самим тим, перформансе оваквог приступа нису сјајне. Такође, и корисник ће видети поновно учитавање странице и овим приступом се губе сви бенефити једностраничних апликација и предности које је донела та модерна архитектура. Међутим, овим је постигнута савршена вертикална подела, која је претходно поменута у микрофронтенд архитектури.



Слика 4-2 Страница за преузимање фиктивне апликације ФОН-а

Код за имплементацију оваквог приступа је изузетно једноставан. Само је потребно над *a* тагом (или било којим тагом који подржава хиперлинкове) поставити *href* атрибут који ће одвести ка другој апликацији.

```
<div class="u-align-left u-container-align-left u-container-style u-custom-color-3 u-expanded-width-lg u-expanded-width-md u-expanded-width-xs u-group u-opacity-95 u-group-1">
  <div class="u-container-layout u-container-layout-1">
    <h1 class="localized u-text u-text-body-alt-color u-title u-text-1">Preuzmi i instaliraj</h1>
    <p class="localized u-custom-font u-font-roboto u-text u-text-grey-10 u-text-2">Pristupi brojnim besplatnim softverima</p>
    <a href="http://www.preuzimanja.fon.bg.ac.rs" class="localized u-btn u-btn-round u-button-style u-palette-1-base u-radius-4 u-btn-1">Preuzmi sada</a>
  </div>
</div>
```

Слика 4-3 Преусмеравање страница путем *a href* тага

Оно што линк не може да сакрије јесте прелаз: свака транзиција је **тврда навигација** (енгл. *hard navigation*), при којој прегледач одбацује документ, стање и преузете датотеке, чиме се губе предности једностраничних апликација описане у 2.3. Оно што постиже бесплатно јесте савршена вертикална подела, сваки део је цела страница у власништву једног тима.

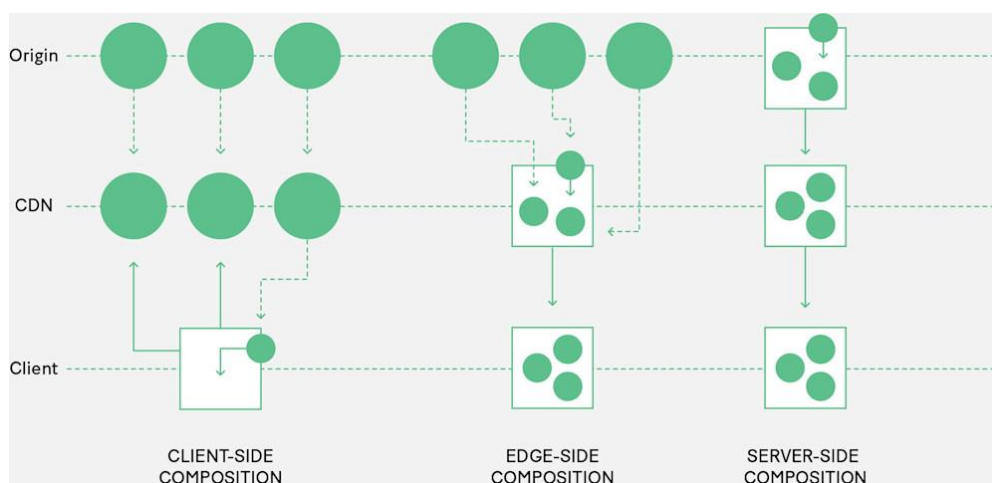
Прва надоградња оваквом систему не би променила технику транзиције, него њену видљивост: да би све апликације биле доступне под истим доменом, уводи се посредник (енгл. *frontend проху*) без пословне логике, чија изведба тимовима или додељује префиксе путања или чита табелу правила коју саме апликације објављују. Један домен решава колачиће, ограничење

дељења ресурса између порекла (енгл. *Cross-Origin Resource Sharing - CORS*) и потребу за више записа унутар Система имена домена (енгл. *Domain Name System - DNS*) и сертификата, уводи место за кеширање и поправља индексирање претраживачима. Оба недостатка следе из истог својства: посредник је нова заједничка инфраструктура коју неко мора да одржава и **једна критична тачка** чији отказ обара све тимове одједном, што се у 4.5 понавља за обмотач на страни клијента.

4.2 Композиција на страни сервера

Претходни одељак довео је више апликација под један домен, али не и на један екран, јер посредник једном захтеву додељује једног власника. **Композиција на страни сервера** уводи корак више: документ се пре него што стигне до прегледача састави од фрагмената које су вратили различити тимови (Geers, 2020).

Најпростија изведба јесу **укључења на страни сервера** (енгл. *Server Side Includes - SSI*), стандардна могућност већине веб сервера: тим власник у свој таг упише директиву укључења, а сервер је у другом кораку замени садржајем довученим са адресе фрагмента. Последица тог двостепеног састављања јесте да време до првог бајта (енгл. *time to first byte - TTFB*) више није својство једне апликације, оно је сума времена потребног да власник врати сопствени таг и времена најспоријег фрагмента у њој. (Geers, 2020). **Укључења на ивици мреже** (енгл. *Edge Side Includes - ESI*) исту идеју премештају у мрежу за испоруку садржаја (енгл. *Content Delivery Network - CDN*), чиме састављање силази ближе кориснику, али стандардног ограничења чекања по фрагменту се не зна, већ они зависе од пружалаца услуге. (Mezzalira, Micro-frontends in context, 2020)



Слика 4-4 Упоредба три типа композиције (Mezzalira, Micro-frontends in context, 2020)

4.3 Композиција на страни клијента

У овом типу композиције клијент учитава више микрофронтендова директно са *CDN* или серверске тачке и тиме је клијентски слој заслужан да покрене приступне тачке сваког од фрагмената, обично кроз JavaScript или HTML датотеке.

Три технике овог одељка леже на једној оси: колико изолације добијају, толико интерактивности плаћају, и обрнуто. Свима је заједничко да састављање обавља прегледач, чиме заузимају клијентску страну и динамично време (3.3).

Елемент `iframe` решава изолацију радикално: у постојећем контексту прегледача отвара сасвим нов контекст, с властитим глобалним објектима, DOM дрветом и учитавањем датотека, па преклапања ни у ком виду нема (Wright, 2022). Изолација је највиша међу техникама овог поглавља, изведба најпростија, а подршка прегледача практично савршена. Цена се, међутим, не исказује само у перформансама: пошто садржај у новом контексту не може да утиче на висину елемента у родитељском документу, родитељ мора висину да зна унапред (Geers, 2020), пре тога тимови су морали да знају само адресу, а сада морају да знају и висину њеног садржаја, чиме се уговор проширио на месту на коме се то не очекује. Уз то, свака инстанца носи цену новог контекста, ограничава распоред, отежава приступачност и не индексира се као део родитељске странице, због чега су оправдани случајеви они у којима изолација јесте захтев, а не последица.

```
<div class="u-layout">
  <div class="u-layout-col">flex
    <div class="u-size-30">flex
      <div class="u-layout-row">flex
        <iframe src="https://www.youtube.com/embed/CPXTXYwWwJM" width="560" height="315" title="YouTube video player"></iframe>flex
      </div>
    </div>
  </div>
  <div class="u-size-30"></div>flex
</div>
```

Слика 4-5 Коришћење `iframe` тага у HTML документу



FON

FON predstavlja idealno mesto za sve one koji žele da se usavršavaju u oblasti naprednih informacionih tehnologija i savremenog poslovnog menadžmenta. Svi studijski programi koji su se do sada realizovali na FON-u imaju jaku kvantitativnu i informatičku podlogu, na koju se nadovezuju znanja iz organizacije, ali i znanja usko vezana za dati studijski program. Pri tome, stručni naziv „diplomirani inženjer organizacionih nauka“ pripada polju tehničko-tehnoloških nauka.

SAZNAJ VIŠE

Слика 4-6 Приказ iframe компоненте на страници

```
<iframe src="https://www.youtube.com/embed/CPXTXVvMwM" width="560" height="315" title="YouTube video player">
  <!DOCTYPE html>
  <html lang="en" dir="ltr" data-cast-api-enabled="true" data-darkreader-mode="dynamic" data-darkreader-scheme="dark" data-darkreader-proxy-injected="true">
    <head>
    </head>
    <body class="date-20250505 en_US ltr site-center-aligned site-as-giant-card webkit webkit-537" dir="ltr">
      <div id="player" style="width: 100%; height: 100%;>
        <div class="html5-video-player ytp-exp-bottom-control-flexbox ytp-modern-caption ytp-livebadge-color ytp-title-enable-channel-logo ytp-fine-scrubbing-exp ytp-embed yt
          -hide-controls" tabindex="-1" id="movie_player" data-version="/s/player/14cfd4c0/player_ias.vflset/en_US/base.js" aria-label="YouTube Video Player">
        </div>
        <script src="/s/player/14cfd4c0/www-embed-player-cc.vflset/www-embed-player-cc.js" name="embed_client" id="base-js" nonce="4eLoR9qIgrFQo3BvhFYLHw"></script>
        <script src="/s/player/14cfd4c0/player_ias.vflset/en_US/base.js" name="player/base" nonce="4eLoR9qIgrFQo3BvhFYLHw"></script>
        <script nonce=writeEmbed();</script>
        <script nonce=if (window.ytcsi) [ytcsi.infoGel({serverTimeMs: 87.0 }, '')];</script>
        <noscript>
        </noscript>
      </body>
    </html>
  </iframe>
```

Слика 4-7 DOM стабло унутар iframe тага

Веб компоненте, чије су три технологије дефинисане у 2.5, ту цену не плаћају, а део изолације задржавају. Као техника композиције њихов допринос није у моделу компоненте, него у томе што уговор о фрагменту постаје декларативан и потпун и сведен на три податка: назив тага, атрибуте којима се фрагменту предаје контекст и адресе датотека које захтева (Geers, 2020). Обавезна цртица у називу елемента по мери служи двоструко, јер стандард њоме одваја елементе по мери од уграђених, а конвенција [тим]-[фрагмент] носи и власништво. Методе животног циклуса постају интеграциони интерфејс, једнак за све тимове без обзира на радни оквир испод, чиме притисак на међу-тимске договоре опада; ограничење је што су синхроне, па фрагменту не дозвољавају да иницијализацију одложи докле жели. Изолација је при том асиметрична, DOM „под сенком“ затвара структуру и стилове, док за *JavaScript* еквивалента нема, што је предмет 8.2. Две оцене претежу: реч је о стандарду веба који се не мења уназад, што за системе дугог века трајања није мала ствар, док недостатак остаје то што фрагмент захтева *JavaScript* на клијенту, пошто стандардног декларативног облика приказивања на серверу нема (Geers, 2020).

```
class MyElement extends HTMLElement {
  static get observedAttributes() {
    return ['moj-atribut']; // Specificiranje atributa koji će biti praćeni
  }

  constructor() {
    super();
    // Inicijalizacija elementa
  }

  connectedCallback() {
    // Kod koji se izvršava kada je element dodan u DOM
  }

  disconnectedCallback() {
    // Kod koji se izvršava kada je element uklonjen iz DOM-a
  }

  attributeChangedCallback(name: string, oldValue: string | null, newValue:
string | null) {
    if (name === 'moj-atribut') {
      // Kod koji se izvršava kada se atribut "moj-atribut" promeni
    }
  }
}

customElements.define('my-element', MyElement);
```

```

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Fon Konkurs</title>
  </head>

  <body>
    <fon-button label="Upisi se na FON">
      <span slot="icon">★</span>
    </fon-button>

    <script>
      class FonButton extends HTMLElement {
        constructor() {
          super();

          this.attachShadow({ mode: "open" });

          this.shadowRoot.innerHTML = `
            <button>
              <slot name="icon"></slot>
              <span id="label"></span>
            </button>
          `;
        }

        static get observedAttributes() {
          return ["label"];
        }

        attributeChangedCallback(name, oldValue, newValue) {
          if (name === "label") {
            this.shadowRoot.getElementById("label").textContent = newValue;
          }
        }

        connectedCallback() {
          this.clickHandler = () => {
            this.dispatchEvent(
              new CustomEvent("fon-click", {
                detail: { message: "Upisali ste se na FON!" },
                bubbles: true,
                composed: true
              })
            );
          };
        }
      }
    </script>
  </body>
</html>

```

```

    };

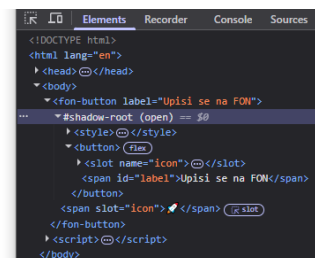
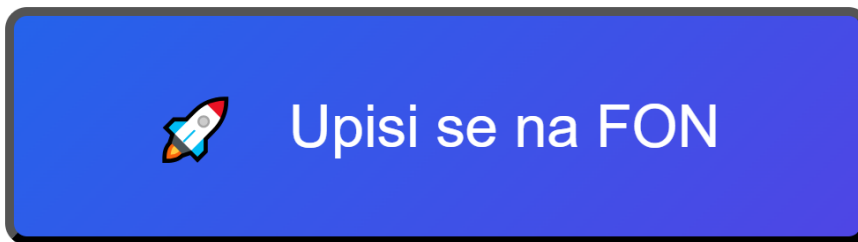
    this.shadowRoot
      .querySelector("button")
      .addEventListener("click", this.clickHandler);
  }

  disconnectedCallback() {
    this.shadowRoot
      .querySelector("button")
      .removeEventListener("click", this.clickHandler);
  }
}

customElements.define("fon-button", FonButton);

document
  .querySelector("fon-button")
  .addEventListener("fon-click", (e) => {
    alert(e.detail.message);
  });
</script>
</body>
</html>

```



Слика 4-8 Прилагођено дугме унутар DOM "под сенком"

single-spa исту идеју подиже с разине фрагмента на разину целе апликације: не саставља делове једног екрана, него више једностраничних апликација у једну. Уговор је овде *JavaScript* објекат с три асинхроне функције, *bootstrap*, *mount* и *unmount*, а обмотач сваку апликацију пријављује позивом *registerApplication*, којем предаје назив, функцију за учитавање кода и функцију активације која из адресе одлучује да ли апликација треба да буде приказана (Geers, 2020). Три својства имају последице за 4.5: учитавање кода је по потреби, а не унапред; свака апликација добија сопствени DOM елемент у који се приказује; и **више апликација може бити активно истовремено**, што је оно што трајно присутно заглавље или навигациони микрофронтенд чини изводљивим. Асинхроне функције животног циклуса предност су у односу на синхроне методе Веб компоненти, али и обавеза: пошто све апликације деле један документ, заборављени ослушкивачи догађаја, тајмери и глобалне променљиве постају туђи проблем.

```

export async function bootstrap(props) {...}
export async function mount(props) {...}
export async function unmount(props) {...}

singleSpa.registerApplication({
  name: "app1",
  activeWhen,
  app,
  customProps: (name, location) => { authToken: "d83jD63UdZ6RS6f70D0" }
});

singleSpa.start();

```

4.4 Module Federation и дељење зависности

DOM „под сенком“ стилове изолује; за *JavaScript* еквивалента нема. Узимајући у обзир да је *JavaScript* у једном документу подразумевано дељен, може се закључити да дељење зависности престаје да буде одлука коју је могуће одложити: библиотеке ће бити или одвојене или дељене, а оба исхода имају цену. (Rappi, The Art of Micro Frontends, 2024)

Разликују се три модела:

- У моделу **без дељења** сваки микрофронтенд у свој пакет укључује све што му треба, што даје највећу независност уз највећи пренос, исти компромис прихваћене сувишности из 4.1.
- У моделу **централног дељења** библиотеке обезбеђује обмотач, а микрофронтендови постају мали; цена је озбиљнија него што изгледа, јер тим који одржава обмотач, па самим тиме преузима техничке одлуке свих микрофронтендова, а модел се лоше скалира с њиховим бројем.
- У моделу **распоређеног дељења** сваки микрофронтенд носи сопствену копију, али се у времену извршавања договара с осталима и користи ону која је већ учитана (Rappi, The Art of Micro Frontends, 2024).

Module Federation јесте изведба трећег модела. Подешава се на обема странама: микрофронтенд објављује назив контејнера и списак модула које излаже, обмотач наводи контејнере које читава, а обе стране наводе библиотеке које нуде за дељење (Rappi, The Art of Micro Frontends, 2024). Тако објављени називи модула, имају природу уговора и не смеју се преименовати без претходног усклађивања међу тимовима, исти облик тихог отказа као код адресе у 4.1, само померен из мреже у фазу градње.

Пошто се контејнери могу угњеждавати, па и микрофронтенд може бити обмотач другима, граница између обмотача и читаног дела постаје танка, због чега се препоручује да дељење

остане **једносмерно**: обмотач учитаном делу не излаже сопствене модуле, јер двосмерно дељење ствара кружну зависност у којој промена у једном микрофронтенду може да поквари други, чиме се укида аутономија због које је подела и уведена, исти појам спрегнутости из 2.5, изражен над зависностима. Стандардизована алтернатива постоји и заснива се на ECMAScript модулима и мапама увоза (енгл. *import map*) уместо на механизму једног састављача.

```
/root-config
├─ src/
│   ├─ index.ejs          # HTML entry
│   ├─ root-config.js     # Single-SPA configuration
│   └─ import-map.json    # Shared dependency map
└─ package.json

{
  "imports": {
    "@mfe/navbar": "http://localhost:8500/navbar.js",
    "@mfe/dashboard": "http://localhost:8600/dashboard.js",
    "react":
      "https://cdn.jsdelivr.net/npm/react@18.2.0/umd/react.production.min.js",
    "react-dom": "https://cdn.jsdelivr.net/npm/react-dom@18.2.0/umd/react-dom.production.min.js"
  }
}
<script type="importmap">
{
  "imports": {
    "lodash": "https://cdn.jsdelivr.net/npm/lodash@4.17.21/lodash.min.js"
  }
}
</script>
```

Механизам, међутим, не одговара на питање *шта* треба делити, а пракса сугерише на подразумевану одлуку да се не дели ништа. Образложење је асиметрија последица: погрешно не-дељење боли мање од погрешног дељења. Од те основе одступа се само ако су задовољени критеријуми:

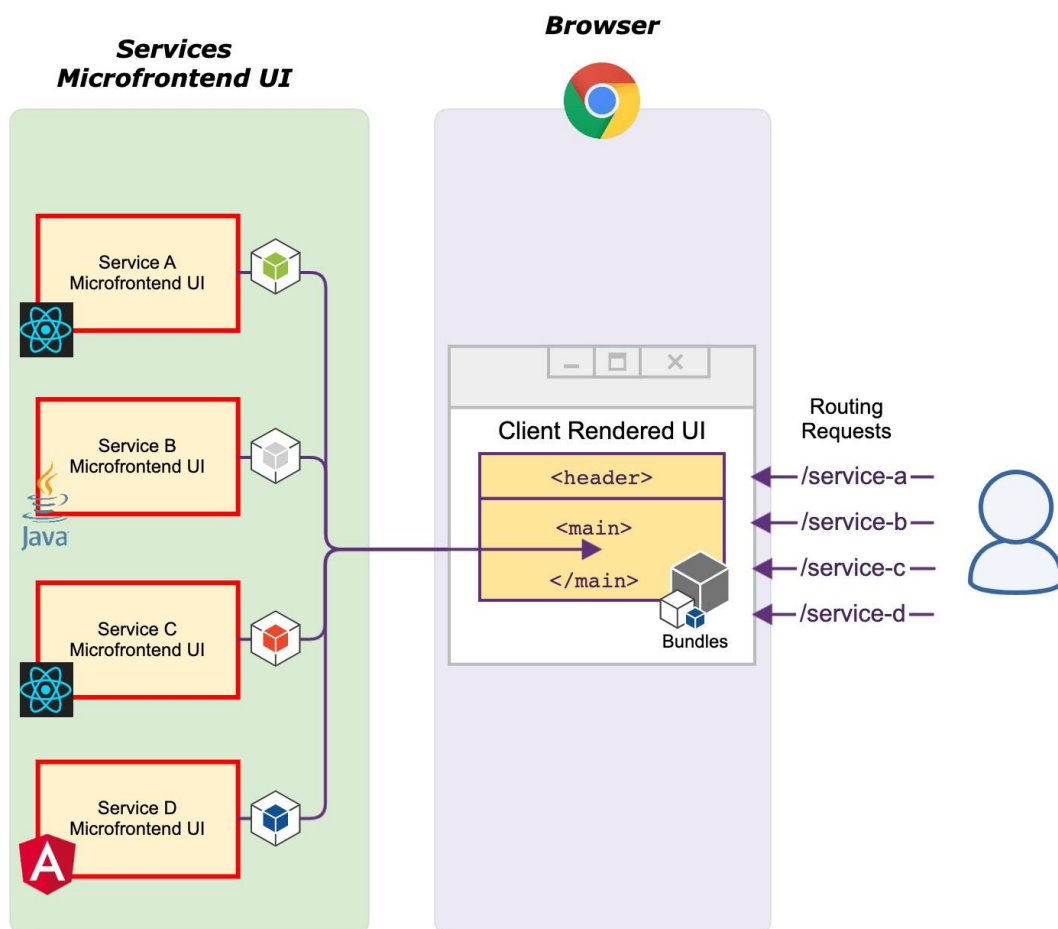
- зависност је **знатне величине**, оквирно најмање 15 до 30 kB, при чему меру треба одмерити према брзини преноса којом располаже просечан циљни корисник;
- користе је **најмање два** микрофронтенда;
- **не постоји више њених верзија у широкој употреби** које међу собом уводе неспојиве промене;

- има **важну улогу у приказивању** компонената;
- **техничка је, а не доменска.**

Последњи критеријум се означава као споран: потреба да се дели доменски модул показатељ је недовољне доменске поделе. Пословни код не треба делити, за помоћне функције препоручује се одвајање, а ако дељење ипак мора да се практикује, онда би га требало издвојити као верзионисан пакет у времену градње, не као зависност у времену извршавања.

4.5 Апликација обмотач и рутирање на страни клијента

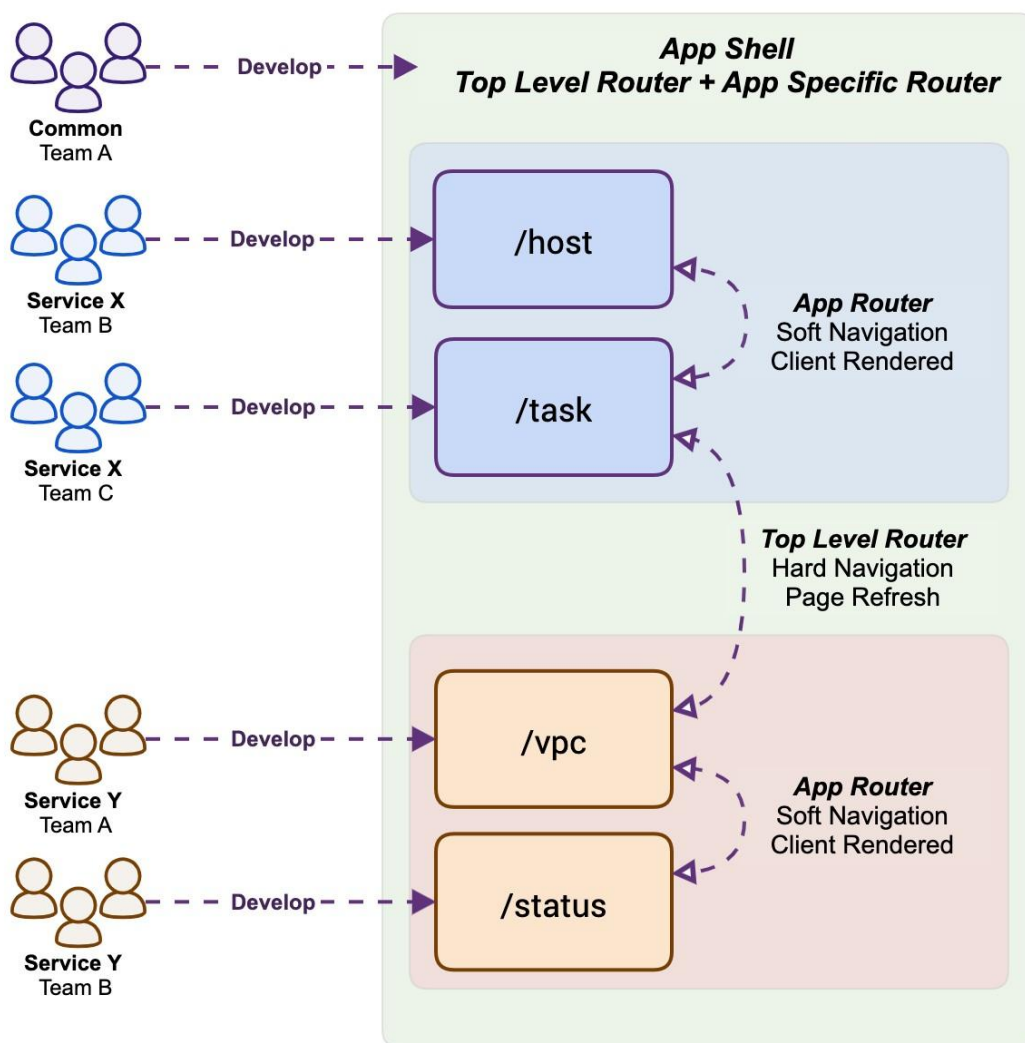
Апликација обмотач (енгл. *application shell*) јесте родитељска апликација свих микрофронтендова: сваки захтев долази њој, она бира део који корисник тражи и приказује га у телу документа (Geers, 2020) (Chen, 2021). Пошто је реч о дељеном коду, треба га држати што простијим и без пословне логике. Обмотач има четири суштинска посла: да обезбеди дељени HTML документ, да адресе преведе у странице тимова, да одговарајућу страницу прикаже и да при навигацији деиницијализује претходну и иницијализује следећу.



Слика 4-9 Апликација обмотач повлачи микрофронтендове на основу захтева (Chen, 2021)

Рутирање на једном нивоу јесте најпростија изведба: обмотач држи једну табелу која свакој адреси додељује компоненту одговорног тима, а општим ослушкивањем клика на документу преузима линкове и нову адресу уписује у историју прегледача. Недостатак је структурни: обмотач мора да зна све адресе апликације, па тим који жели да измени постојећу или дода нову мора да измени и поново испоручи обмотач, спрегнутост је тиме постављена на најгоре место, зато што обмотач треба да буде што неутралнији са аспекта инфраструктуре (Geers, 2020) (Chen, 2021).

Рутирање на два нивоа тај недостатак укида: обмотач из адресе чита само префикс и одлучује који је тим одговоран, а рутер самог тима из целе адресе одређује коју страницу приказује користећи **меку навигацију** (енгл. *soft navigation*). Добија се више једностраничних апликација обједињених у једну, при чему се обмотач мења само ако се уводи нов тим или ако се мења префикс постојећег (Geers, 2020).



Слика 4-10 Рутирање на једном и два нивоа (Chen, 2021)

Рутирање на два нивоа где обмотач разрешава само префикс:

```
const routes = {
  "/product/": "decide-pages",
  "/checkout/": "checkout-pages",
  "/": "inspire-pages",
};

function findComponentName(pathname: string): string | undefined {
  const prefix = Object.keys(routes).find((key) => pathname.startsWith(key));
  return routes[prefix];
}
```

Уговор између обмотача и тимова тиме добија и нормативно правило: пословна логика припада коду тимова, а поуздан показатељ да је спрегнутост претесна јесте то да испорука једне функционалности од стране једног тима захтева измену обмотача (Geers, 2020). Најизраженији облик тог правила јесте да **стање не треба премештати у обмотач**: дељени садржалац стања раван је дељеној бази података, а то је управо спрегнутост због које је дистрибуирани стил и уведен (2.2). Механизми који то решавају без дељеног стања предмет су 5. поглавља.

5 Комуникација и управљање стањем

Како остварити размену међу микрофронтендовима без чврсте повезаности је главна тематика при пројектовању дистрибуираног система. При добро постављеним границама тимова потреба за обимном разменом у прегледачу треба да буде мала, пошто корисник задатак довршава у интерфејсу једног тима, а размена је потребна углавном на местима предаје, и да се већина те размене међу тимовима одвија преко адресе. Обим комуникације је, отуда, мера квалитета граница: када нова функционалност захтева да два тима податке шаљу један другом напред и назад, то је поуздан показатељ да границе нису добро постављене, а исправка није бољи канал него померање одговорности или проширење обима једног од домена (Geers, 2020).

Овде се механизми уређују по две осе: по **топологији учесника** (однос садржавања, однос суседства, или без икаквог односа) и по **услову истовремености**, да ли су обе стране живе у истом документу у истом тренутку. Поглавље се завршава Табелом 5-1, која те осе спаја у поступак одлучивања.

5.1 Три смера: родитељ→фрагмент, фрагмент→родитељ, фрагмент↔фрагмент

Када више микрофронтендова дели један екран, разликују се три смера размене, и сваки има свој природни механизам.

Родитељ → фрагмент. Родитељска страница шаље податак фрагменту који је прима као **атрибут**. Фрагмент реагује тако што унапред региструје које атрибуте посматра и обрађује сваку њихову промену. Ток је једносмеран и одговара обрасцу који је познат под називом „својства надоле, догађаји нагоре“ (енгл. *props down, events up*): измењено стање родитеља силази атрибутима, а обавештење се враћа догађајем.



Слика 5-1 Комуникација родитеља и фрагмента

Фрагмент → родитељ. Овде се не проучава механизам, него забрана која му претходи. Тим који поседује фрагмент технички има приступ читавом документу и може сам да измени део странице који му не припада, и управо то не сме. Чист облик јесте уговор: фрагмент објављује **прилагођени догађај** (енгл. *Custom Event*), а власник странице на њега реагује у свом делу.

Фрагмент ↔ фрагмент. Кад два фрагмента нису у односу садржавања, тј. није један у оквиру другог него су на истом нивоу хијерархије, онда постоје три начина комуникације.

- Први, да фрагмент сам пронађе други у документу и **директно** позове његову функцију, и изричито се одбацује: непосредно ослањање на туђи елемент уводи чврсту спрегнутост, композиција екрана се више не може мењати, а уклањање фрагмента даје последице које нико није предвидео.
- Други је **посредовање преко родитеља**, спајањем првих двају смера: решење је чисто и ток размене је изричито моделован, али сваку промену у комуникацији тада морају да имплементирају два тима.
- Трећи је **магистрала догађаја**, где фрагменти објављују на општи канал а претплатници реагују, па родитељ не мора ни да зна да размена постоји, што спрегнутост смањује на најмању меру и што је предмет одељка 5.2.

5.2 Магистрала догађаја: прилагођени догађаји и *Broadcast Channel API*

Магистрала догађаја у прегледачу не захтева никакву библиотеку. **Прилагођени догађај** је интерфејс који се користи како би се прикачили прилагођени подаци у догађај који генерише апликација и подржани су од стране свих прегледача. Догађај се објављује **на елементу**, а не непосредно на глобалном објекту прозора, из два разлога: сачувано је порекло догађаја, што је потребно кад на екрану постоји више примерака истог фрагмента. Затим одређени елемент се региструје и ослушкује изабране догађаје. Елемент који објављује и који ослушкује догађај могу бити у различитим фрагментима, чиме се омогућава комуникација између микрофронтендова у хоризонталној подели. (Mezzalira, Micro-frontends in context, 2020)

```
// креирање прилагођених догађаја
const pronadjenaMacka = new CustomEvent("pronadjenaZivotinja", {
  detail: { name: "macka" },
});
const pronadjenaPas = new CustomEvent("pronadjenaZivotinja", {
  detail: { name: "pas" },
});

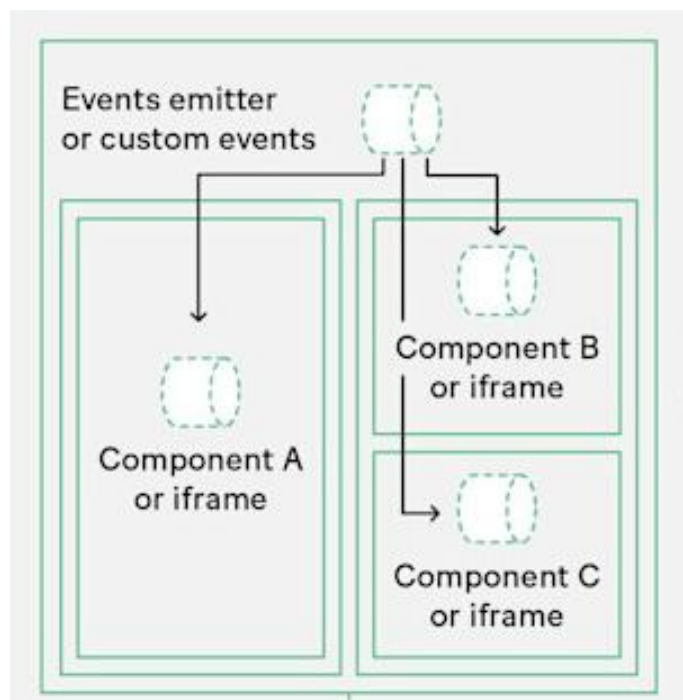
const element = document.createElement("div"); // креирање елемента
```

```

element.addEventListener( // регистрација слушатеља догађаја
"pronadjenaZivotinja",
(e) => console.log(e.detail.name)
);

// емитовање догађаја
element.dispatchEvent(pronadjenaMacka);
element.dispatchEvent(pronadjenaPas);
// "macka" и "pas" се исписују

```



Слика 5-2 Комуникација фрагмената уз помоћ догађаја (Mezzalira, Micro-frontends in context, 2020)

Такав механизам се оцењује као најважнији образац комуникације међу микрофронтендовима, и због отпорности: догађаји су слабо спрегнути по својој природи и не ломе се ако руковалац није регистрован или ако онај који их обично објављује није ту, а самим обликом поручују да им испорука није временски загарантована. Најпоузданији и најбржи начин да се уведу јесте управо стандардни механизам DOM-а.

Размена структура података путем догађаја уводи додатну спрегнутост, носивост треба држати на најмањој мери, а догађај треба користити за обавештење а не за пренос података; модели и доменски објекти остају унутар границе једног тима. Треба размењивати само најмању количину потребну за интеграцију. Практична последица за пројектовање уговора јесте да догађај носи идентификатор, а прималац сам дохвати оно што му треба од свог серверског слоја (5.6).

Broadcast Channel API је други стандардизован облик објављивања и претплате, али са другачијим досегом: канал се отвара по имену, поруке се шаљу и примају методама тог канала, а досеже **друге документе истог порекла** (Geers, 2020) (Nwamba, 2024). То је његова одлучујућа предност над догађајима DOM-а, који не излазе из свог документа, и разлог зашто се он не бира између двају фрагмената на истом екрану, него онда када исти корисник има отворено више картица. Пожељна је употреба именованих канала као границе: општи канал за размену међу тимовима и посебан канал за унутрашњу размену једног тима, на коме је допуштена и богатија структура, раздвајање које смањује ризик од нехотичне спрегнутости.

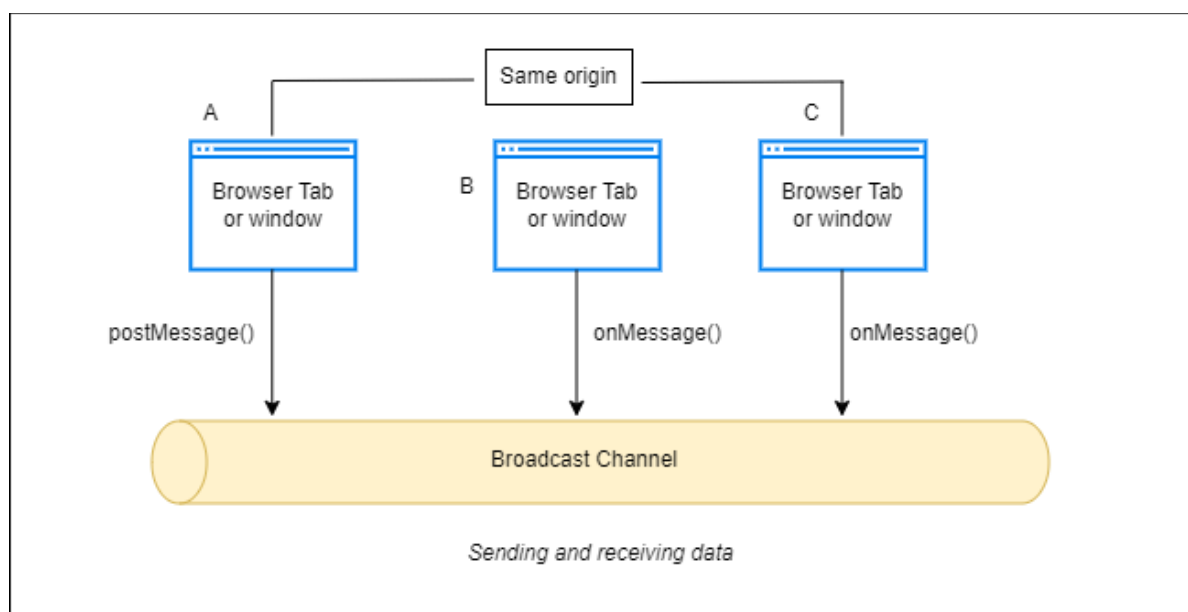
```
const channel = new BroadcastChannel('auth'); // креирање канала

channel.postMessage({ // слање поруке
  type: 'AUTH_UPDATE',
  payload: { user: { name: 'Veljko Blagojević' }, isAuthenticated: true },
});

channel.addEventListener( // регистрација слушатеља догађаја
  'message',
  event => console.log(event.data) // шта год је прослеђено у postMessage
);

channel.addEventListener(// регистрација слушатеља догађаја
  'messageerror',
  event => console.error('Greska pri deserijalizaciji poruke:', event)
);

channel.close(); // затварање канала
```

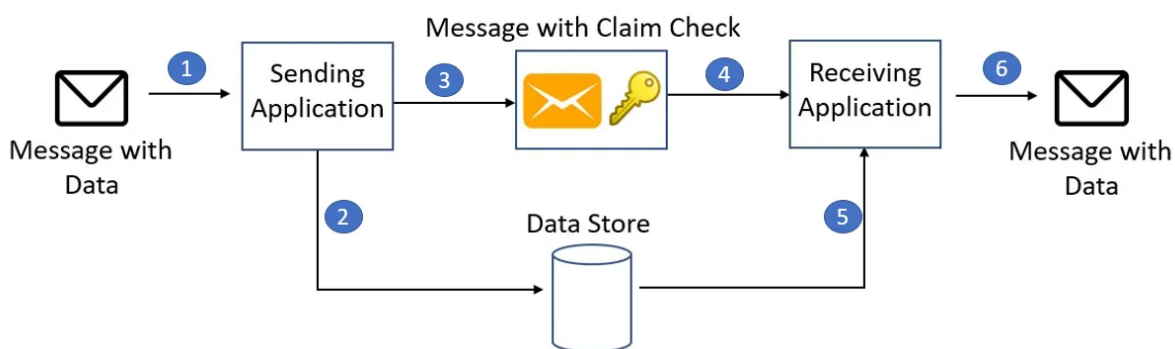


Слика 5-3 Broadcast Channel

5.3 Пренос преко адресе

Уводни одељак поглавља већ је најавио да се већина размене међу тимовима одвија преко адресе, и то на местима предаје. Тај механизам се од свих досад изложених издваја по услову истовремености. Атрибути елемента, прилагођени догађај који се издиже и магистрала догађаја претпостављају да су обе стране монтиране у истом документу у истом тренутку, а Broadcast Channel API да су оба документа отворена. Постоји, међутим, предаја при којој ниједна од тих претпоставки не важи: прелаз преко границе монтирања, где онај који предаје податак напушта екран пре него што онај који га прима на њега дође. Тада преостаје адреса (енгл. *Uniform Resource Locator* - *URL*), тј. податак смештен у путању или у параметре из адресе. Њено одлучујуће својство јесте то да не захтева истовременост, јер га прималац може читати кад год се сам монтира.

Правило које одељак 4.2 већ поставља за догађаје, да носивост буде идентификатор а прималац сам дохвати оно што му треба, за адресу важи исто, и то из два независна разлога. Први је трајање: пошто адреса надживљава размену, све што је у њу уписано остаје у историји прегледача, па у адресу не сме ништа ући што је захтева поверљивост. Други је облик: адреса је ограничене дужине и цела је видљива кориснику, што из ње искључује обимне и сложене објекте као преносне податке. Оно што остаје јесте идентификатор доменског објекта, никада сам објекат. Приступ где се шаље само једноставан идентификатор, а онда се од примаоца очекује да на основу тог јединственог податка извуче велики објекат је варијација Claim Check патерна (Mosyan, 2023).



Слика 5-4 Claim Check патерн (Mosyan, 2023)

5.4 Глобални контекст и аутентификација

Део података није ни порука ни доменско стање, него **контекст**: језик и држава корисника, изабрана валута, статус пријаве, окружење у коме систем ради (Geers, 2020). Издваја се зато што има три својства која га одвајају од свега досад изложеног, потребан је свима, по природи је намењен само читању, и представља инфраструктурни посао који треба решити једном за све тимове а не понављати у сваком фрагменту.

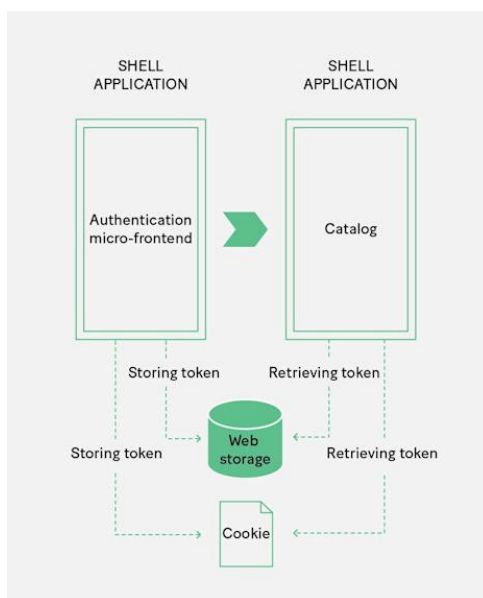
Аутентификација је тежи случај зато што није само инфраструктурна: она носи и пословну логику, па питање није техничко него питање граница. (Geers, 2020) одговор изводи из мисија тимова, тим у чијем домену се пријава природно налази постаје носилац аутентификације, објављује страницу или фрагмент за пријаву на који остали упућују корисника, а статус пријаве се затим преноси успостављеним механизмима попут OAuth-a и JWT-a.

Стање је, најзад, оно што се **не** дели. Поставља се тако да свој тим држи своје стање, јер га дељење чврсто спреже, чини систем осетљивијим и, што је важније, редовно бива злоупотребљено као канал комуникације, тако што један део упише вредност у нади да ће други приметити промену.

5.5 Локално складиште и колачићи

Критеријум по коме се овај одељак одваја од претходних јесте време: реч је о ономе што мора да преживи документ.

Веб складиште (енгл. *Web Storage*) разликује два рока трајања: локално складиште надживљава затварање прегледача, док складиште сесије (енгл. *Session Storage*) траје колико и картица у којој је настало. Колачићи (енгл. *Cookies*) су старији механизам са другачијом ценом: шаљу се уз сваки захтев ка свом пореклу, што их и чини природним носиоцем контекста кад се страница саставља на серверу, али и оптерећује сваки захтев.



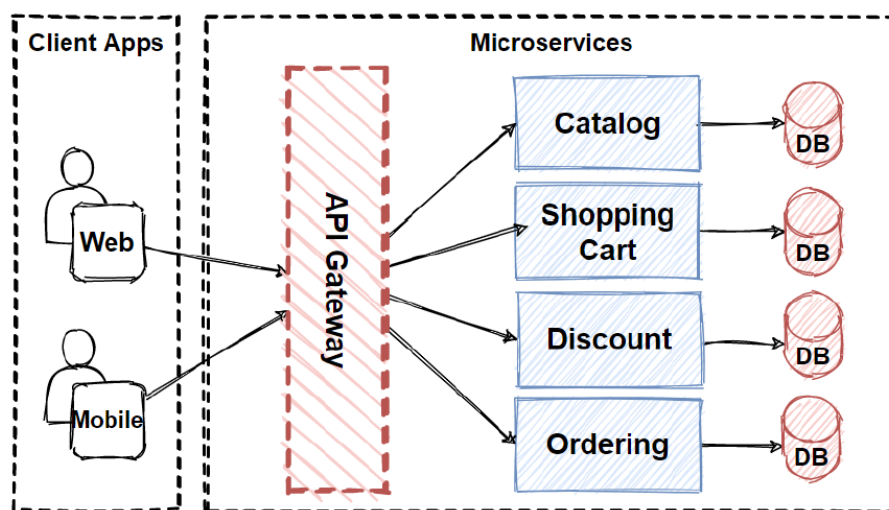
Слика 5-5 Веб складиште и колачићи

Дисциплина која из свега следи јесте да је складиште **медијум, а не канал**: упис вредности никог не обавештава. Онај који хоће да други фрагмент реагује мора и даље да објави догађај, па складиште и магистрала нису замена један за другог него се допуњују, једно носи трајање, друго обавештење.

5.6 Комуникација са серверским слојем

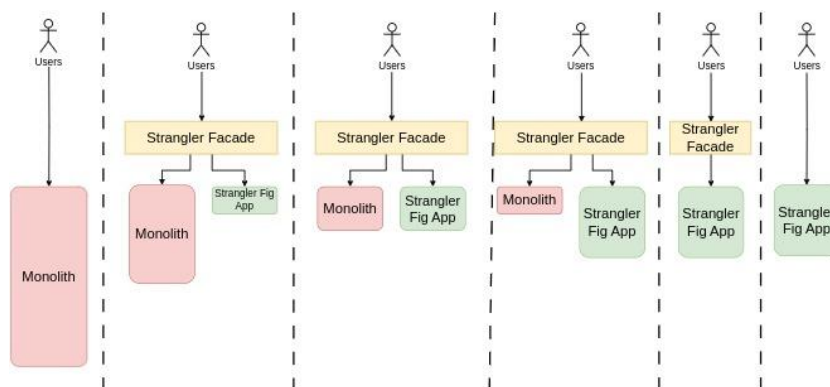
Постоје неколико приступа како ће се изложити API приступне тачке са серверске стране како би клијент могао да их прочита. У дистрибуираном систему и микрофронтенд архитектури најчешће се циља на коришћењу једног од наредна три патерна:

API капија (енгл. *API gateway*) уводи једну улазну тачку испред већег броја сервиса и на њој обавља усмеравање, проверу и састављање одговора, али и оно што је њена права вредност: тзв. ивичне функције, проверу токена, ауторизацију, ограничавање броја позива, међускладиштење, мерење и бележење, на једном месту, уместо да их сваки сервис понавља (Mezzalira, Building Micro-Frontends, 2025). Капија при том може да сакрије и унутрашњи протокол, па клијент остаје на уговорима а не на технологији сервиса. Мане су: једна тачка отказа, због чега капија мора да буде покренута у више примерака; уско грло у управљању, кад свака измена уговора чека тим који поседује капију; и додатни корак у путу захтева, дакле нешто дуже време одзива (Ozkaya, 2021).

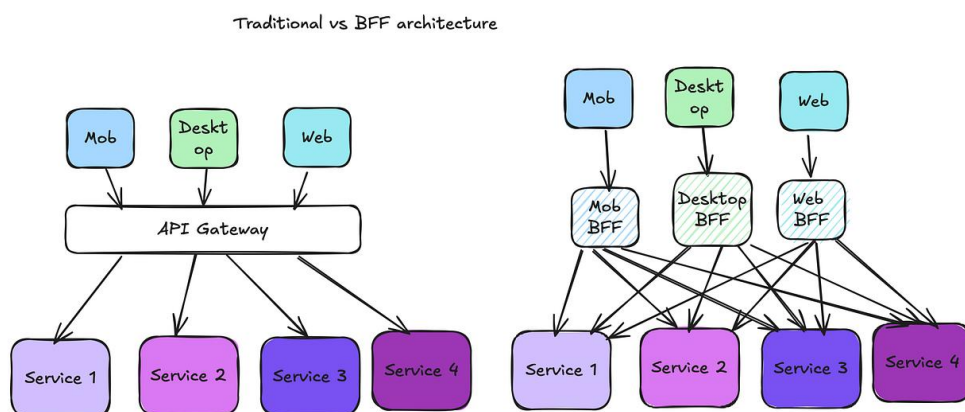


Слика 5-6 API капија у микросервисној архитектури (Ozkaya, 2021)

Backend for Frontend (BFF), продужава капију тако што уводи по једну улазну тачку **по врсти клијента** (Mezzalira, Building Micro-Frontends, 2025). Добитак је двострук: одговор се саставља од више унутрашњих позива и враћа у облику који одговара приказу, чиме се смањује и број обраћања и количина неупотребљених података, а различите платформе добијају оно што им заиста треба уместо најмањег заједничког именитеља (Das, 2024). BFF је уз то и природно средиште поступне замене монолита по обрасцу смокве давителке (енгл. *strangler fig*), пошто на једном месту преусмерава позиве на нове сервисе како они настају, а може се делити и по поддомену уместо по врсти клијента, што допушта различито скалирање делова (Villavicencio, 2023).

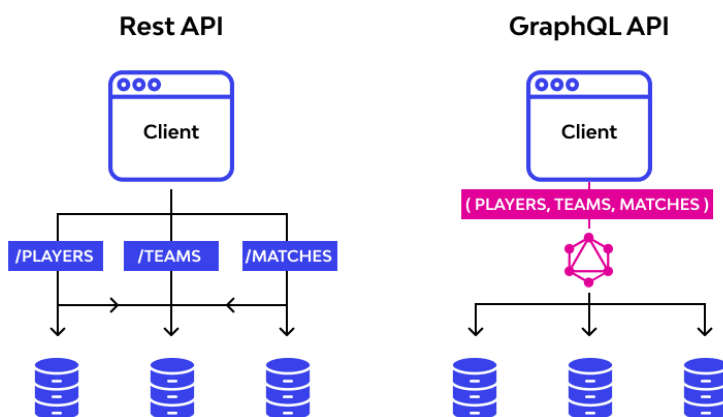


Слика 5-7 Миграција монолита уз помоћ обрасца смокве давителјке (Villavicencio, 2023)



Слика 5-8 Поређење API капије и BFF обрасца (Das, 2024)

GraphQL, објављен 2015. године, јесте језик упита с извршним окружењем на серверу над типизираним шемом, и уз микрофронтендове се помиње због једне особине: шему треба обликовати **из угла приказа**, а не из угла ресурса позадине, што је обрнут смер од уобичајеног пројектовања REST API-ја и што одговара фрагментима с местимичним и различитим потребама за подацима (Mukhadin, 2026). Он има једну улазну тачку, обично се не издаје у верзијама и намерно се не дели по доменама; тимску независност уместо тога чува **федерација шема**, којом се шеме тимова састављају у један граф (Mezzalira, Building Micro-Frontends, 2025).



Слика 5-9 REST API наспрам GraphQL API (Mukhadin, 2026)

5.7 Репликација података међу тимовима

Последњи облик размене не одвија се у прегледачу и зато се лако може пропустити, а он је онај који одлучује о томе да ли је аутономија тимова стварна. Претходни одељак претпоставља правило: микрофронтенд разговара **само са серверским слојем свог тима**, никада с крајњом тачком другог тима. Образложење није стилско. Позив ка туђем API-ју уводи спрегнутост, ствара зависност међу тимовима у времену извршавања и, што је најскупље, укида изолованост, пошто се властити део више не може ни покренути ни тестирати без туђег система. Решење овог проблема је **репликација** потребног подскупа, власник објављује ток измена, а потрошач одржава сопствену копију само оних поља која му требају и освежава је у позадини.

Табела 5-1. Механизми размене међу микрофронтендовима по топологији и услову доступности

Механизам	Топологија учесника	Услов доступности	Носилац уговора	Пример податка који се препоручује да путује
Атрибути елемента	родитељ → фрагмент (садржавање)	исти документ, фрагмент прикачен	назив ознаке и списак атрибута	вредности контекста
Издизање прилагођеног догађаја	фрагмент → родитељ (садржавање)	исти документ	назив догађаја	обавештење о радњи корисника
Магистрала догађаја	суседни фрагменти, без садржавања	исти документ, оба монтирана истовремено	назив догађаја и облик носивости	обавештење с идентификатором
Broadcast Channel API	документи истог порекла (картице)	оба документа отворена	назив канала и облик поруке	промена сесије или заједничког стања
Адреса (путања и параметри из адресе)	прелаз преко границе монтирања	не захтева истовременост	облик адресе	идентификатор доменског објекта
Веб складиште и колачићи	исто порекло, кроз време	не захтева истовременост; не обавештава	назив кључа и облик записа	контекст који мора да преживи документ
Дељено стање с власником	суседни фрагменти	исти документ	одвојени интерфејси за читање и писање	најмањи скуп потребан за позив API-ја
API капија, BFF	клијент ↔ серверски слој	први захтев при читавању	облик речника и уговори API-ја	адресе крајњих тачака и подаци домена
Репликација преко тока измена	серверски слој ↔ серверски слој	асинхроно, у позадини	облик тока измена	подскуп матичних података

Читана одозго надоле, табела показује и цену: што је механизам нижи, самим тиме је и већи домашај, али се плаћа или уговором који мора да испоштује више страна, или зависношћу коју прегледач не пружа сам. Прва два реда захтевају сагласност двају тимова о једном називу; средњи редови додају облик носивости; последња три захтевају и договор о инфраструктури. Отуда правило којим се 5. поглавље и завршава: механизам се не бира по томе шта је изражајније, него по томе шта је **најмање што задовољава топологију** и услов доступности датог случаја.

6 Кориснички интерфејс и систем дизајна

Претходна поглавља показала су како се микрофронтендови дефинишу, како се састављају, како се рутирају и како комуницирају. Сва четири одговора тичу се **односа међу деловима система**, а доследност приказа је својство које се не види ни у једном таквом односу, види се на екрану, и то тек када се два фрагмента нађу један уз други. Ако су границе добро постављене, састављање изведено у прегледачу, рутирање у власништву обмотача и комуникација сведена на најмању меру, интерфејс тако добијеног система још увек може да изгледа као да су га саставила четири различита произвођача, и по правилу тако и изгледа, зато што свака од тих четири исправних одлука повећава самосталност делова, а самосталност је управо оно што доследност разграђује.

Доследност корисничког интерфејса међу микрофронтендовима имплементираним у различитим радним оквирима, и по којој цени. Питање има два дела и она се разликују по томе шта се на њих може одговорити. Први део, да ли је могуће, има одговор у литератури и он је потврдан, уз услове; тај део је предмет овог поглавља. Други део, по којој цени, захтева измерен артефакт, па се на њега одговара у 12.8.

Излагање прати једну нит. Прво се показује да је систем дизајна једино место у овој архитектури на коме се самосталност тимова **намерно ограничава**.

6.1 Систем дизајна наспрам библиотеке компонената

Систем дизајна се одређује као заједнички визуелни језик организације, шири од библиотеке компонената: он обухвата принципе, обрасце, правила и терминологију, а библиотека компонената је његова изведена, извршна форма. Разлика није терминолошка педантерија, него има последицу за све што следи.

Препознатљивост брэнда, доследно понашање према кориснику, брзина рада тимова који не измишљају исто дугме поново, и лакша сарадња дизајнера и програмера. Прве две су **својства производа** и виде се на екрану, па се на артефакту могу проверити. Друге две су **својства организације** с више тимова.

6.2 Изузетак од начела о дељењу

Начело ове архитектуре је **уклањање заједничког**: заједничко стање је анти-патерн (9.2), дељена зависност је трошак који се плаћа и који се тешко враћа (4.4), непосредно увожење туђег кода кроз границу је укидање границе (4.3), а подразумевани одговор на питање шта делити је, ништа (Rappl, The Art of Micro Frontends, 2024). Систем дизајна је једино место где

се на исти проблем одговара **додавањем** заједничког. То је, дакле, једини намеран изузетак од начела које носи целу архитектуру.

Образложење је могуће показати а не само изнети, зато што га четири независна извора изводе на четири различита начина и долазе до истог финалног закључка. (Geers, 2020) у поглављу о тимовима и границама заступа правило „*не бој се копије*“: дуплирање кода је по правилу мања цена од повезаности коју дељење уводи, и одмах затим сам наводи библиотеку образаца као пример на који то правило **не** може да се примени. (Mezzalana, Building Micro-Frontends, 2025) исту границу поставља оштрије и с обрнутог краја: његово упозорење да је „*погрешна апстракција далеко скупља од дуплираног кода*“ стоји уз тврдњу да су „*основне компоненте савршено место за централизацију*“, дакле аутор који најстроже упозорава на прерано дељење за овај један случај тврди да је дељење тачно. (Rufus, 2023) дели цело поглавље забрани прекомерне употребе дељеног кода међу микрофронтендовима, због пакла зависности (енгл. *dependency hell*) и распоређеног монолита који из тога следе, и наводи један изузетак: „*једна ствар која идеално може да се дели међу микрофронтендовима је библиотека компонента интерфејса*“. (Rappl, The Art of Micro Frontends, 2024) поставља пет критеријума за то шта се дели и подразумевано правило да се не дели ништа, а унутар истог поглавља *дизајн корисничког интерфејса* наводи као једну од девет области којима се управља, дакле област у којој се одсуство заједничког узима као недостатак а не као подразумевано стање.

Та четири исказа су проверљив налаз, а не низ подударних цитата, зато што су сва четири **изузети од властитих правила тих истих аутора**. Систем дизајна није изузет зато што је користан, него зато што је **проблем који решава другачијег реда од проблема које дељење ствара**. Повезаност коју дељење уводи је технички трошак, и плаћа се у брзини испоруке и у слободи надограђивања. Недоследност коју дељење уклања је трошак који плаћа корисник, и он се не види ни у једној метрици испоруке.

6.3 Критеријуми за дељење

Систем дизајна, задовољава свих пет Rappl-ових критеријума за и против дељења зависности, што га чини не изузетком, него **јединим случајем који их све испуњава истовремено**.

(Geers, 2020) наводи три осе за ту одлуку и трећа је она која одлучује.

- Прва је **сложеност**.
- Друга је **вредност поновне употребе**, где његов пример вреди задржати: картица производа изгледа као очигледан кандидат за заједничку библиотеку, појављује се на много места, али свака страница на којој се појављује од ње жели нешто своје, па

централна компонента убрзо носи десетак прекидача којима њени потрошачи бирају понашање. Његово разрешење је занимљивије од самог упозорења: картица остаје заједничка, али оно што се у њој разликује не постаје прекидач, него **место за уграђивање** садржаја који даје потрошач.

- Трећа је **доменска одређеност**, и она се проверава једним питањем: који тим има интерес да ову компоненту мења и зашто. Ако је одговор „сваки, због изгледа“, компонента је заједничка; ако је „један, због свог посла“, није, како год да изгледа.

7 Испорука, аутоматизација и тестирање

Микрофронтенд архитектура се од модуларног монолита не разликује по изгледу изворног кода, и један и други имају јасне границе, именоване уговоре и међусобно независне делове. Разлика је у томе **шта се засебно испоручује**, а то није својство кода него својство тока испоруке. Фрагмент који се не може распоредити без осталих јесте модул, а не микрофронтенд, колико год му граница у коду била чиста. Отуда и практична последица: **карактеристична својства ове архитектуре су у току испоруке, а не у изворном коду.**

7.1 Моно-репозиторијум, поли-репозиторијум и стратегија рада с гранама

Моно-репозиторијум представља приступ у којем се више апликација, сервиса или микрофронтендова развија и одржава у оквиру једног заједничког репозиторијума, док поли-репозиторијум подразумева да свака апликација или микрофронтенд има сопствени репозиторијум. Избор између ова два приступа утиче на начин управљања зависностима, дељење кода, организацију тимова и независност испоруке. Моно-репозиторијум олакшава дељење кода, управљање зависностима и локални развој, али може повећати спрегнутост пројеката, док поли-репозиторијум обезбеђује већу изолацију и независност тимова, уз цену дуплирања кода и тежег управљања заједничким стандардима. Због тога се као опште правило издваја да **топологија репозиторијума треба да следи топологију тимова, а не број микрофронтендова**: ако више тимова независно развија и испоручује своје делове система, поли-репозиторијум је природнији избор, док је за један тим или тесно координисану организацију моно-репозиторијум рационалнији.

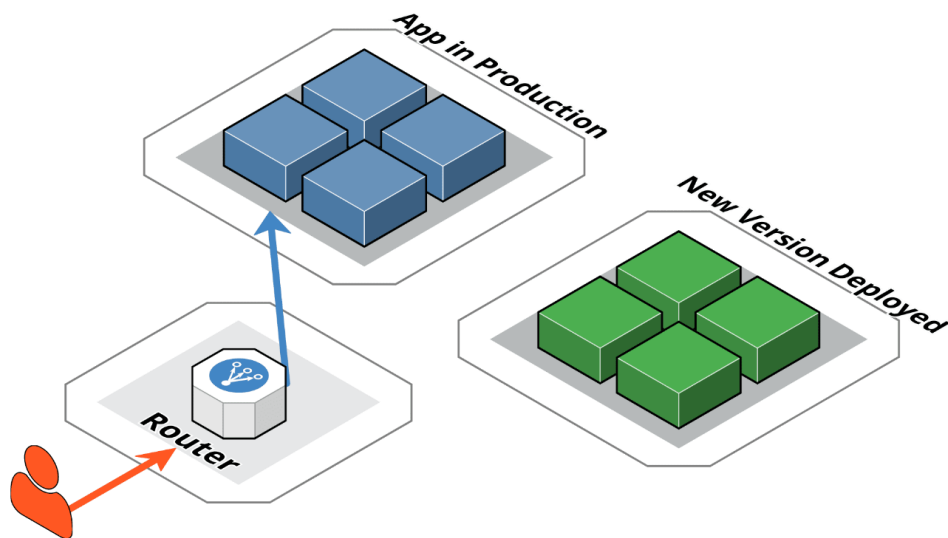
Без обзира на избор, моно-репозиторијум не сме да наруши независну испоручивост микрофронтендова. Независност се у том случају не обезбеђује самом структуром репозиторијума, већ током испоруке који препознаје измењене фрагменте и распоређује само њих. У погледу рада с гранама, суштински принцип је коришћење краткотрајних грана и честе интеграције, чиме се избегавају дуготрајне гране и сложени конфликти приликом спајања.

7.2 Верзионисање, повратак на претходну верзију и прекидачи функционалности

У микрофронтенд архитектури верзионисање и управљање издањима захтевају могућност да се различите верзије појединачних фрагмената јасно именују и да се у тренутку извршавања одреди коју верзију корисник добија. Због тога сваки фрагмент треба да има сопствену верзију, док механизам за откривање фрагмената треба да садржи њихове верзије, адресе и друге

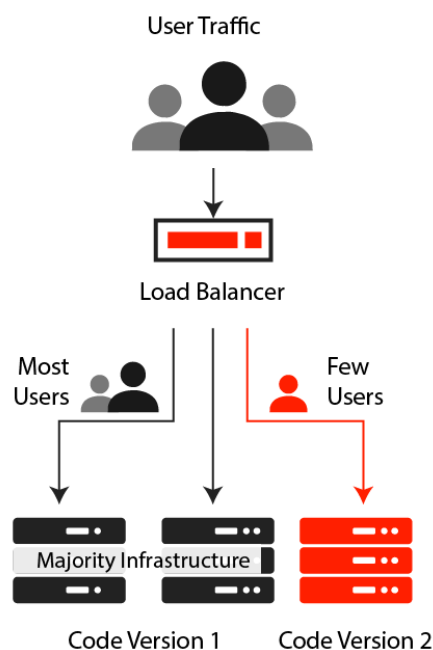
потребне мета-податке. Овај механизам омогућава да се саобраћај усмерава на одређену верзију фрагмента и представља основу за стратегије као што су *Blue-Green* и *Canary*.

Blue-Green омогућава брз повратак на претходну верзију пребацивањем саобраћаја на претходно окружење. Захтева додатан скуп свих хардверских компоненти на које ће друга верзија бити испоручена, а затим преко рутера усмерава саобраћај на један од та два скупа. (Maruthi, 2022)



Слика 7-1 Blue-Green испорука (Maruthi, 2022)

Canary омогућава постепено увођење нове верзије ограниченом делу корисника. Приступ је сличан као и код Blue-Green, међутим главна разлика је што се само одређени подскуп корисника преусмерава на одређени подскуп нових функционалности (Rastogi, 2022).



Слика 7-2 Canary испорука (Rastogi, 2022)

Повратак на претходну верзију може се реализовати поновним распоређивањем претходне верзије или преусмеравањем саобраћаја, али је при том неопходно проверити компатибилност са тренутним стањем остатка система.

Прекидачи функционалности (енгл. *feature flag/toggle*) имају другачију улогу: они контролишу понашање унутар једне верзије, док расподела саобраћаја одређује коју верзију фрагмента корисник добија.

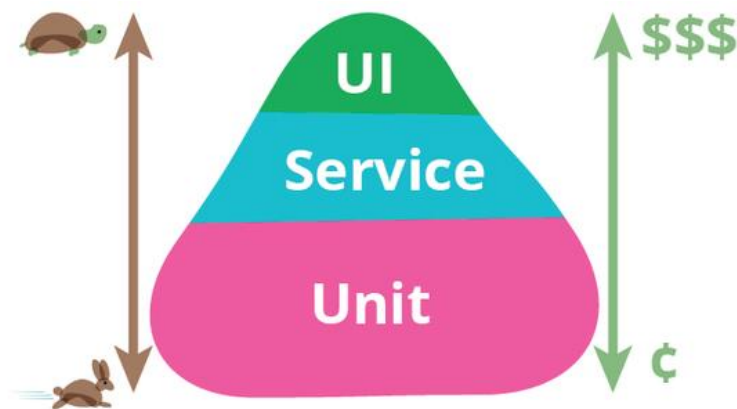
7.3 Миграција са монолитног фронтенда

Миграција са монолитног фронтенда на микрофронтенд архитектуру може се спровести постепеним издвајањем делова постојеће апликације, изградњом новог фронтенда изнад постојећег позадинског система или потпуном заменом система одједном. Најчешћи приступ је постепена миграција, при којој нови фрагменти преузимају поједина подручја примене док монолит постепено губи своју улогу. Пре миграције целог система корисно је применити приступ на мањем, реалном делу апликације како би се провериле техничке и организационе претпоставке.

Избор стратегије миграције пре свега одређује колико дуго је потребно одржавати стари и нови систем истовремено. Постепена миграција смањује домашај појединачних грешака и омогућава више мањих испорука, али продужава период двоструког одржавања. Потпуна замена је бржа у коначном преласку, али носи највећи ризик јер нема постепеног преласка нити једноставног повратка на претходно стање. Због тога стратегију миграције треба посматрати пре свега као одлуку о прихватљивом трајању двоструког одржавања, а не као избор саме технологије.

7.4 Стратегија тестирања и функције прикладности

Стратегија тестирања у микрофронтенд архитектури задржава уобичајену пирамиду тестова, али се већи део тестирања усмерава на појединачне фрагменте и њихове непосредне уговоре. Јединичних и интеграционих тестова треба да буде више, док свеобухватних тестова целог система (енгл. *End to End – E2E*) треба да буде мање и да буду пажљиво одабрани, јер тестирање целог система преко великог броја граница постаје сложено, споро и подложно палим тестовима због лажне негативности (енгл. *false negatives*) (Fowler, 2012). Фрагмент треба да проверава сопствени уговор, док тестови целокупног система треба да проверавају да ли више фрагмената правилно функционише заједно.



Слика 7-3 Пирамида тестова (Fowler, 2012)

Поред функционалних тестова, потребно је аутоматски проверавати и архитектонска својства система. Функције прикладности представљају извршиве провере којима се, на пример, могу контролисати величина пакета, перформансе, статичка анализа, покривеност тестовима, безбедност зависности и правила специфична за микрофронтенд архитектуру. Оне су посебно значајне у пројектима без производног окружења јер омогућавају да се одређена архитектонска својства објективно провере директно на артефакту.

7.5 Локални развој и изолација фрагмената

Локални развој микрофронтендова захтева начин да се фрагмент развија без потребе да се истовремено покрећу сви остали делови система. Један приступ је изолација фрагмента помоћу лажних суседа или посебне развојне странице која омогућава приказ различитих стања фрагмента. Лажни суседи треба да буду засновани на уговору фрагмента, док велики број сложених лажних имплементација може указивати на лоше постављене границе између фрагмената. Други приступ је локално повезивање са стварним, већ распоређеним суседним фрагментима.

Import Map Overrides

This developer tool allows you to view and override your import maps. [See documentation for more info.](#)

Module Status	Module Name	Domain	Filename
● Override	app1	localhost:8080	app1.js
● Default	react	cdn.jsdelivr.net	react.production.min.js
● Default	react-dom	cdn.jsdelivr.net	react-dom.production.min.js

Слика 7-4 Промена домена микрофронтенда при локалном развоју

Изолација и рад са стварним суседима не представљају међусобне замене, јер проверавају различите аспекте система. Изолација проверава да ли фрагмент испуњава сопствени уговор, док рад са стварним суседима проверава да ли се тај уговор правилно користи у стварном саставу. За оба приступа неопходно је да адресе фрагмената могу да се мењају у току извршавања, односно да постоји механизам за њихово откривање и замену. На тај начин локални развој постаје још једна последица потребе за индирекцијом између апликације обмотача и конкретне верзије фрагмента.

7.6 Развојно искуство

Развојно искуство представља важан аспект микрофронтенд архитектуре јер сложеност коју архитектура уводи највише осећају сами програмери. Добра развојна поставка треба да олакша креирање нових фрагмената, разумевање њихових уговора, тестирање и локални рад са целокупним саставом. Због тога су основни предуслови подршка у уобичајеном развојном окружењу и механизам за аутоматско креирање новог фрагмента из унапред дефинисаног обрасца.

Виши ниво развојног искуства подразумева централизовану документацију, уједначена правила за статичку анализу и обликовање кода, потпуне описе типова, припремљено тестирање, лажне суседе и развојна окружења за изолацију фрагмената. Додатне могућности укључују развој без мреже, алате за преглед састава фрагмената и посебан развојни портал. Ове могућности не треба посматрати као строго дефинисане фазе зрелости, већ као различите механизме за решавање конкретних проблема у развоју.

8 Безбедност и изолација микрофронтендова

Безбедност микрофронтенд архитектуре разликује се од безбедности монолитне апликације пре свега због расподеле одговорности између више фрагмената и тимова, као и због чињенице да се фрагменти у клијентској композицији извршавају у заједничком окружењу. Безбедносне мере могу имати домаћај документа, када важе за целу састављену апликацију, или домаћај за појединачни фрагмент, када се односе на појединачну испоруку.

8.1 Стандарди веб платформе као граница

Основу безбедности чине механизми које обезбеђује сама веб платформа. **Полиса безбедности садржаја** (енгл. *Content Security Policy - CSP*) представља листу дозвољених извора ресурса и може ограничити, између осталог, учитавање скрипти, чиме представља важну заштиту од одређених облика *Cross-site scripting - XSS* напада (Rappl, The Art of Micro Frontends, 2024).

За сложене микрофронтенд системе посебно је значајан режим *Report-Only*, који омогућава откривање стварних ресурса и крајњих тачака које користе различити фрагменти пре увођења рестриктивне политике.

HTTP Strict Transport Security - HSTS приморава прегледач да комуникацију са одређеним домаћином обавља преко HTTPS-а (енгл. *Hypertext Transfer Protocol Secure*), чиме се смањује ризик од напада посредника и пресретања осетљивих података (Rappl, The Art of Micro Frontends, 2024).

Преузимање ресурса са различитих извора (енгл. *Cross-origin resource sharing - CORS*) одређује којим пореклима сервер дозвољава приступ својим ресурсима. На тај начин се контрола приступа ресурсима фрагмента спроводи на његовом пореклу, док CSP контролише којим ресурсима сама састављена страница верује. Ове две политике имају различит домаћај и морају бити међусобно усклађене (Rappl, The Art of Micro Frontends, 2024).

8.2 Границе скриптова

За разлику од CSS-а, за *JavaScript* у оквиру клијентске композиције не постоји непосредан механизам који би различитим микрофронтендовима дао засебна окружења извршавања (Rappl, The Art of Micro Frontends, 2024). Основни изолациони механизам веб платформе је порекло (енгл. *origin*), а фрагменти који се извршавају у истом документу деле исто окружење. Последица тога је да један фрагмент може приступити DOM-у других фрагмената, делити веб складиште, користити заједничке објекте и комуницирати преко заједничке магистрале догађаја.

На клијенту потпуну изолацију може да обезбеди *iframe*, јер фрагменту даје засебно порекло, а додатним *sandbox* ограничењима могу се контролисати његове могућности (Rappl, The Art of Micro Frontends, 2024) (Mezzalira, Building Micro-Frontends, 2025).

Серверска композиција представља другачији архитектонски приступ код којег се изолација може спровести на месту извршавања, а не у заједничком окружењу прегледача.

Из тога следи важан закључак: **микрофронтенд није безбедносна граница у клијентској композицији**. Он је пре свега јединица испоруке и организационе одговорности, док се стварна изолација мора обезбедити механизмима веб платформе и процесима испоруке.

8.3 Складиштење приступног токена

Избор начина чувања приступног токена посебно је значајан у микрофронтенд архитектури. Код веб складишта (*localStorage* или *sessionStorage*) *JavaScript* може директно да прочита токен, што у клијентској композицији значи да га може прочитати и било који други фрагмент у истом пореклу и домену. Поред тога је могуће користити и веб складиште и колачиће као начине дељења токена.

У контексту клијентске композиције, предност колачића је у томе што смањује системски домет потенцијалног XSS напада: компромитовани фрагмент не може директно да прочита приступни токен. Међутим, место чувања токена не одређује права корисника, ауторизација се и даље мора спроводити на серверској страни (Mezzalira, Building Micro-Frontends, 2025).

8.4 Дељене зависности

Безбедност микрофронтенд архитектуре није само техничко питање већ и питање организације. Дељене зависности представљају заједничку површ напада: када више фрагмената користи исту зависност, њена рањивост потенцијално погађа све кориснике те зависности, а њена надоградња захтева координацију више тимова. Због тога се препоручује да се дељење зависности не подразумева, већ да се за сваки случај оправда посебним критеријумима.

Посебну пажњу захтевају **надзор и бележење догађаја**. OWASP међу релевантним претњама издваја рањиве зависности и недостатке у бележењу и надзору, али су потребни и контрола ланца снабдевања и процес прегледа доприноса. У микрофронтенд систему надзор мора да обухвати и ниво целог документа и ниво појединачних фрагмената, како би било могуће утврдити шта је испоручено, када и од кога.

9 Анти-патерни микрофронтенд архитектуре

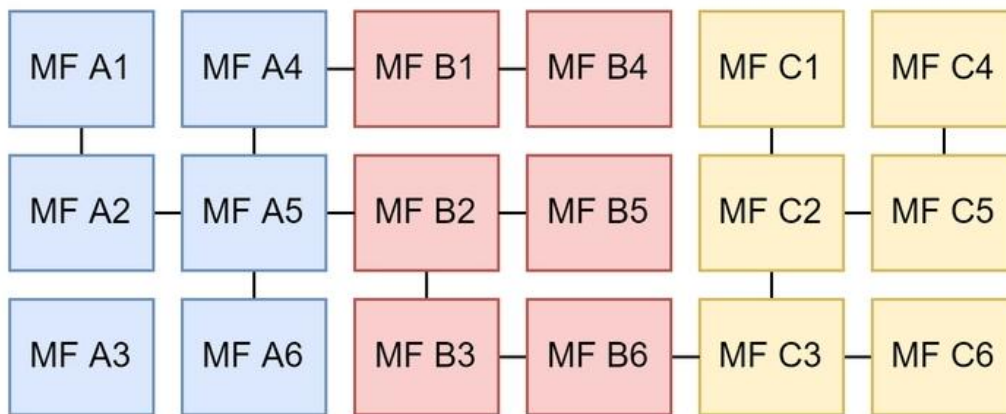
Анти-патерн представља решење које се често бира јер у почетку делује прикладно, али дугорочно производи више штете него користи, при чему за њега постоји препознатљив начин преуређења. Према томе, појединачна грешка није анти-патерн, већ образац који се понавља. У контексту микрофронтенд архитектуре важно је разликовати анти-патерн од компромиса: одлука чија је цена позната и за коју постоји механизам њеног ограничавања не мора представљати анти-патерн (Neill, Laplante, & DeFranco, 2011).

Заједничка карактеристика анти-патерна разматраних у овом поглављу јесте да представљају примену пракси које су биле корисне у монолитним системима, али се примењују на местима где се нарушавају границе независне испоруке. Тако избегавање понављања кода може довести до преурађене апстракције, једно место истине може довести до дељења стања, компонентно размишљање до погрешне гранулације, а слобода избора технологије до недостатка управљања.

9.1 Погрешна гранулација: микрофронтенд или компонента

Један од најважнијих архитектонских избора јесте одређивање границе микрофронтенда. Величина сама по себи није довољан критеријум: микрофронтенд треба да представља **заокружену пословну целину** и да може да се независно испоручује, док је компонента пре свега елемент корисничког интерфејса и њој је потребно дати контекст како би била корисна у пословном аспект (Mezzalana, Building Micro-Frontends, 2025). Као грубу оријентацију, литература наводи да више од пет микрофронтендова на једном приказу може указивати на прекомерно ситну поделу.

Ипак, број фрагмената треба посматрати као **симптом, а не као критеријум**. Важније је колико комуникације ствара подела. Превише ситна подела доводи до великог броја веза између фрагмената, чиме се губи корист независности. Сличан проблем се описује и код микросервиса као *Grains of Sand* и *Big Ball of Distributed Mud*: превише ситне целине захтевају накнадно повезивање како би обавиле користан посао. Из тога следи да је добру гранулацију прикладније процењивати према кохезији и количини комуникације него према фиксном броју фрагмената (Richards & Ford, 2025) (Rappl, Top 10 Micro Frontend Anti-Patterns, 2024).

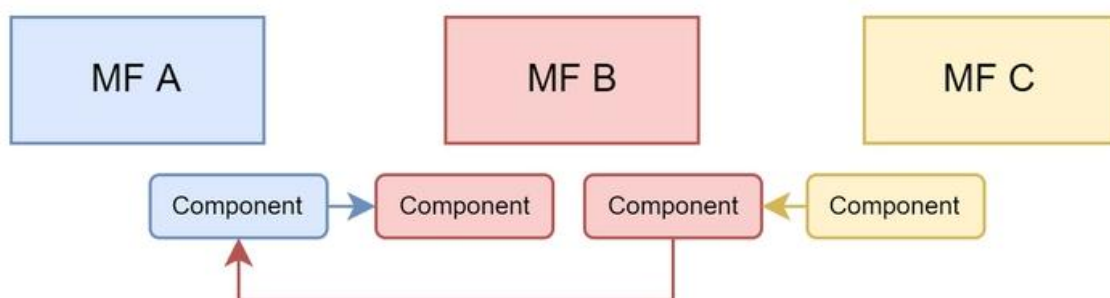


Слика 9-1 Ситна подела микрофронтендова (Rappl, Top 10 Micro Frontend Anti-Patterns, 2024)

При томе, границу је боље поставити да фрагменти ипак буду крупнији. Прекомерно ситна подела захтева спајање фрагмената и ствара комуникацију која је претходно била интерна, док раздвајање превеликог фрагмента захтева промене у домену, стању и власништву. Стога се као практично правило издваја да микрофронтенд треба да прати пословну способност и кохезију, а не величину корисничког интерфејса.

9.2 Дељење стања и двосмерна зависност

Дељење стања и двосмерна зависност могу се посматрати као два облика истог проблема: губитка јасног смера зависности између фрагмената. Код дељеног стања више фрагмената чита и мења исте податке, док двосмерна зависност ствара кружну зависност у коду. У оба случаја фрагменти престају да буду истински независни (Mezzalana, Building Micro-Frontends, 2025).

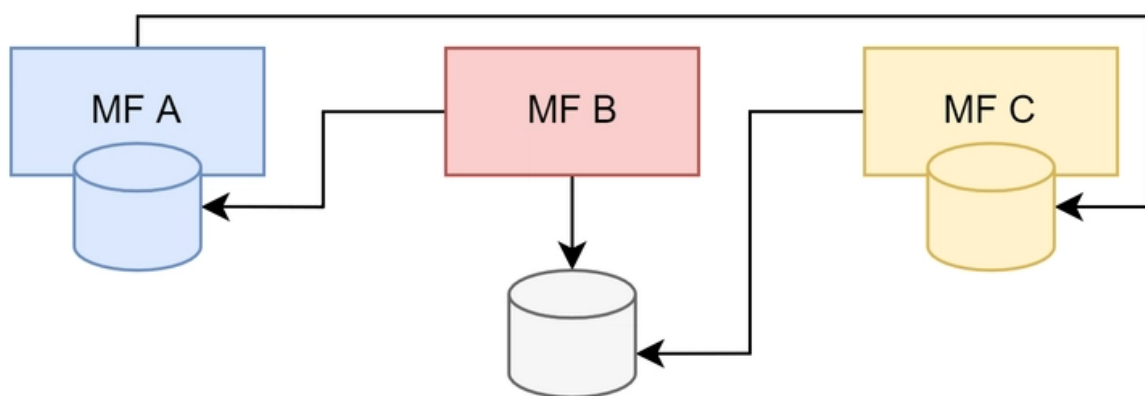


Слика 9-2 Чврста спрегнутост микрофронтендова (Rappl, Top 10 Micro Frontend Anti-Patterns, 2024)

Препоручени приступ је да управљање стањем остане унутар фрагмента, а да се између фрагмената преносе само неопходне поруке путем догађаја. Ипак, прелазак са заједничког стања на комуникацију догађајима не уклања спрегнутост сам по себи. Ако се кроз поруку догађаја преноси читав доменски модел, фрагменти и даље морају да деле његов облик и

значење. Због тога је погодније да поруке буду што мање, односно да садрже само информацију неопходну примаоцу, као што су идентификатор и назив догађаја.

Заједничко складиште, магистрала догађаја или контекст који апликација обмотач прослеђује фрагментима представљају различите облике посредовања. Њихов избор зато није избор између спрегнутости и њеног одсуства, већ избор **од чега ће фрагмент зависити**. Најпогоднији је посредник са најмањим и најстабилнијим уговором. Ако два фрагмента морају да деле исто пословно стање или имају снажну међусобну зависност, то може бити знак да су границе фрагмената погрешно постављене и да их треба преиспитати (Mezzalana, Building Micro-Frontends, 2025).

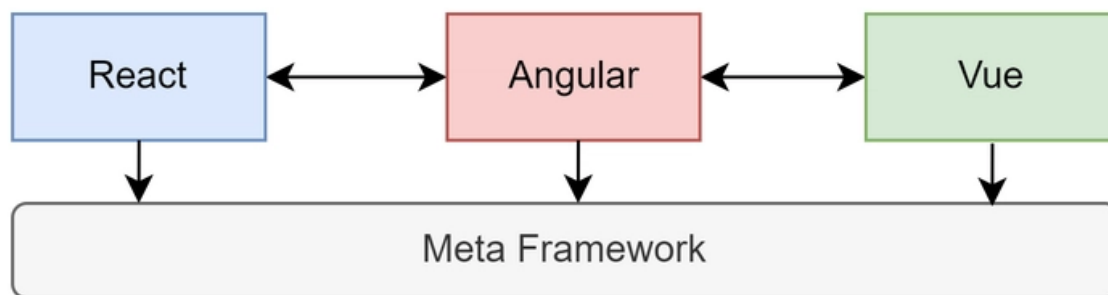


Слика 9-3 Неконзистентност дистрибуираних података (Rappl, Top 10 Micro Frontend Anti-Patterns, 2024)

9.3 Анархија: вишеструки оквири без механизма управљања

Анархија не подразумева само употребу више радних оквира, већ **одсуство јасних смерница, стандарда и механизма управљања**. Тиме долази до силоса знања и недостатка могућности да програмери брзо пређу у други тим. Због тога сама употреба више технологија не представља нужно анти-патерн.

Постоји више мана које овај приступ доноси. Неке су организационе, као што су повећано когнитивно оптерећење и различити начини решавања проблема, док је једна директно техничка: сваки оквир може донети сопствене зависности и пакете, чиме расте количина кода коју корисник преузима и смањује се могућност дељења кеша (енгл. *cache*). Овај други трошак не може се решити само организационим договором, јер га директно плаћа корисник. (Rappl, Top 10 Micro Frontend Anti-Patterns, 2024)



Слика 9-4 Велики број различитих радних оквира (Rappl, Top 10 Micro Frontend Anti-Patterns, 2024)

Да би архитектура са више радних оквира остала контролисана, потребни су јасно дефинисани дозвољени оквири, заједнички систем дизајна, јединствени уговор о укључивању фрагмента, аутоматска провера тих уговора и поступак промене заједничких правила. Некад је и нужно користити више радних оквира јер је систем у процесу постепене миграције на микрофронтенд архитектуру. У случају да је приступ рада са више оквира уведен као привремено решење, потребно је дефинисати и рок и одговорност за његово уклањање. У супротном, привремена хетерогеност лако постаје трајно стање и прераста у анархију.

9.4 Недостатак слоја против корупције и преурањена апстракција

Недостатак слоја против корупције (енгл. *Anti-Corruption Layer*) и преурањена апстракција представљају два различита случаја истог проблема: неправилног управљања обликом података и логике на граници између система. **Слој против корупције** омогућава да се наслеђени или спољни систем прилагоди уговору микрофронтенд архитектуре без ширења његових специфичности на остатак система. Насупрот томе, преурањена апстракција износи заједнички код изван фрагмента пре него што је потврђено да је тај образац заиста стабилан (Mezzalira, Building Micro-Frontends, 2025).

Посебан ризик представљају заједничке библиотеке које садрже пословну логику. Поновна употреба кода може нарушити ограничени контекст јер знање о једном послу прелази границе којима припада. Тиме долази и до такозваног „*Пакла зависности*“ и/или појма „*дистрибуирани монолит*“, у којем више тимова постаје зависно од измена једне библиотеке.

Заједнички код није апсолутно забрањен. Библиотеке корисничких компоненти и код помоћних функција генеричког карактера могу бити оправдани, али треба избегавати дељење пословног знања између фрагмената. Као практичан критеријум може се посматрати и темпо промена: заједничка библиотека треба да се мења ређе од фрагмената који је користе, јер у

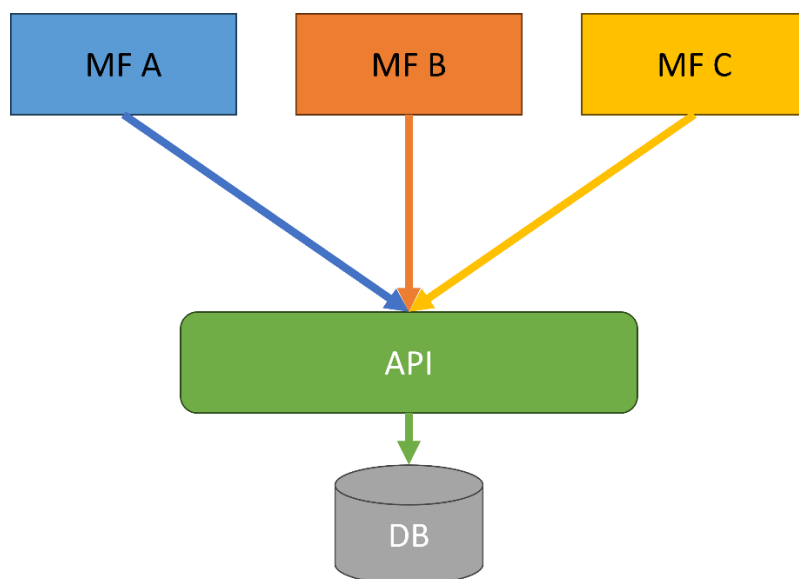
супротног постаје извор скривене спрегнутости и намеће свој ритам издавања осталим фрагментима.

Слој против корупције треба посматрати као привремени механизам који омогућава постепену интеграцију и чије уклањање треба планирати. Ако остане без рока и власника, и сам може постати трајна зависност.

9.5 Вишеструки позиви ка бекенду

Повећање броја позива ка бекенду специфично је по томе што није нужно последица погрешне одлуке програмера, већ може настати као природна последица разбијања монолитне странице на више независних фрагмената.

Проблем се може јавити у облику поновљених захтева за истим подацима или у облику већег броја различитих захтева који повећавају број мрежних позива. За такве случајеве погодни су механизми који не захтевају промену самих фрагмената, попут краткотрајног кеширања одговора и посредничког обједињавања захтева. Овакви механизми задржавају независност фрагмената јер они не морају да знају један за другог.



Слика 9-5 Више микрофронтовних позива исти API

Насупрот томе, обмотач који преузима комуникацију са бекендом за више фрагмената може поново увести спрегнутост коју микрофронтенд архитектура треба да уклони. Увођењем таквог посредника указује се да сваки модул тада постаје зависан од посредника, док сам посредник може представљати *Architecture Sinkhole* ако само прослеђује податке без сопствене логике. Због тога је боље прво прихватити повећан број независних захтева, а оптимизацију увести тек када је она заиста потребна и када се може спровести без нарушавања граница фрагмената.

10 Анализа проблема истраживања

Претходних осам поглавља описивало је механизме без предмета. Ово поглавље уводи предмет, на основу којег ће се описивати примена механизма у наредним поглављима.

Излагање тече од домена ка захтеву и од захтева ка структури. Одељак 10.1 описује домен, учеснике и радни ток, и из њега изводи једну особину домена која целом раду даје предмет. Одељак 10.2 исказује функционалне захтеве и означава који су од њих остварени. Одељак 10.3 преводи доменске бриге у архитектонске карактеристике. Одељак 10.4 идентификује поддомене и ограничене контексте, чиме добија списак *делова*. Одељак 10.5 показује на овом домену оно што је 2.4 тврдило уопште, зашто монолитан клијентски слој овом проблему не одговара, и то показује као тврдњу која се могла и оповргнути. Одељак 10.6 исказује захтеве поверљивости и трага приступа.

10.1 Домен, корисници и радни ток лекара

Домен је **амбулантни специјалистички преглед и медицинска документација која из њега настаје**. Систем не покрива болничко лечење, лабораторијску дијагностику, наплату услуга ни размену података с другим установама; покрива један заокружен догађај, сусрет лекара и пацијента у заказаном термину, и трајни запис који после тог сусрета остаје.

Лекарево време је најређи ресурс у систему. Термин је унапред ограниченог трајања, у њему се обавља и стручни и документациони посао, чекање на одзив, тражење података на другом екрану, поновно уписивање онога што систем већ зна, и све то се плаћа пацијентовим временом за преглед. Постојећи текст то каже сликовито (*„лекар је уско грло система“*); овде се исто исказује као захтев: **прекид у радном току није непријатност корисничког интерфејса, него губитак клиничког времена.**

Табела 10-1 Учесници и њихове надлежности у домену

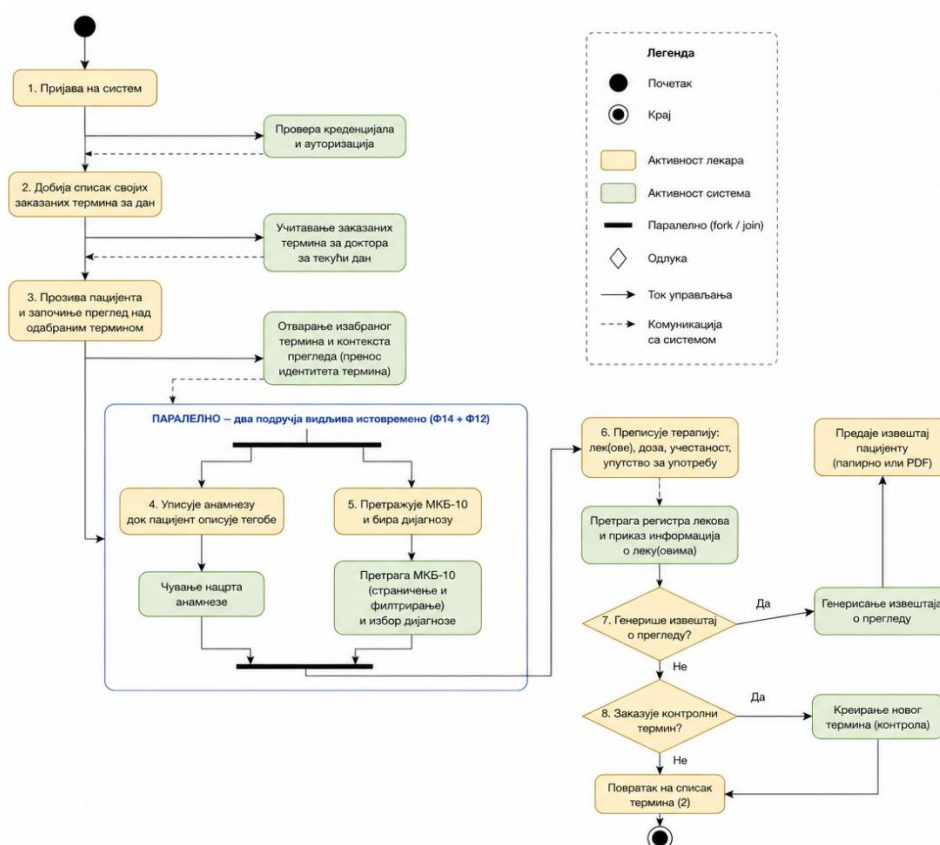
Учесник	Улога у домену	Шта покреће	Шта сме да види
Лекар	примарни учесник; носилац стручне одлуке и аутор записа	заказивање, преглед, дијагнозу, терапију, извештај	податке пацијената које лечи
Пацијент	секундарни учесник; субјекат података	отказивање сопственог термина	искључиво сопствене податке
Администратор	системски учесник, ван клиничког тока	учитавање референтних података, надзор трага приступа	мета-податке о приступу, не садржај записа

Из табеле следи прва тврдња која се даље користи: **Провера надлежности не сме зависити од тога који је екран отворен.** Право лекара да види једног пацијента не зависи од тога који

је екран отворен, него од тога да ли између њих двоје постоји термин; право пацијента зависи од идентитета, а не од путање.

Радни ток. Слика 10-1 приказује ток у облику дијаграма активности:

1. лекар се пријављује на систем;
2. добија списак својих заказаних термина за дан;
3. прозива пацијента и започиње преглед над одабраним термином;
4. док пацијент описује тегобе, лекар уписује анамнезу;
5. паралелно с уписивањем, претражује међународну класификацију болести (МКБ-10) и бира дијагнозу;
6. преписује терапију, један или више лекова, свакоме дозу, учестаност и упутство за употребу;
7. по потреби генерише извештај о прегледу и предаје га пацијенту, у папирном облику или као PDF;
8. по потреби заказује контролни термин и враћа се на списак термина.



Слика 10-1 Дијаграм активности радног тога лекара

Три особине тог тока нису очигледне из његовог описа, а свака од њих је у даљем тексту носива.

Прво, у једном термину лекар користи четири скупа података различитог животног века и различитог власништва. Списак термина је трансакциони: мења се сваког дана и мења га сама установа. Запис о прегледу је клинички документ: настаје једном, накнадно се не преправља и има правног и стручног адресата изван система. МКБ-10 је спољашња класификација: не мења је ни лекар ни установа, ревидира се на туђој шеми, користи се само за читање и велика је, у изграђеном систему референтни скуп шифри је реда величине неколико милиона знакова, што је податак који одређује и начин претраге. Регистар лекова је такође спољашњи и такође се користи само за читање, али се мења чешће од класификације.

Друго, ток садржи један тренутак у ком два подручја морају да буду видљива истовремено. Корак 5 није корак после корака 4, него уз њега: дијагноза се бира *док* је анамнеза видљива, зато што анамнеза јесте основ дијагнозе. Уколико избор дијагнозе захтева одлазак на други екран и повратак, лекар мора да задржи у глави оно што је управо записао, а то је управо она врста прекида коју полазна брига домена забрањује. Захтев који из тога следи исказан је у 10.2 као Ф12 и Ф14 заједно, а не одвојено.

Треће, ток садржи и један прелаз који је обрнут по природи. Између корака 2 и 3 мења се предмет рада: са списка термина прелази се на један термин, и од тог тренутка списак није потребан. Ту подручја **не** треба да буду видљива истовремено; потребно је да идентитет одабраног термина преживи прелаз.

Најзад, једна честа претпоставка овде се изричито одбацује. Из тога што су подручја раздвојива **не** следи да их корисник опажа као раздвојена: лекару је преглед један посао, а не четири. Захтев који из тога следи припада 10.3 и тамо је наведен као конзистентност интерфејса; овде је важан због смера извођења, зато што свака подела коју 10.4 предложи мора да остане невидљива у корисничком искуству чак и када је у испоруци стварна.

Домен је погодан за испитивање микрофронтенд архитектуре јер истовремено садржи функционално раздвојене целине, различите ритмове промене, заједничке корисничке токове, потребу за комуникацијом између делова и захтеве за независним учитавањем и контролом приступа. На тај начин показна апликација не служи само као пример примене већ идентификује конкретне архитектонске ситуације у којима се предности и ограничења микрофронтенд приступа могу експериментално проверити.

10.2 Функционални захтеви

Функционални захтеви исказани су као способности по учеснику и означени идентификаторима Ф1–Ф24, на које се даља поглавља позивају. Табела 10-2 их наводи заједно с учесником и доменским подручјем.

Табела 10-2 Функционални захтеви

Ид	Захтев	Учесник	Подручје
Ф1	Регистрација лекара, с одабиром специјалистичког одељења	лекар	приступ
Ф2	Регистрација пацијента	пацијент	приступ
Ф3	Пријава у систем и одјава	сви	приступ
Ф4	Преглед и измена сопственог профила и лозинке	сви	приступ
Ф5	Списак специјалистичких одељења	лекар	приступ
Ф6	Приказ сопствених заказаних термина у временском прегледу	лекар, пацијент	заказивање
Ф7	Заказивање термина пацијенту, уз одбијање преклапања	лекар	заказивање
Ф8	Отказивање термина	лекар, пацијент	заказивање
Ф9	Померање термина на друго време	лекар	заказивање
Ф10	Претрага пацијената при заказивању	лекар	заказивање
Ф11	Претрага и филтрирање лекара по одељењу и специјализацији	пацијент	заказивање
Ф12	Претрага каталога МКБ-10	лекар	дијагноза
Ф13	Претрага регистра лекова и приказ форме лека	лекар	терапија
Ф14	Евидентирање прегледа: анамнеза, дијагноза, време почетка	лекар	преглед
Ф15	Преписивање терапије: лек, доза, учестаност, упутство	лекар	терапија
Ф16	Приказ историје прегледа једног пацијента	лекар	преглед
Ф17	Претрага прегледа по пацијенту, лекару, дијагнози и периоду	лекар	преглед
Ф18	Збирни приказ пацијента (термини, прегледи, терапије)	лекар	преглед
Ф19	Генерисање извештаја о прегледу	лекар	документација
Ф20	Преузимање извештаја као PDF документа	лекар, пацијент	документација
Ф21	Приказ сопствених извештаја и претрага извештаја	пацијент, лекар	документација
Ф22	Приказ статистике прегледа и дијагноза	лекар	надзор
Ф23	Приказ трага приступа медицинским подацима	администратор	надзор
Ф24	Учитавање референтних података (МКБ-10, лекови, одељења)	администратор	надзор

Један захтев тражи два подручја истовремено, а два захтева траже пренос између екрана. Ф12 и Ф14 морају да буду задовољени у истом тренутку; Ф6 и Ф14 морају да буду задовољени један после другог, уз пренос идентитета термина. Ниједна од двеју обавеза не следи из појединачног захтева, обе следе из **пара** захтева, и то је разлог зашто списак захтева сам по себи није довољан као опис проблема.

10.3 Нефункционални захтеви

Функционални захтеви из 10.2 казују шта систем ради; они не разликују систем који то ради у пола секунде од система који то ради у пет, ни систем који се испоручује недељно од система који се испоручује једном у пола године. Та разлика је предмет архитектонских карактеристика, за које је 2.1 поставило три услова, да карактеристика није доменски специфична, да утиче на структуру и да је критична за успех система, и дало табелу 2-1 као речник.

Метод и његово ограничење. Карактеристике се изводе из три извора: из **изричитих доменских брига** које наручилац именује, из **захтева пројекта** који их подразумевају, и из **прећутног доменског знања** које нико не изговара зато што се у датом домену узима за очигледним.

Додатна препорука и ограничење које пројектовање система треба да поштује каже: **треба изабрати што мање карактеристика**. Разлог није штедња у документу, него то да свака карактеристика уводи структурну обавезу, а обавезе се међусобно потиру. Списак који следи стога има седам ставки, и међу њима су прве три означене као **водеће**, у складу с препоруком да се одабере горња тројка.

Табела 10-3 Архитектонске карактеристике

Карактеристика	Извор	Шта захтева домен
Испоручивост	захтев пројекта	четири скупа података из 10.1 мењају се различитим ритмом; подручје које се не мења не сме да буде поново испоручено да би се изменило подручје које се мења
Перформансе	изричита доменска брига	прекид у термину је губитак клиничког времена; претрага каталога не сме да значи преузимање целог каталога, а почетно учитавање не сме да чека сва подручја
Поверљивост	прећутно доменско знање	право приступа податку о пацијенту не следи из тога који је екран отворен (10.1, Табела 10-1); мора да буде проверљиво по захтеву и остављати траг (10.6)
Проширивост	захтев пројекта	девет способности из 10.2 још нема клијентско остварење; додавање подручја не сме да значи измену постојећих
Конзистентност интерфејса	изричита доменска брига	лекару је преглед један посао, па подела подручја не сме да буде видљива као разлика у изгледу или понашању контрола
Тестабилност	захтев пројекта	клиничка одлука се записује једном и не преправља се, што значи да грешка у упису има трајну последицу
Доступност	прећутно доменско знање	отказ једног подручја не сме да онемогући цео термин; неисправна претрага каталога не сме да спречи упис анамнезе

Зашто су испоручивост, перформансе и поверљивост водеће. Одабир следи из тога које би неиспуњење учинило систем неупотребљивим у домену, а не само лошијим. Испоручивост је водећа зато што је она једина карактеристика коју захтева *особина података* из 10.1, независно од броја корисника и брзине мреже: пошто МКБ-10 мења спољашња установа а списак термина сама ординација, систем у ком те две измене имају исти циклус испоруке трајно плаћа туђи ритам. Перформансе су водеће зато што их захтева најскупљи ресурс у домену: време учесника. Поверљивост је водећа зато што је она једина карактеристика чије неиспуњење није лош производ него штета за самог пацијента. Остале четири нису мање реалне, али су све четири последице које се могу поправити накнадном изменом, док се ниједна од првих три не може.

10.4 Идентификација поддомена и ограничених контекста

Одељак 3.2 је поставио инструмент: границе фрагмената налазе се у домену, преко поддомена и ограничених контекста, а не у слоју технологије, и тамо је остављено да се инструмент примени када постоји домен на коме се разлика види. Овај одељак је та примена.

Поступак који се препоручује за одвајање поддомена је *event storming*, радионица на којој учествују менаџери производа, тестери и програмери, зато што се поддомени откривају из **разлике** у томе како исти посао виде људи различитих улога.

Доменски вођено пројектовање врши поделу на **основне** (енгл. *core*), **потпорне** (енгл. *supporting*) и **опште** (енгл. *generic*) поддомене, где основни носи оно у чему систем даје вредност, потпорни му је потребан а сам по себи није разлика, а општи је решен проблем који се преузима. Примењено на домен из 10.1, разврставање је недвосмислено и корисно: **преглед** је основни поддомен, зато што медицинска документација јесте оно због чега систем постоји; **заказивање** је потпорни, зато што је термин услов прегледа а не његова сврха; **каталог МКБ-10** и **регистар лекова** су општи, зато што их систем не производи него преузима у затвореном облику; **приступ** (пријава, идентитет, надлежност) је општи.

Део се може одвојити **функционално**, по подручју посла, или **технички**, по улози у изградњи стране. Његов критеријум за исправност границе је локалност промене. Граница је добра ако се измена једног захтева задржи унутар тог једног дела, а провера самосталности је оно што се описује као „угаси један део“: ако се један део искључи, остатак треба да остане употребљив, само с мање могућности.

Примењено на списак из 10.2, оба теста дају исти исход и он је наведен у Табели 10-4.

Табела 10-4 Поддомени, ограничени контексти и врста границе

Поддомен	Врста	Захтеви	Ритам промене	Тест „угаси један део“
Преглед и документација	основни, функционални	Ф14, Ф15, Ф16–Ф21	мења се са стручном праксом и с обликом документа	без њега термин нема исход; остатак система остаје употребљив
Заказивање	потпорни, функционални	Ф6–Ф11	мења се са организацијом рада ординације	без њега нема нових термина, али се постојећи преглед доврши
Каталог болести	општи, функционални	Ф12	мења га спољашња установа, ретко	без њега се анамнеза уписује, дијагноза не бира
Регистар лекова	општи, функционални	Ф13	мења се чешће, али и даље споља	без њега се преглед евидентира без терапије
Приступ и идентитет	општи, функционални	Ф1–Ф5	најстабилнији део система	без њега ништа друго није употребљиво
Обавештавање, навигација, сесија	унакрсна област, техничка	— (в. 10.2, четврти налаз)	мења се са сваким подручјем помало	без њих сваки поједини захтев ради, а систем није употребљив као целина

10.5 Ограничења монолитног фронтенда у посматраном домену

Шта би у проблему из 10.1 стварно било изгубљено ако би клијентски слој остао један, недељив део.

Прво, један циклус испоруке за четири различита ритма промене. Из 10.1 и Табеле 10-4 следи да измену подручја дијагнозе покреће спољашња установа, измену заказивања сама ординација, а измену записа о прегледу стручна пракса. У недељивом клијентском слоју свака од тих измена улази у исти процес изградње и исти акт испоруке, што значи да ревизија класификације болести носи ризик за екран на ком лекар уписује анамнезу, иако с њим нема ништа заједничко. Испоручивост је у 10.3 водећа карактеристика управо због те особине података, и она је прва која у монолитном клијенту не може да буде задовољена, не зато што је испорука тешка, него зато што је **недељива**.

Друго, један технолошки избор за подручја различитих захтева. Претрага каталога од неколико милиона знакова и упис клиничког записа с најстрожим захтевом за тачношћу нису исти проблем, а у недељивом клијенту морају да се решавају истим средствима, зато што у једном пројекту постоји једна верзија сваке зависности. Последица није естетска: она значи да ниједно подручје не може да промени средство пре него што га промене сва подручја.

Треће, један домен отказа. Доступност је у Табели 10-3 наведена као прећутан захтев домена: неисправна претрага каталога не сме да спречи упис анамнезе. У недељивом клијентском слоју то не може да се гарантује, зато што грешка при учитавању било ког дела може срушити документ у ком су сви делови. У подељеном клијенту тај захтев постаје лакше испуњив, један микрофронтенд може да откаже, али **остали ће наставити са радом неометано.**

Монолитан клијентски слој био би тачан одговор на овај проблем под три услова: да сва подручја имају исти ритам промене и истог носиоца промене; да ниједно подручје не носи захтев за перформансама или тачношћу који се битно разликује од осталих; и да укупна количина функционалности остаје таква да један тим може да је држи у глави.

10.6 Захтеви поверљивости и трага приступа медицинским подацима

Поверљивост је у Табели 10-3 наведена као водећа карактеристика и као прећутно доменско знање: у посматраном домену поверљивост медицинских података представља основно ограничење система, а не опциону функционалност.

Право приступа не следи из места у систему, него из односа између особа. Из Табеле 10-1 следи да лекар сме да види податке пацијената које лечи, а не све пацијенте, и да пацијент сме да види само своје податке, не и туђих пацијената. То право не зависи ни од отвореног екрана ни од путање ни од редоследа радњи, него од постојања термина између два учесника, што значи да провера мора да се обави при **сваком приступу податку.**

Сваки приступ медицинском податку мора да остави траг. Разлог је особина домена, а не техничка предострожност: пацијент не може да опази да су његови подаци прочитани, па је траг једини начин да неовлашћен приступ постане уочљив после чина. Из тога следи и шта траг мора да садржи, ко је приступио, чему, којег типа и када, и, што је важније, шта не сме да садржи. Запис о употреби не сме да постане други примерак самог податка: **траг о приступу медицинском запису који садржи садржај тог записа удвостручио је поверљиви податак и уместо надзора изложио другу рањивост.** Захтев је стога исказан у ограниченом облику, мета-подаци о приступу, никада клинички садржај (Rappl, The Art of Micro Frontends, 2024).

Након одјаве или истека сесије, приступ заштићеним ресурсима мора бити онемогућен у свим деловима клијентског система. У микрофронтенд архитектури то захтева јединствено управљање стањем аутентификације или механизам којим се промена стања сесије прослеђује свим активним деловима система.

11 Архитектура решења

При пројектовању архитектуре овог система потребно је донети четири одлуке: шта чини фрагмент, како се фрагменти састављају, како се рутирају и комуницирају, и на који начин приступити пројектовању серверског слоја.

11.1 Примена оквира за одлучивање: дефинисање граница фрагмената

Прва одлука, шта чини један микрофронтенд: Табела 10-4 наводи пет поддомена и једну унакрсну област, са ритмом промене и с исходом теста „угаси један део“ за сваки. Тај улаз, међутим, није и одговор, и 10.4 то изричито каже: **не мора сваки ограничени контекст постати сопствени микрофронтенд (3.2).**

Табела 11-1 Од ограниченог контекста до фрагмента

Ограничени контекст (10.4)	Постаје	Радни оквир	Основ одлуке
Преглед и документација	самосталан фрагмент	Angular	основни поддомен, сопствени ритам промене, сопствени екран
Заказивање	самосталан фрагмент	React	потпорни поддомен на сопственом екрану, независан ритам
Каталог болести	самосталан фрагмент	Vue	спољашњи носилац промене, сопствени захтев за перформансама, дели екран с прегледом
Приступ и идентитет	фрагмент и библиотека	React (екрани)	екрани су вертикална подела; стање сесије нема приказ
Обавештавање и навигација	два фрагмента	без оквира	различит уговор о рутама (5.1, 3.1)

11.2 Композиција

Друга одлука оквира има два дела која се често мешају у један: **где** се поглед саставља и **чиме**.

Оквир из 4.2 нуди три места, сервер, ивични чвор и клијент. Основ одлуке да се микрофронтендови састављају на клијенту је сам облик апликације: систем је интерактивно радно средство, а не страница чији садржај треба да буде читљив пре него што се извршни код учита.

За само састављање користи се *Module Federation*, механизам градитеља *webpack* описан у 4.4.

Дељење зависности се постиже уз помоћ *Module Federation* и библиотеке *single-spa* које дају могућност да се одређене зависности ставе као синглтон (енгл. *singleton*). Увезене зависности као што су библиотеке радних оквира се деле међу више микрофронтендова и савршени су

кандидати да буду означени са синглтон ознаком чиме се гарантује да ће само једна верзија једанпут да се повуче са интернета и искористи за све фрагменте и странице које је захтевају.

На три осе простора типова (3.3) одлуке овог одељка дају следеће вредности: састављање је **динамичко** (у извршном времену), место је **клијент**, а тип поделе је комбинација и хоризонталног и вертикалног приступа.

11.3 Рутирање

Трећа одлука следи прву и другу без нове слободе: где се поглед саставља, тамо се и рутира, па је рутирање на клијенту.

Оно што је обмотач морао да изгради сам вредно је навођења, зато што одговара на питање колико затечена библиотека покрива. У овом решењу обмотач гради три ствари које библиотека не даје: табелу рута као податак, функцију активације која руту монтира на основу путање, и капију не монтира фрагмент док токен сесије није уписан (изузев фрагмента за ауторизацију и нотификације који мора да функционише и без токена сесије). Прве две су предмет 12.2.

11.4 Комуникација: типизирана магистрала догађаја

Четврта одлука оквира најчешће се поставља као једно питање, „како микрофронтендови разговарају“, и то је погрешно постављено питање. Одлука овог решења није један механизам него **правило по коме се механизам бира**, а правило гласи: механизам одређује *топологија*.

Основ тог правила је у литератури, али разложен на два места која га заједно дају. (Rappl, The Art of Micro Frontends, 2024) у састављању апликација разликује три вида комуникације, догађаје, дељене податке и **пружање компонената**, дакле разликује их по томе *шта* се предаје. (Geers, 2020) исте видове разликује по томе *где* учесници стоје: родитељ према фрагменту, фрагмент према родитељу, фрагмент према фрагменту, и делови који нису на истом екрану.

Табела 11-2 Механизам према топологији, и шта се њиме предаје

Однос учесника	Изабрани механизам	Шта путује	Зашто не магистрала
родитељ → фрагмент	атрибути елемента	три скалара (стање сесије, име, улога)	садржавање већ носи однос; фрагмент је чиста функција својих атрибута
фрагмент → родитељ	прилагођени догађај	само врста намере („одјава“)	родитељ је једини претплатник који по положају може да постоји
фрагмент ↔ фрагмент на истом екрану	типизирана магистрала догађаја	идентификатор изабране дијагнозе	(магистрала <i>јесте</i> избор овде)
преко границе монтирања	адреса странице (параметри из адресе)	идентификатор заказаног прегледа	магистрала не памти, а прималац још није монтиран
међу картицама прегледача	Broadcast Channel API	само врста промене („пријављен“/„одјављен“)	магистрала не прелази границу документа

Обмотач при регистрацији предаје контекст сесије **свакој** апликацији, једнообразно, као својство композиције; то је управо канал родитељ → фрагмент, а предат је свима зато што би обмотач који зна коме шта треба био учесник кога је најтеже поново испоручити.

Дељено складиште у овом решењу **не садржи податке домена**, ниједан преглед, ниједан пацијент, ниједна шифра дијагнозе не пролази кроз њега; сваки фрагмент своје податке чита сам кроз клијента ка серверском слоју. Држи стање сесије и ништа друго. Такође треба нагласити да сесија не припада ниједном поддомену, а свака подела која би је сместила у један чини тај део обавезним за све остале.

11.5 Оквир за одлучивање

Обавеза овог одељка је да га испуњени свако од питања за одлучивање које су изучаване до сад, с тим што испуњавање овде значи три ствари: коју је вредност решење узело, шта је оквир нудио а решење одбило, и **где у изворном коду та одлука стварно живи**. Пета колона је оно што испуњени оквир чини проверљивим: одлука која се не може показати ни у једној датотеци није одлука него намера.

Табела 11-3 Оквир за одлучивање испуњен за MicroMedic

Одлука	Оквир допушта	Изабрано	Одбијено, уз цену	Носилац у коду
Тип поделе	хоризонтална или вертикална	обе, у истом решењу	/	табела рута обмотача (<i>layout, exceptRoutes</i>)
Композиција	клијент, ивица, сервер	клијент , преко <i>Module Federation</i>	сервер (губи прво бојење), ивица (не постоји)	конфигурација градње обмотача (списак порекла)
Рутирање	клијент, ивица, сервер	клијент , и то само у обмотачу што га чини рутирањем првог нивоа	објава руте из фрагмента (састав нечитљив на једном месту)	табела рута и функције активације
Комуникација	емитер догађаја, прилагођени догађаји, веб складиште, низ упита	сва четири, по топологији	/	Табела 12-2

Решење је узело оба типа поделе. Артефакт у истом документу монтира вертикално подељен део (пријава држи цело видно поље) и хоризонтално подељен екран (преглед и каталог упоредо).

Оквир одабира комуникације је испуњен у целини, и то је необично. Оквир наводи четири механизма као списак с кога се бира; решење користи сва четири, зато што су му топологије међу деловима четири различите.

11.6 Серверски слој

Серверски слој остаје један програм с једном базом података. **Алтернатива је постојала и била је очигледна.** Одељак 10.4 је поддомене већ издвојио, па се на серверу нудило исто разлагање које је на клијенту изведено: по један самостално испоручив програм за заказивање, преглед, каталог, регистар лекова, приступ и извештавање. Предмет овог рада је **клијентска** подела. Подела серверског слоја увела би другу независну променљиву у испитивање, па се разлика у испорученом понашању више не би могла приписати ономе што рад испитује.

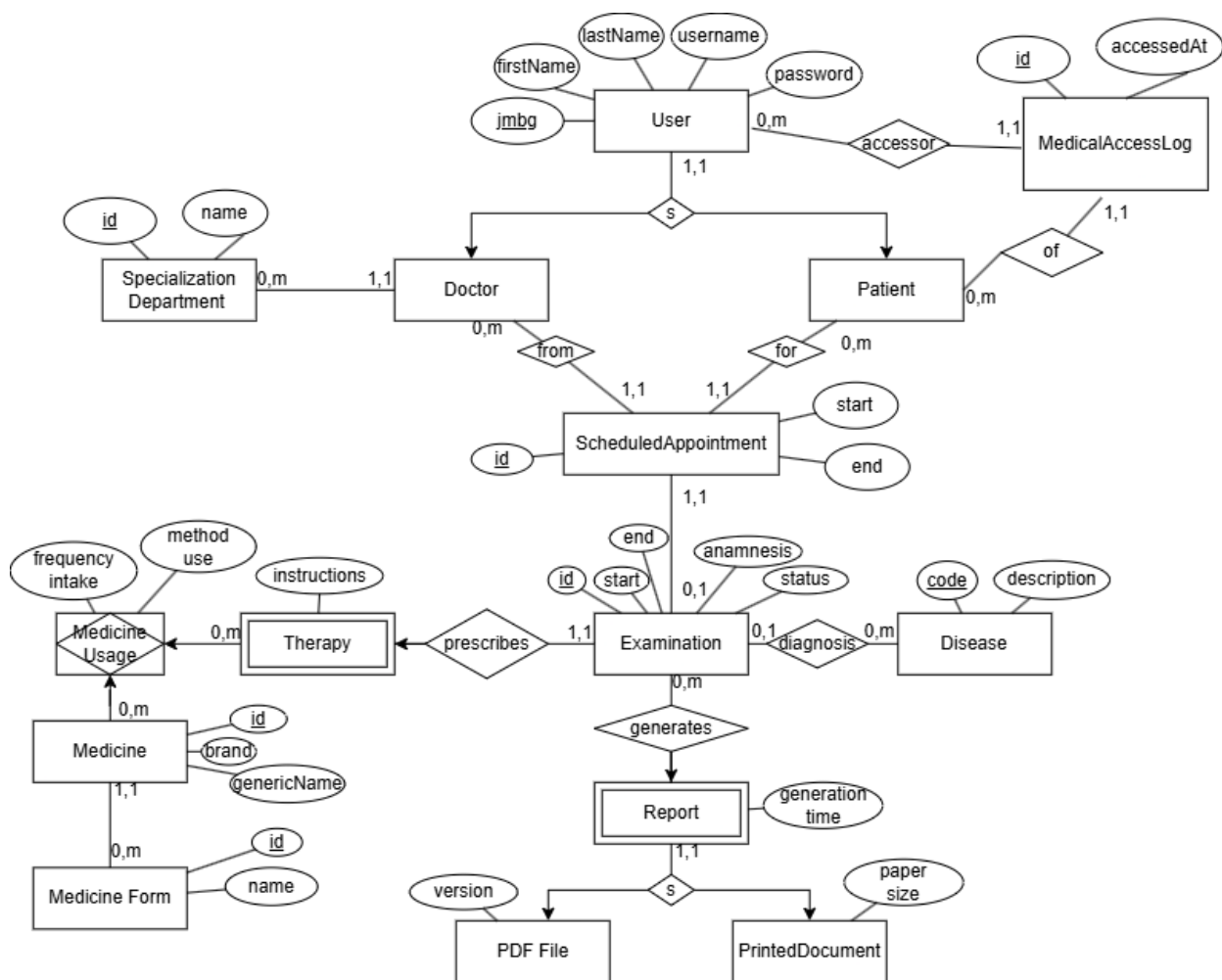
У решењу постоји трострука провера приступа и слојеви се разликују по крупноћи. Прво су подударана путања у поставци безбедности, која су намерно крупна: она деле свет на јавно (три захтева пријаве и три читања референтних података), на оно што захтева улогу лекара (упис заказивања и упис прегледа) и на све остало, које захтева пријављеног корисника. Друго су означавања захтеваног права **на методама сервиса**, у решењу их је једанаест. Треће је чувар

приступа реду података, који спроводи једно правило: лекар сме да прочита пацијента само ако између њих постоји заказан преглед, а пацијент само своје. Управо то правило објашњава зашто је граф објеката из наредног пасуса такав какав је.

Овај троструки распоред је оно што чини целу клијентску поделу безбедном, и вредно је рећи зашто. Шест самостално испоручених фрагмената значи шест места која могу да пошаљу захтев, и ниједно од њих није место на коме се одлучује. Право на сваки појединачни податак одлучује се на серверу, при сваком захтеву, независно од тога који је фрагмент послао захтев. Да је ишта од тога било пренето на клијента решење би било мање безбедно од монолита с којим се пореди.

Траг приступа, и оно што у њега никада не сме да уђе. Обавеза из 10.6 испуњава се засебном табелом трага (енгл. *audit log*), чији је садржај сведен намерно: ко је приступио, у којој улози, којој врсти ресурса, ком ресурсу, чијем пацијенту и када. Шест колона и ништа више, **ниједно име, ниједна дијагноза, ниједна анамнеза, ниједан слободан текст.** Основ тог реза је онај који 10.6 поставља као захтев: траг приступа читају људи који над самим записом немају право, зато што је њихов посао надзор а не лечење. Траг који би цитирао запис био би, дакле, други примерак истог записа под слабијим правилима приступа, тј. управо оно што треба да спречи. Правило се исказује у једној линији: **траг бележи да је приступ био, никада шта је виђено.**

Доменски и релациони модел. Пресликаних класа је тринаест; корисник је апстрактан, с наслеђивањем преко спојених табела и три подтипа, а **улога је функција подтипа а не сачувана колона**, што значи да ниједан упис не може да произведе корисника чија улога не постоји. Четири класе које носе медицински запис имају “меко брисање” (енгл. *soft delete*) и надзорна поља, па се такав запис не уклања него означава. Кључна одлука је, међутим, у облику графа: преглед **не носи** ни пацијента ни лекара непосредно, него их досеже само кроз заказани преглед. То је оно што правило чувара приступа из претходног пасуса чини изводљивим као **један** упит над једном везом, уместо скупа правила по врсти записа.



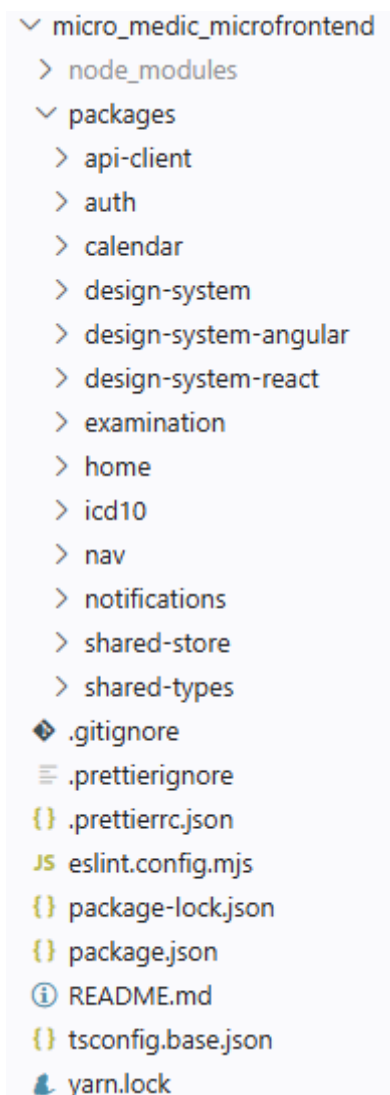
Слика 11-1 Проширени модел објекти везе MicroMedic

12 Имплементација клијентског слоја

Претходно поглавље изложило је одлуке; ово показује шта је од њих изграђено. Предмет је клијентски слој система MicroMedic: један репозиторијум, седам самостално испоручивих фрагмената у четири различита оквира и обмотач који не приказује ништа своје.

12.1 Структура моно-репозиторијума и радни простори

Клијентски слој је један моно-репозиторијум с тринаест пакета под директоријумом *packages/**, корен директоријум држи само наредбе које се преносе на пакете, попис зависности алата за проверу и заједничку основу подешавања *TypeScript* преводиоца. Тврдња да фрагменти нису спрегнути мора се, дакле, доказати нечим другим, графом увоза међу пакетима, а не тиме што их је могуће испоручити из различитих репозиторијума, што овај артефакт не показује.



Слика 12-1 Листа пакета на клијентском слоју

Тринаест пакета се деле на две врсте које се разликују по томе шта производе. **Седам их гради свој артефакт:** сваки има властиту *webpack.config.js* и из ње излази контејнер са именом *remoteEntry.js*. **Шест не гради ништа**, то су библиотеке (уговор о типовима, клијент према серверу, дељено складиште и три пакета система дизајна), а њихова наредба *build* је *tsc --noEmit*, што је само провера. Потрошачи их не увозе као објављене пакете него као изворни код, преко пресликавања путања у *tsconfig.base.json*.

Библиотеке у овом репозиторијуму немају свој испоручиви артефакт, него се уграђују у сваког потрошача, па у времену градње постоји толико њених примерака.

Једна појединост показује шта моно-репозиторијум стварно дели, и она није код. Наредба *wsrn --parallel serve* покреће статички сервер у сваком радном простору који такву наредбу има.

12.2 Апликација обмотач: рутирање, функције активације, капија аутентификације

Обмотач је пакет *home* и он **не приказује ништа своје**: сав његов извор су три датотеке *TypeScript*-а, табела рута, функција активација и покретач.

Табела рута је представљена кроз наредни код (*packages/home/src/routes.ts*):

```
const CHROME_HIDDEN_ROUTES = ['/login', '/register'] as const;
export const ROUTES: readonly RouteEntry[] = [
  { name: 'header', load: () => import('nav/Header'),
    routes: ['*'], exceptRoutes: CHROME_HIDDEN_ROUTES },
  { name: 'footer', load: () => import('nav/Footer'),
    routes: ['*'], exceptRoutes: CHROME_HIDDEN_ROUTES },
  { name: 'notifications',
    load: () => import('notifications/Notifications'),
    routes: ['*'], public: true },

  /*
   * ---- Vertical split ----
   { name: 'auth', load: () => import('auth/Auth'),
     routes: [LOGIN_ROUTE, '/register'], layout: 'full', public: true },

  /*
   * ---- Horizontal split ----
   { name: 'examination', load: () => import('examination/Examination'),
     routes: ['/examination'] },
   { name: 'icd10', load: () => import('icd10/ICD10'),
     routes: ['/examination'] },
   { name: 'calendar', load: () => import('calendar/Calendar'),
     routes: [DEFAULT_ROUTE] } ];
```

Четири поља табеле рута носе одлуке које другде имају последице. **Име** апликације истовремено је и место монтирања, зато што библиотека монтира у елемент `#single-spa-application:<име>`; то значи да су ти називи уговор с документом обмотача, и да су *header* и *footer* названи по положајима а не по пакету *nav* из кога долазе. **Учитавање је функција**, а не већ учитан модул, и то омогућава лењо читавање (енгл. *lazy loading*).

```
<!DOCTYPE html>
<html lang="en">
  <head>
  </head>
  <body>
    <div id="single-spa-application:auth"></div>

    <div class="mm-container">
      <div id="single-spa-application:header"></div>
      <main id="mm-main" tabindex="-1">
        <div class="mm-split mm-split--primary mm-region">
          <div id="single-spa-application:examination"></div>
          <div id="single-spa-application:icd10"></div>
        </div>

        <div id="single-spa-application:calendar" class="mm-region"></div>
      </main>

      <div id="single-spa-application:footer"></div>
    </div>
    <div id="single-spa-application:notifications"></div>
  </body>
</html>
```

Поклапање се врши по целим сегментима и провера пријаве се користи као повратни позив (packages/home/src/activity.ts, скраћено):

```
export function matchesRoute(pathname: string, route: string): boolean {
  if (route === '*') return true;
  const normalized = route.length > 1 && route.endsWith('/') ? route.slice(0, -1) :
route;
  return pathname === normalized || pathname.startsWith(`${normalized}/`);
}

export function createActivityFn(entry: RouteEntry, isAuthenticated: () => boolean):
(location: Location) => boolean {
  return (location: Location): boolean => {
    if (!entry.routes.some(route => matchesRoute(location.pathname, route))) return
false;
    if (entry.exceptRoutes?.some(route => matchesRoute(location.pathname, route)))
return false;
    return entry.public === true || isAuthenticated();
  }; };
```

Прво: поређење је по **целим сегментима** пута, а не почетком низа. Пуко поклапање `startsWith` методе учинило би да се `/calendar` поклопи са `/calendar-archive`, а `/log` да се поклопи са `/login`, дакле да се апликација монтира на страници о којој ништа не зна. Друго, и важније: `isAuthenticated` предаје се као **повратни позив**, а не као **логичка вредност**. Библиотека ову функцију саставља једном, а позива је при свакој промени руте; ухваћена логичка вредност била би, стога, оно што је сесија била при покретању документа, за читав живот документа.

Однос према 4.5 треба исказати без ублажавања. Тамо су изложена два нивоа рутирања, а овде је изведено **само рутирање обмотача**: ниједан фрагмент не држи свој рутер. Рутирање на два нивоа овде, дакле, није одбијено као лошије, него није било потребно.

12.3 Топологија микрофронтендова

Табела која следи је попис изведене топологије од седам пакета (обмотач и шест које он региструје).

Табела 12-1 Топологија микрофронтендова у изведеном систему

Порт	Пакет	Име контејнера	Изложени модули	Оквир
3001	home	home	— (чист домаћин)	TypeScript, single-spa 6
3002	icd10	icd10	./ICD10	Vue 3
3003	nav	nav	./Header, ./Footer	прилагођени елементи
3004	examination	examination	./Examination	Angular 22
3006	auth	auth	./Auth	React 19
3007	notifications	notifications	./Notifications	прилагођени елементи
3009	calendar	calendar	./Calendar	React 19

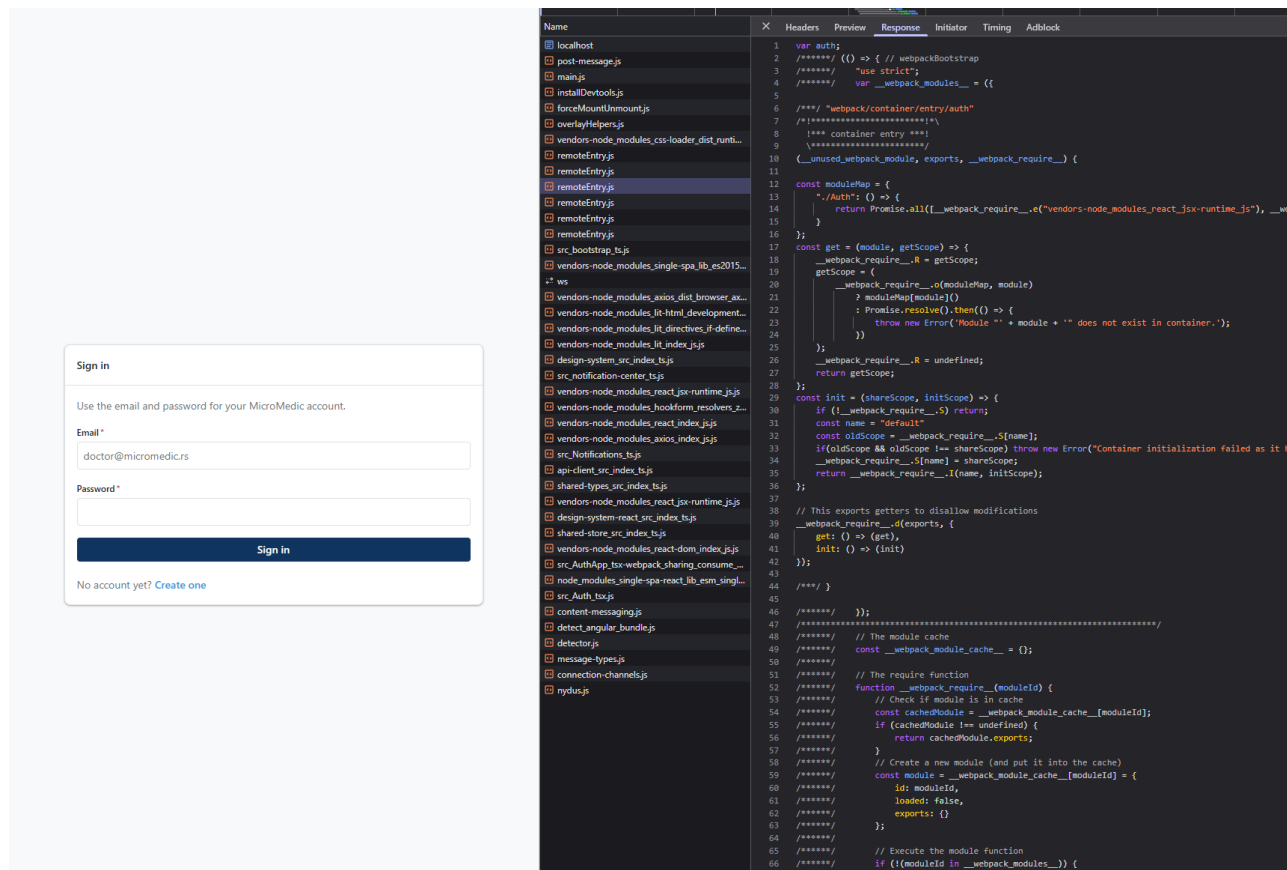
Удаљени модули се дефинишу у облику `име@адреса`, зарад лењог учитавања (енгл. *lazy loading*) (`packages/home/webpack.config.js`, скраћено):

```
const REMOTES = {
  nav: 'nav@http://localhost:3003/remoteEntry.js',
  notifications: 'notifications@http://localhost:3007/remoteEntry.js',
  icd10: 'icd10@http://localhost:3002/remoteEntry.js',
  examination: 'examination@http://localhost:3004/remoteEntry.js',
  calendar: 'calendar@http://localhost:3009/remoteEntry.js',
  auth: 'auth@http://localhost:3006/remoteEntry.js'
};

new ModuleFederationPlugin({
  name: 'home', library: { type: 'var', name: 'home' },
  filename: 'remoteEntry.js', remotes: REMOTES,
  remoteType: 'script', shared: { 'single-spa': { singleton: true } })
```

Облик име@адреса је сопствени лењи облик градње: скрипта контејнера уводи се при **првом** увозу, а удаљени модул се преузима само на рути која га монтира.

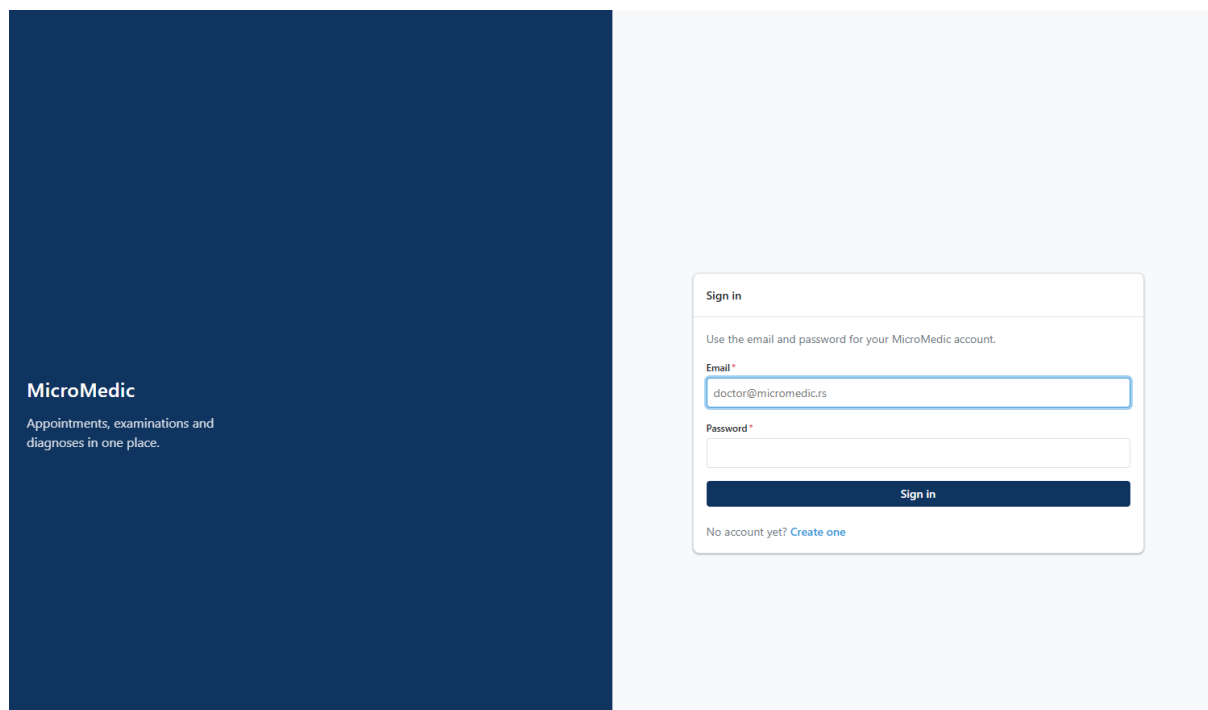
На слици се види читавање главног документа који са собом повлачи auth страницу као и дељених зависности као што су design-system, api-client, react, single-spa-react.



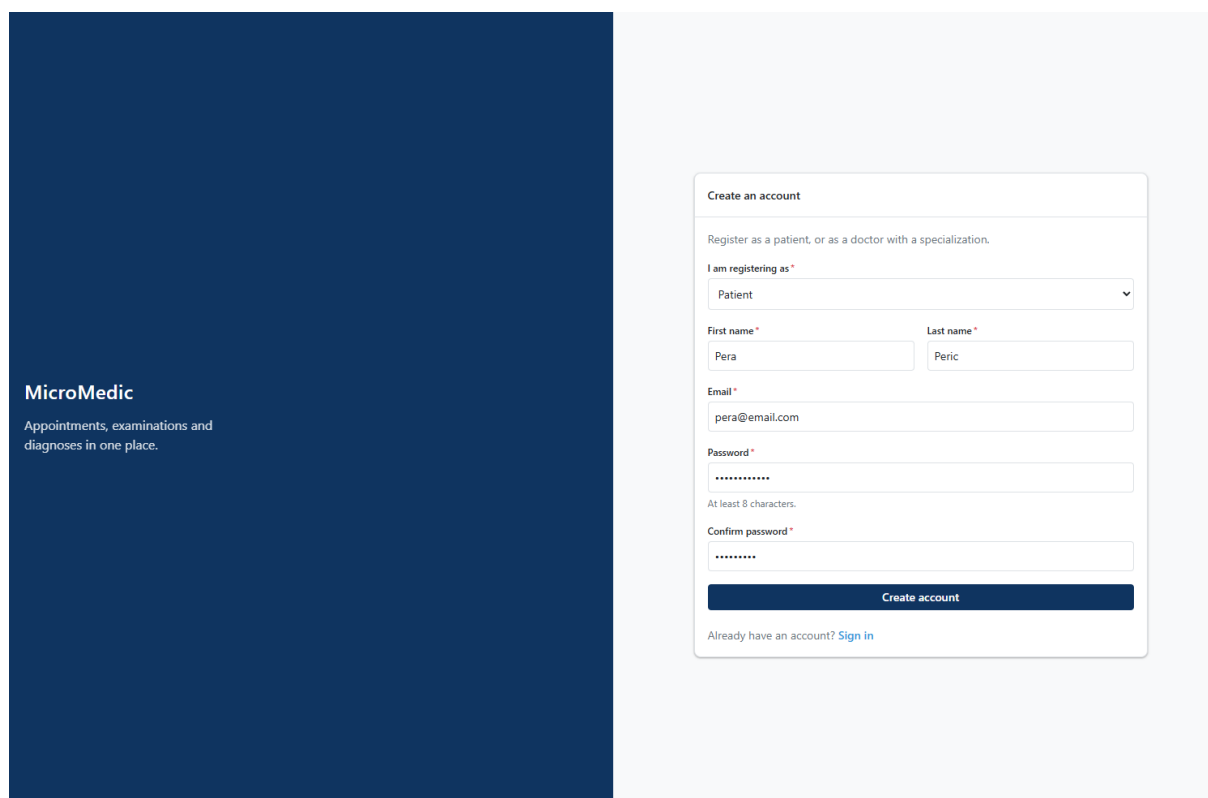
Слика 12-2 Иницијално повлачење зависности и auth пакета

12.4 Вертикална подела: пакет auth

Пакет auth јесте изведени пример вертикалне поделе. На рутама /login и /register он **није** део састављене странице, него **јесте** страница: у том се и састоји разлика према хоризонталној подели из 12.5, где два фрагмента деле једно видно поље.



Слика 12-3 Страница за пријаву



Слика 12-4 Страница за регистрацију

12.5 Хоризонтална подела: examination уз icd10

На рути /examination обмотач саставља два независно испоручена фрагмента у једно видно поље: образац за унос прегледа, који је Angular 22 на порту 3004, и каталог болести по класификацији МКБ-10, који је Vue 3 на порту 3002.

Основ поделе је **доменски, а не технички**. Лекар при прегледу пише анамнезу, а успут се **консултује** са шифарником дијагноза. То су, дакле, два различита посла која се одвијају у исто време на истом екрану, и подела прати њих.

Хоризонтална подела у документу обмотача се може видети кроз два div тага која се налазе један поред другог у HTML дефиницији (packages/home/public/index.html, скраћено):

```
<main id="mm-main" tabindex="-1">
  <div class="mm-split mm-split--primary mm-region">
    <div id="single-spa-application:examination"></div>
    <div id="single-spa-application:icd10"></div>
  </div>
</main>
```

The screenshot displays the MicroMedic Examination form, which is horizontally split into two main sections. The left section, titled 'Examination', contains patient information (Name: Veljko Blagojevic, Email: veljkoblagojevic008@gmail.com, Appointment: Aug 25, 2026, 1:00 AM - 1:30 AM, SCHEDULED), a list of previous examinations, an 'Examination started' timestamp (25-Aug-2026 02:05), an 'Anamnesis' field with a text entry about a headache, a 'Diagnosis' field with a selected code G44311, 'Therapy instructions' (Rest, try to sleep as much as possible), and a table of 'Prescribed medicines' (MAXIDINE and BRUFEN). The right section, titled 'ICD-10 diagnoses', shows a search for 'headache' resulting in 41 matches, with a list of codes including G44311, G44319, G44021, G44029, and G44321. The form is recorded as 'Miodrag Grujic' and includes a 'Sign out' button.

Слика 12-5 Хоризонтална подела examination и icd10 фрагмента

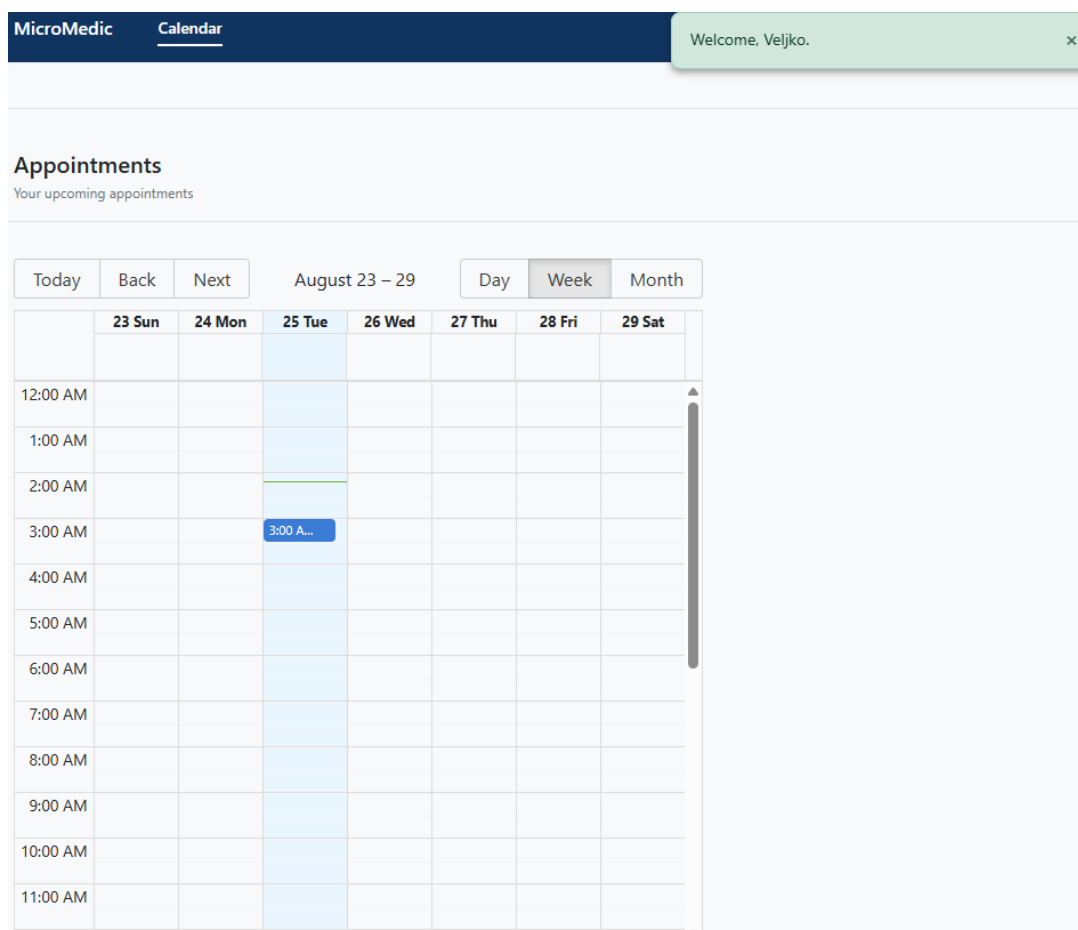
На слици је у црвеном оквиру са леве стране приказан фрагмент examination, док је на истом екрану са десне стране, уоквирен у жуто, фрагмент под називом ICD10 (МКБ-10 међународна класификација болести).

Ниједан од двају фрагмената не увози други, и ниједан то не може. Прво, у изворном коду ниједног од двају пакета нема увоза оног другог, једина спрега је магистрала догађаја и заједнички елементи система дизајна. Друго, и јаче: Angular компонента **не може** увести Vue компоненту, зато што су то два различита система приказа с различитим животним циклусима. Ту је, дакле, прегледач једино могуће место интеграције; тиме је договорена дисциплина замењена физичком препреком.

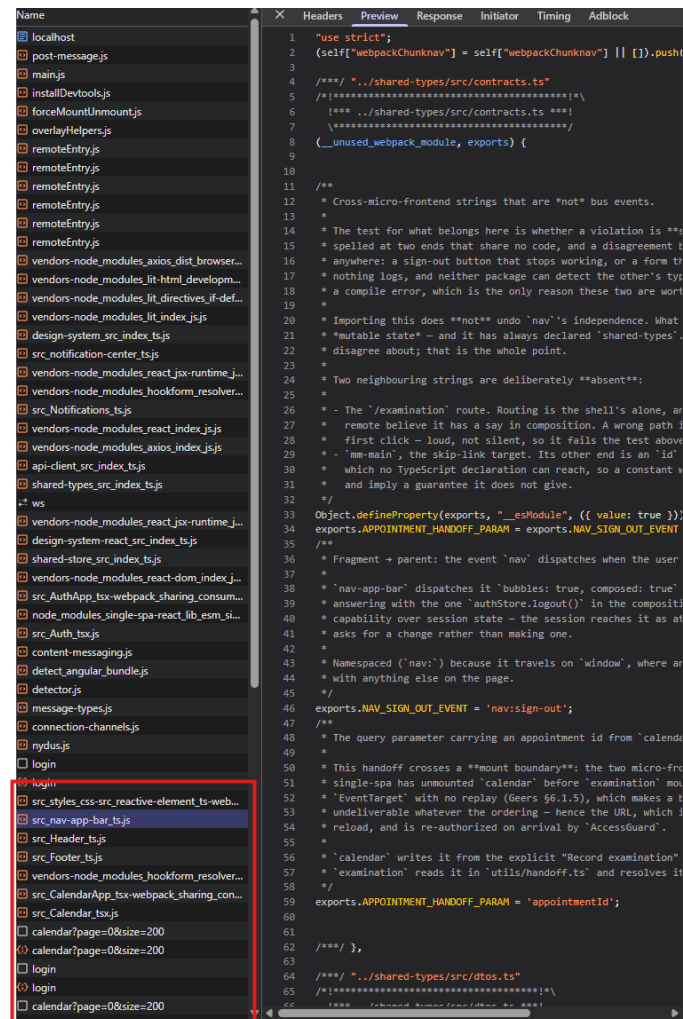
Уговор између њих је један догађај: каталог објављује изабрану болест, а форма је прима и приказује у свом делу за дијагнозу.

12.6 Увек монтирани фрагменти: nav и notifications

Два фрагмента нису везана ни за једну руту: заглавље с подножјем, у пакету nav на порту 3003, и слој обавештења, у пакету notifications на порту 3007. Оба су написана као **прилагођени елементи у чистом језику TypeScript, без иједног радног оквира.**



Слика 12-6 Приказ nav и notifications фрагмента након пријаве



Слика 12-7 Учитавање calendar, nav, header и footer скрипти тек након пријаве корисника

Навигациони банер не чита дељено складиште. Стање сесије прима као својства елемента: да ли је корисник пријављен, његово име и улога, а њих поставља обмотач апликација. Тиме фрагмент постаје чиста функција својих својстава, а писац сесије остаје на једном месту.

Дугме за одјаву на навигационом банеру не дира сесију. Оно објављује догађај који се издиже, а обмотач га ослушкује на прозору и тек он изводи одјаву над дељеним складиштем. Тиме фрагмент саопштава намеру, а не последицу, и остаје без права уписа у сесију.

Слој обавештења има само две сопствене одлуке, и обе су доменске а не приказне. Прва је трајање по типу, где је **грешка трајна док се не отклони**, а порука о успеху се сама склања након неколико секунди. Друга је да се читав низ обавештења **брише при одјави**, зато што порука може носити име пацијента из сесије која је управо престала, из чега следи да је ово истовремено и безбедносна одлука.

12.7 Комуникација између фрагмената: изведени механизми

Механизми су изложени у 5.1–5.4, а изабрани у 11.4; овде стоји само шта је од тога изведено, где то стоји на изворном коду.

Табела 12-2 Изведени механизми комуникације и место сваког у артефакту

Механизам	Смер	Место у артефакту
Својства елемента	обмотач → фрагмент	сесија у header заглавље (nav/src/session-attributes.ts)
Издигући прилагођени догађај	фрагмент → обмотач	одјава (nav:sign-out, обрађена у home/src/index.ts)
Магистрала догађаја	фрагмент → фрагмент суседи на истом екрану	избор дијагнозе (icd10 → examination)
Јединствени показивач ресурса	преко границе монтирања тј. уз помоћ параметара из адресе	предаја заказаног прегледа (calendar → examination)
Broadcast Channel API	међу картицама истог корисника	измена сесије (shared-store/src/session-channel.ts)

Први ред представља комуникацију где обмотач сесију понуди **сваком** регистрованој фрагменту, једнообразно. Код за дати механизам је приказан у листингу са `projectSession` методом. Наредни листинг показује преводјење стања сесије у својства елемента.

Атрибути сесије у коду се разрешавају (packages/nav/src/session-attributes.ts, скраћено):

```
export function toAttributes(session: SessionSnapshot): AttributeMap {
  const authenticated = session.isAuthenticated && !!session.user;
  const name = session.user ? `${session.user.firstname}
${session.user.lastname}`.trim() : '';

  return {
    [ATTR_AUTHENTICATED]: authenticated ? '' : null,
    [ATTR_USER_NAME]: authenticated && name ? name : null,
    [ATTR_USER_ROLE]: authenticated ? (session.role ?? session.user?.role ?? null)
: null,
  };
}

export const projectSession: AttributeProjector = (props: CustomElementMountProps):
AttributeSource | null => {
  const session = props.session;
  if (session === undefined || session === null) return null;
  if (!isSessionContext(session)) {
    return { get: () => toAttributes(SIGNED_OUT), subscribe: () => () => {} };
  }
  return {
    get: () => toAttributes(session.getState()),
    subscribe: (onChange) => session.subscribe(onChange),
  };
};
```

Други ред приказан је у исечку кода који користи прилагођени именовани догађај (packages\home\src\index.js и packages\nav\src\nav-app-bar.ts, скраћено):

```
// компонента навигације
this.dispatchEvent(new CustomEvent(NAV_SIGN_OUT_EVENT, { bubbles: true, composed:
true }));

// index.ts
window.addEventListener(NAV_SIGN_OUT_EVENT, () => {
  authStore.logout();
});
```

Трећи ред је имплементиран уз помоћ сервиса који руководи са магистралом догађаја (packages\examination\src\state\event-bus.service.ts).

```
@Injectable({ providedIn: 'root' })
export class EventBusService {
  private readonly destroyRef = inject(DestroyRef);

  listen<T extends EventType>(
    event: T,
    listener: (payload: EventPayloadMap[T]) => void
  ): void {
    const unsubscribe = eventBus.on(event, listener);
    this.destroyRef.onDestroy(unsubscribe);
  }

  emit<T extends EventType>(event: T, ...args: EmitArgs<T>): void {
    eventBus.emit(event, ...args);
  }

  notify(type: 'success'|'error'|'info'|'warning', message: string): void {
    this.emit(EventTypes.NOTIFICATION_SHOW, { message, type });
  } }

this.bus.listen(EventTypes.ICD10_DISEASE_SELECTED, ({ disease }) => {
  this.diagnosisSignal.set(disease);
  this.submitErrorSignal.set(null);
});
```

MicroMedic Calendar Examination Miodrag Grujicic Doctor Sign out

Examination

Recording as Miodrag Grujicic

Patient

Name Veljko Blagojevic
Email veljko.blagojevic008@gmail.com
Appointment Aug 25, 2026, 3:00 AM – 3:30 AM **SCHEDULED**

Previous examinations

Aug 21, 2026, 10:01 PM	C9102 — Acute lymphoblastic leukemia, in relapse
Aug 18, 2026, 7:38 PM	O0480 — (Induced) termination of pregnancy with unspecified complications
Aug 18, 2026, 7:03 PM	Z3A13 — 13 weeks gestation of pregnancy
Aug 18, 2026, 6:37 PM	O0480 — (Induced) termination of pregnancy with unspecified complications
Aug 18, 2026, 6:34 PM	O0480 — (Induced) termination of pregnancy with unspecified complications

Showing the 5 most recent of 7.

Examination started *

25-Aug-2026 02:12

Defaults to now. Cannot be in the future.

Anamnesis *

What the patient reports — symptoms, onset, history...

Stored as the examination's anamnesis.

Diagnosis *

G912 (Idiopathic) normal pressure hydrocephalus

ICD-10 diagnoses

Select a code to attach it to the examination.

Search ICD-10

Code or description — J06, bronchit

SENT TO EXAMINATION

G912
(Idiopathic) normal pressure hydrocephalus
Clear

G912
(Idiopathic) normal pressure hydrocephalus

O0489
(Induced) termination of pregnancy with other complications

O0480
(Induced) termination of pregnancy with unspecified complications

Z3A10
10 weeks gestation of pregnancy

Z3A11

Previous Page 1 of 7171 (71704 codes) Next

Слика 12-8 Одабир дијагнозе у icd10 фрагменту окида догађај у магистрали који examination фрагмент ослушкује и уписује у формулар

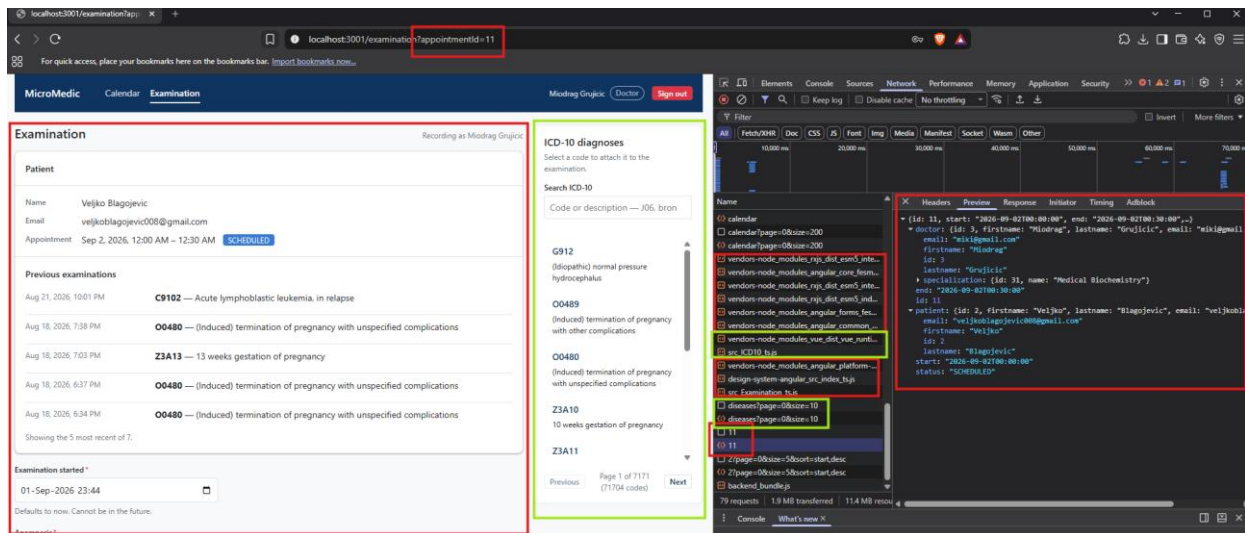
Четврти ред приказан кроз једноставно слање и читавање идентификатора као параметра из адресе (packages\calendar\src\CalendarApp.tsx, packages\examination\src\ExaminationApp.ts):

```
const onRecordExamination = useCallback(
  (appointment: ScheduledAppointmentDto) => {
    navigate(`/examination?${APPOINTMENT_HANDOFF_PARAM}=${appointment.id}`);
  },
  [navigate]
);

export function readAppointmentHandoff(): number | null {
  if (typeof window === 'undefined') return null;

  const raw = new
  URLSearchParams(window.location.search).get(APPOINTMENT_HANDOFF_PARAM);
  if (raw === null || raw.trim() === '') return null;
  const id = Number(raw);
  if (!Number.isInteger(id) || id <= 0) return null;

  return id; }
```



Слика 12-9 Параметар из адресе као предајни податак при транзицији и комуникацији calendar и examination фрагмента

Пети ред

```
export interface SessionMessage {
  type: 'signed-in' | 'signed-out';
}

export interface SessionChannel {
  publish(message: SessionMessage): void;
}

const CHANNEL_NAME = 'micro_medic:session';

function isSessionMessage(value: unknown): value is SessionMessage {
  if (typeof value !== 'object' || value === null) return false;
  const { type } = value as { type?: unknown };
  return type === 'signed-in' || type === 'signed-out';
}

export function openSessionChannel(
  onMessage: (message: SessionMessage) => void
): SessionChannel | null {
  if (typeof BroadcastChannel === 'undefined') return null;

  let channel: BroadcastChannel;
  try {
    channel = new BroadcastChannel(CHANNEL_NAME);
  } catch (error) {
    console.warn('[SessionChannel] Could not open the session channel:',
error);
    return null;
  }

  channel.addEventListener('message', (event: MessageEvent<unknown>) => {
```



```

        if (isSessionMessage(event.data)) onMessage(event.data);
    });

    return {
        publish(message: SessionMessage): void {
            try {
                channel.postMessage(message);
            } catch (error) {
                console.warn('[SessionChannel] Could not publish a session
message:', error);
            }
        },
    };
}

export interface AuthState {
    token: string | null;
    user: UserDto | null;
    role: Role | null;
    isAuthenticated: boolean;
}

function readStoredUser(): UserDto | null {
    let raw: string | null = null;
    try { raw = localStorage.getItem(USER_KEY); } catch { return null; }
    if (!raw) return null;

    try {
        const parsed = JSON.parse(raw) as UserDto | null;
        return parsed && typeof parsed === 'object' ? parsed : null;
    } catch {
        console.warn('[AuthStore] Discarding unparseable persisted user.');
        localStorage.removeItem(USER_KEY);
        return null;
    }
}

function readStoredToken(): string | null {
    try { return localStorage.getItem(TOKEN_KEY); }
    catch { return null; }
}

function createAuthStore() {
    let token: string | null = readStoredToken();
    let user: UserDto | null = readStoredUser();
    let isAuthenticated: boolean = !!token && !!user;
    const subscribers: Set<Subscriber> = new Set<Subscriber>();

    let sessionChannel: SessionChannel | null = null;

```

```

configureApiClient({
  getAuthToken: () => token,
  onUnauthorized: () => store.logout(),
});

function getState(): AuthState {
  return { token, user, role: user?.role ?? null, isAuthenticated };
}

function notify(): void {
  const snapshot = getState();
  subscribers.forEach((subscriber) => {
    try { subscriber(snapshot); }
    catch (error) { console.error('[AuthStore] Error in subscriber
callback:', error); }
  });
}

function persist(): void {
  if (token) localStorage.setItem(TOKEN_KEY, token);
  else localStorage.removeItem(TOKEN_KEY);

  if (user) localStorage.setItem(USER_KEY, JSON.stringify(user));
  else localStorage.removeItem(USER_KEY);
}

const store = {
  getState,

  login(newToken: string, newUser: UserDto): void {
    token = newToken; user = newUser; isAuthenticated = true;
    persist(); notify();
    eventBus.emit(EventTypes.AUTH_LOGIN, { user: newUser });
    sessionChannel?.publish({ type: 'signed-in' });
  },

  logout(): void {
    token = null; user = null; isAuthenticated = false;
    persist(); notify();
    eventBus.emit(EventTypes.AUTH_LOGOUT);
    sessionChannel?.publish({ type: 'signed-out' });
  },

  subscribe(subscriber: Subscriber): () => void {
    subscribers.add(subscriber);
    return () => { subscribers.delete(subscriber); };
  },
};

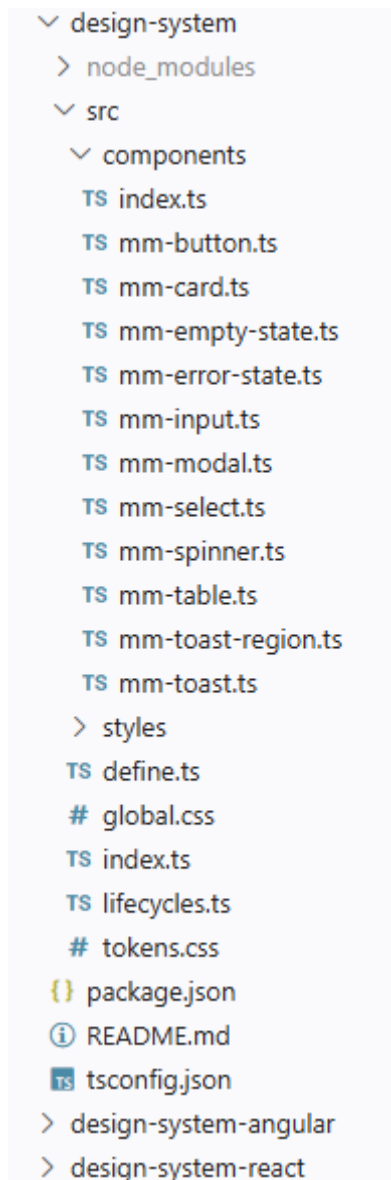
```

[illegible]

12.8 Систем дизајна: једна изведба, четири окружења

83

увоза пакета, а не позив који потрошач изводи, што значи да фрагмент који жели елементе увезе пакет и то је сав његов посао.



Слика 12-11 Списак компоненти унутар система дизајна

Пример једне компоненте, дугмета, је приказана у наредном листингу. Потребно је да имплементира `render()` методу наслеђену од родитељске `LitElement` класе. Све остале прилагођене компоненте прате сличан приступ.

```
import { LitElement, html, css } from 'lit';

export type ButtonVariant = 'primary' | 'secondary';
export type ButtonSize = 'sm' | 'md' | 'lg';
export type ButtonType = 'button' | 'submit' | 'reset';

export class MmButton extends LitElement {
  accessor variant: ButtonVariant = 'primary';
  accessor size: ButtonSize = 'md';
```

```

accessor disabled = false;
accessor loading = false;
accessor type: ButtonType = 'button';
accessor label = '';

private handleClick(event: Event) {
  if (this.disabled || this.loading) {
    event.preventDefault();
    event.stopImmediatePropagation();
    return;
  }
  if (this.type === 'submit' || this.type === 'reset') {
    const form = this.closest('form');
    if (form) {
      event.preventDefault();
      if (this.type === 'submit') {
        form.requestSubmit();
      } else {
        form.reset();
      }
    }
  }
}

render() {
  const busy = this.disabled || this.loading;
  return html`
    <button
      class="variant-${this.variant} size-${this.size}"
      type=${this.type}
      ?disabled=${busy}
      aria-busy=${this.loading ? 'true' : 'false'}
      aria-label=${ifDefined(this.label ?? undefined)}
      @click=${this.handleClick}
    >
      ${this.loading
        ? html`<span class="spinner" part="spinner"></span>`
        : html`<slot>${this.label}</slot>`}
    </button>
  `;
}

defineElement('mm-button', MmButton);

```

Како су ово елементи који су написани по стандарду Веб компоненти, могу се користити у било којим радним оквирима али је потребан одређен вид адаптације. За React радни оквир потребно је користити `createComponent` методу. Међутим, како сваки фрагмент који је написан у React радном оквиру, не би морао да врши ово адаптацију, направљен је посреднички пакет,

као централно место, који сви фрагменти имплементирани у React радном оквиру могу да користе. Исечак кода који приказује ту адаптацију је у листингу (packages/design-system-react/src/create-component.ts, скраћено):

```
export function createComponent<
  TElement extends HTMLElement,
  TEvents extends Record<string, EventName | string> = {},
>(tagName: string, elementClass: { new (): TElement; prototype: TElement },
events?: TEvents) {
  return litCreateComponent({
    react: React,
    tagName,
    elementClass,
    events,
  });
}

export const MmButton = createComponent('mm-button', MmButtonElement);

export const MmInput = createComponent('mm-input', MmInputElement, {
  onInput: 'mm-input',
  onChange: 'mm-change',
  onBlur: 'mm-blur',
} as const);
```

Сличан приступ имплементације посредничког пакета је искоришћен и за Angular варијанту:

```
@Directive()
export abstract class MmElementDirective<TElement extends HTMLElement> implements
OnChanges {
  protected readonly elementRef = inject<ElementRef<TElement>>>(ElementRef);
  get nativeElement(): TElement {
    return this.elementRef.nativeElement;
  }
  protected readonly nonForwardedInputs: ReadonlySet<string> = new Set<string>();
  ngOnChanges(changes: SimpleChanges): void {
    const element = this.elementRef.nativeElement as unknown as Record<string,
unknown>;

    for (const [name, change] of Object.entries(changes)) {
      if (this.nonForwardedInputs.has(name)) continue;
      if (change.currentValue === undefined) continue;
      element[name] = change.currentValue;
    }
  }
  protected forward<T>(emitter: EventEmitter<CustomEvent<T>>, event: Event): void
{ emitter.emit(event as CustomEvent<T>);}
```

Vue варијанта је могла да буде имплементирана на еквивалентан начин, међутим, као показни пример није. Vue је овде контролни случај, он непримитивну вредност поставља као својство објекта у документу, а ослушкивача догађаја региструје на уобичајен начин, па му посреднички пакет не треба и **design-system-vue намерно не постоји**. Од њега је потребно само једно правило при изградњи (`packages/icd10/webpack.config.js`, скраћено):

```
compilerOptions: {isCustomElement: (tag) => tag.startsWith('mm-')}
```

Кумулацију пакета система дизајна и њихово коришћење кроз фрагменте у MicroMedic систему можемо видети формализовано кроз табелу 13'3.

Табела 12-3 Адаптација по потрошачу и недостатак који надомешта

Потрошач	Адаптација
nav, notifications (прилагођени елементи)	ништа
icd10 (Vue 3)	једно правило при изградњи и једна изјава типова
calendar, auth (React 19)	Посреднички пакет design-system-react
examination (Angular 22)	Посреднички пакет design-system-angular

12.9 Дељено стање и заједнички приступ сервисима

Дељено променљиво стање је у 9.4 наведено међу анти-патернима, па је изведба овде дужна да покаже под којим је условима је имплементирано. Три ограничења су оно чиме је ово стање издвојено од анти-патерна из 9.4.

- Прво, **садржај је само сесија**, ниједан доменски податак није унутар дељеног стања, па не постоји ништа што би два фрагмента могла да затекну у стању осим тога ко је пријављен, нити да имају имплицитну комуникацију преко дељеног стања.
- Друго, **приступ је намерно закључан**. Сви фрагменти осим писаца добијају „замрзнуту копију“ (путем *JavaScript* методе *Object.freeze({object})*) објекта стања сесије, дакле вредност у коју се не може уписати. Претплата на промену и читање тренутног стања јесу цео њихов приступ.
- Треће, **писци** су именовани и **малобројни**: фрагмент за пријаву уписује сесију при успешној пријави, обмотач је брише при одјави, а клијент према серверском слоју је брише када серверски слој одбије захтев због неважећег токена. Ниједан други фрагмент не може да упише, зато што му се предаје замрзнута копија без могућности уписа.

12.10 Вишеструки оквири као намерни оглед

Одељак 9.3 вишеструке оквири без механизма управљања назива анархијом. MicroMedic систем који у себи носи **три радна оквира, једну библиотеку прилагођених елемената и два фрагмента без оквира**, React 19, Angular 22, Vue 3, библиотеку Lit и, у два фрагмента, ниједан радни оквир, па је дужан да покаже управљање, а не само исход.

Механизма управљања који су у артефакту стварно на месту је четири, и сваки од њих спречава по један одређен облик анархије:

1. **Један систем дизајна за све оквири (12.8).** Без њега сваки оквир доноси властиту колекцију компонената, што је облик анархије који се први види, визуелно на корисничком интерфејсу.
2. **Уговори су дефинисани коришћењем типовима.** Без њега свака страна сама претпоставља облик онога што прима, па неусклађеност постаје видљива тек у извршном времену.
3. **Један рутер и једно место састављања (12.2).** Ниједан удаљени модул не одлучује о својој рути.
4. **Једна обавезна провера у испоруци (12.11),** с једном наредбом за све пакете.

Четврта ставка носи налаз који је нађен у изведби и који је, у управљању вишеструким оквирима, лако промашити: **јачина управљања је јачина најслабије провере.**

Пет оквира у једном документу јесте пет извршних окружења за учитавање, а понављање заједничких зависности међу самостално испорученим садржаоцима је трошак који дељење ублажава али не укида. Тврдња овог поглавља није да је овај приступ бесплатан и без последица, него да је **надзиран**.

12.11 Остварене безбедносне мере

Остварено је неколико безбедносних мера, на страни и клијента и сервера, независно.

На страни сервера (11.6) је имплементирано више битнијих мера:

- одобрени попис извора за унакрсне захтеве CORS, сведен на портове фрагмената;

```
@Configuration
public class CorsConfig {

    @Value("${app.cors.allowed-origins}")
    private String allowedOrigins;
```



```

@Bean
public CorsConfigurationSource corsConfigurationSource() {
    org.springframework.web.cors.CorsConfiguration configuration = new
org.springframework.web.cors.CorsConfiguration();
    List<String> parsedAllowedOrigins =
Arrays.stream(allowedOrigins.split(","))
        .map(String::trim)
        .filter(origin -> !origin.isBlank())
        .toList();
    configuration.setAllowedOrigins(parsedAllowedOrigins);
    configuration.setAllowedMethods(java.util.List.of("GET", "POST",
"PUT", "DELETE", "OPTIONS"));
    configuration.setAllowedHeaders(java.util.List.of("*"));
    configuration.setAllowCredentials(true);
    org.springframework.web.cors.UrlBasedCorsConfigurationSource
source = new
org.springframework.web.cors.UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/**", configuration);
    return source;
}
}

```

- одобрење на методама сервиса а не на самим контролерима;

```

@Slf4j
@Service
@RequiredArgsConstructor
public class CalendarService {

    public static final int APPOINTMENT_MAX_DURATION_HOURS = 2;
    private final ScheduledAppointmentRepository
scheduledAppointmentRepository;
    private final AccessGuard accessGuard;

    @PreAuthorize("hasAuthority('ROLE_DOCTOR')")
    @Transactional
    public ScheduledAppointment createAppointment(AppointmentRequest
appointment) {
        if (appointment == null) {
            throw new IllegalArgumentException("Appointment request cannot
be null");
        }

        if (appointment.end().isBefore(appointment.start())) {
            throw new IllegalArgumentException("Appointment end time
cannot be before start time");
        }

        if
(appointment.end().isAfter(appointment.start().plusHours(APPOINTMENT_MAX_D
URATION_HOURS))) {
            throw new IllegalArgumentException("Appointment duration
cannot exceed " + APPOINTMENT_MAX_DURATION_HOURS + " hours");
        }

        ...
        return scheduledAppointmentRepository.save(scheduledAppointment);
    }
}

```

- провера власništva над редом података, која при сваком приступу медицинском податку уписује и запис у траг приступа;

```
@Service
@RequiredArgsConstructor
public class AccessGuard {

    private final UserService userService;
    private final MedicalAccessRecorder recorder;

    ...

    @Transactional
    public void requirePatientAccess(Long patientId) {
        User current = userService.getCurrentUser();
        boolean isSelf = current instanceof Patient &&
current.getId().equals(patientId);
        boolean isTreatingDoctor = current instanceof Doctor
&&
scheduledAppointmentRepository.existsByDoctorIdAndPatientId(current.getId(
), patientId);

        if (isSelf || isTreatingDoctor || isAdmin(current)) {
            recorder.record
(MedicalAccessLog.AccessedResourceType.PATIENT, patientId, patientId,
current);
            return;
        }

        throw new UnauthorizedActionException("User does not have access
to patient with ID: " + patientId);
    }
}
```

- одбијање покретања апликације ако тајна за потпис токена није ваљана;

```
@Service
public class JwtSecretValidator {

    private static final int MIN_KEY_BYTES = 32; // H256
    @Value("${app.jwt.secret}")
    private final String secretKey;

    ...

    @PostConstruct
    void validate() {

        if (secretKey == null || secretKey.trim().isEmpty()) {
            fail("app.jwt.secret is missing or blank. Set the JWT_SECRET env
variable to a Base64-encoded key of at least 256 bits");
            return;
        }

        int keyBytes;
        try {
            keyBytes = java.util.Base64.getDecoder().decode(secretKey).length;
        } catch (RuntimeException e) {
            fail("app.jwt.secret is not a valid Base64-encoded string");
            return;
        }
    }
}
```

```

        if (keyBytes < MIN_KEY_BYTES) {
            fail(String.format("app.jwt.secret is too short. It must be at least
%d bytes (256 bits), but was %d bytes", MIN_KEY_BYTES, keyBytes));
        }
    }
}

```

- ограничење броја захтева на API приступним тачкама.

```

@Component
public class RateLimitingFilter extends OncePerRequestFilter {

    private static final int CAPACITY = 20;
    private static final Duration WINDOW = Duration.ofMinutes(1);
    private static final int MAX_TRACKED_CLIENTS = 10_000;

    private final Map<String, Bucket> buckets = new ConcurrentHashMap<>();

    private final ObjectMapper objectMapper;
    @Value("${app.rate-limit.trust-forwarded-for:false}")
    private final boolean trustForwardedFor;

    ...

    @Override
    protected void doFilterInternal(
        @NonNull HttpServletRequest request,
        @NonNull HttpServletResponse response,
        @NonNull FilterChain filterChain)
        throws ServletException, IOException {

        String path = request.getRequestURI();
        if (!path.startsWith("/api/auth")) {
            filterChain.doFilter(request, response);
            return;
        }

        Bucket bucket = resolveBucket(getClientIp(request));
        if (bucket == null) {
            reject(response, WINDOW.toSeconds());
            return;
        }

        ConsumptionProbe probe = bucket.tryConsumeAndReturnRemaining(1);
        if (probe.isConsumed()) {
            filterChain.doFilter(request, response);
        } else {
            reject(response, secondsUntilRefill(probe));
        }
    }

    private void reject(HttpServletResponse response, long retryAfterSeconds)
    throws IOException {
        ApiError apiError = new ApiError(
            HttpStatus.TOO_MANY_REQUESTS.value(),
            "Too many requests. Please try again later.",
            null,
            LocalDateTime.now()
        );

        response.setStatus(HttpStatus.TOO_MANY_REQUESTS.value());
        response.setContentType(MediaType.APPLICATION_JSON_VALUE);
    }
}

```

```

        response.setHeader(HttpHeaders.RETRY_AFTER,
Long.toString(retryAfterSeconds));
        response.getWriter().write(objectMapper.writeValueAsString(apiError));
    }

    private @Nullable Bucket resolveBucket(String clientId) {
        Bucket existing = buckets.get(clientId);
        if (existing != null) {
            return existing;
        }

        if (buckets.size() >= MAX_TRACKED_CLIENTS) {
            evictRefilledBuckets();
            if (buckets.size() >= MAX_TRACKED_CLIENTS) {
                return null;
            }
        }

        return buckets.computeIfAbsent(clientId, k -> createNewBucket());
    }

    private void evictRefilledBuckets() {
        buckets.values().removeIf(bucket -> bucket.getAvailableTokens() >=
CAPACITY);
    }

    private String getClientIp(@NonNull HttpServletRequest request) {
        if (trustForwardedFor) {
            String forwarded = request.getHeader("X-Forwarded-For");
            if (forwarded != null && !forwarded.isBlank()) {
                return forwarded.split(",", 2)[0].trim();
            }
        }
        return request.getRemoteAddr();
    }
}

```

Безбедносне мере остварене на страни клијента:

- **изоловање стилова** областима приказа елемената система дизајна;
- **нормализација текста** над свим што прелази границу поверења у заглављу;
- **брисање низа обавештења при одјави** (12.6), пошто порука може носити име пацијента из сесије која је престала;
- **сесија као стање с једним писцем и с приступом без права уписа** за све остале (12.9).

13 Закључак

Мастер радом покривени су принципи, технике, предности и недостаци које примена микрофронтенд архитектуре носи. Изложени су појам софтверске архитектуре и архитектонске карактеристике као критеријум по коме се архитектонски стилови упоређују, монолитни и дистрибуирани стилови и настанак фронтенд монолита као последице раста једностраничне апликације. Затим је обрађена сама микрофронтенд архитектура и њени механизми: технике композиције и рутирања, начини комуникације међу деловима интерфејса, систем дизајна као носиоца конзистентности изгледа, безбедносне границе и, на крају, анти-патерни које таква подела омогућава. Уз тај теоријски део изграђен је и вреднован артефакт: MicroMedic, информациони систем за заказивање и евиденцију лекарских прегледа, чији је клијентски слој реализован као више независно испоручивих микрофронтендова намерно писаних у различитим радним оквирима.

Најоспораванија одлука коју микрофронтенд архитектура допушта јесте употреба више радних оквира у једном корисничком интерфејсу, коју литература најчешће описује као анти-патерн. Изградњом и вредновањем артефакта показало се да та оцена није својство разноликости, него последица одсуства механизма управљања. Разноликост оквира по себи не ствара ни неусаглашен изглед ни отежано одржавање; њих ствара то што нико није одредио које су одлуке заједничке и чиме се спроводе. Када су те одлуке одређене - једна изведба сваке заједничке компоненте, једна лествица слојева приказа, један начин обраћања серверском слоју, један типизиран канал за размену међу фрагментима и обавезна провера типова у свим пакетима - употреба више оквира остаје контролисана одлука. Оно што дели контролисану више-оквирност од анархије није број оквира, него постојање тих одлука и постојање начина да се спроведу.

Из истог налаза следи и оно што се о конзистентности изгледа може пренети на друга решења. Конзистентност између делова интерфејса писаних у различитим оквирима не мора да се одржава дисциплином аутора, него се може обезбедити конструкцијом. Ако су заједничке компоненте написане једном, као елементи које прегледач сам познаје, и ако свој приказ смештају у DOM „под сенком“, спољашњи код не може да промени његову унутрашњост, не јер је забрањено него зато што је технички неизводљиво, што је доказ постављене добре инфраструктуре, где систем не дозвољава кршење основаних правила, чак и уколико постоји таква намера. Свака измена изгледа тада важи за све делове истовремено, без обзира на оквир у коме је део написан. Показало се и да везивни слој између таквих компонената и оквира није трошак која се плаћа по сваком оквиру, него надомештање тачно одређеног недостатка

појединог оквира: један од употребљених оквира не захтева ништа осим једне линије кода. Тиме уобичајена тврдња да више-оквирност нужно множи количину кода губи општост.

Трећи налаз тиче се комуникације међу деловима интерфејса. Начин на који се два дела обавештавају не одређује то који су то делови, него њихов однос у саставу екрана. Део садржан у другом делу податке добија својствима која му надређени поставља, а обавештење уназад шаље догађајем који се издиже на родитељске компоненте. Делови приказани један уз други на истом екрану обавештавају се заједничким каналом. При прелазу с једног екрана на други, међутим, податак се носи у адреси, зато што други део у тренутку слања још не постоји. Из тога следи да је досежност особина канала, а не пара учесника: пре питања како ће се два дела обавестити стоји питање да ли су икада приказана истовремено.

Четврти налаз одређује докле микрофронтенд архитектура посеже у погледу безбедности. Она уводи стварне границе између подручја одговорности, али безбедносне границе на страни клијента не може да успостави. Свака провера у обмотачу или у делу интерфејса јесте навигација и служи томе да корисник не дође до празног или бесмисленог екрана, док одобравање приступа подацима остаје на серверском слоју и мора се извршити при сваком захтеву. Изолације међу деловима, у смислу у коме је она особина одвојених процеса, нема ни као могућности, зато што сви делови једне стране деле једно извршно окружење једног документа. За домен у коме се обрађују медицински подаци то није формалност: поверљивост и потпуност трага приступа не могу постати својство поделе интерфејса, него остају обавеза серверског слоја, а подела интерфејса не сме да их учини мање прегледним.

Микрофронтенд архитектура плаћа се у изворном коду: један пројекат постаје више пројеката, свака заједничка зависност постаје предмет договора, а састављање целине премешта се у прегледач. Оно што се за то добија је да један тим објави свој део не питајући остале, да се различити ритмови измена раздвоје и да отказ једног дела не обори остатак система, по природи је везано за процес испоруке и постаје видљиво само кроз њега. Отуда и обим вредновања у овом раду: својства која се исказују у структури решења проверена су на изграђеном артефакту, док су она која се исказују у његовој испоруци остала описана.

Циљ рада није био да микрофронтенд архитектуру представи као бољи избор од монолитне. Њене су користи по природи користи испоруке, па се и одлука о њеној примени доноси на основу тога постоји ли више тимова који исти интерфејс мењају различитим ритмовима и потреба да своје делове објављују независно једни од других. Где тога има, све што је овим радом утврђено о границама, конзистентности изгледа, комуникацији и безбедносним границама важи и примењиво је. Где тога нема, исте границе међу поддоменима могу се

поставити и унутар једног пројекта, уз мањи број непогодности које ова архитектура доноси, а одлука о самој подели може сачекати. Растућа употреба микрофронтенд архитектуре сама по себи не треба да буде разлог за њено увођење и на другим пројектима.

Правци даљих истраживања следе из истог разлога. Први је постављање процеса који гради и објављује само измењени део интерфејса, чиме постају мерљива питања која су овде остала само описана: количина кода коју прегледач преузима по појединој рути и кроз читаво путовање корисника, време од измене до њене видљивости кориснику, и понашање решења када један његов део није доступан. Други је организациона страна поделе, коју ниједан технички механизам не замењује; расподела власништва над подручјима, договор о заједничким зависностима и начин на који се примећује да је граница међу деловима почела да се нарушава. Прво даје мере, друго даје носиоце тих мера, а заједно попуњавају другу половину одговара на питање под којим условима микрофронтенд архитектура исплаћује своју цену.

Микрофронтенд архитектура тако није техника клијентског слоја него техника испоруке, и по томе се процењује. MicroMedic је показао да се њене границе могу поставити и одржати добро подељенима, да се конзистентност изгледа међу различитим радним оквирима може обезбедити добром инфраструктуром, и да употреба више оквира уз одређене механизме управљања остаје контролисана одлука, а не анти-патерн. Оно што остаје да се покаже није у архитектури, него око ње.

Литература

- Chen, K. C. (2021, Август 24). *Micro Frontend Guide: Technical Integrations*. Преузето Септембар 1, 2026 са Trend Micro: https://www.trendmicro.com/de_de/research/21/h/micro-frontend-guide-technical-integrations.html
- Clements, P., Bachmann, F., Bass, L., Garlan, D., Ivers, J., Little, R., . . . Stafford, J. (2010). *Documenting Software Architectures: Views and Beyond, Second Edition*. Boston: Addison-Wesley.
- Collins, P., Gowans, A., Ackerman, J. S., & Scruton, R. (2026, Јул 20). *architecture - Expression of technique*. Преузето Септембар 1, 2026 са Britannica: <https://www.britannica.com/topic/architecture>
- Conway, M. E. (1968). *Conway's Law*.
- Das, D. K. (2024, Октобар 13). *Backend For Front End- Architectural Pattern*. Преузето Септембар 1, 2026 са Medium: <https://medium.com/@dipakkrdas/backend-for-front-end-architectural-pattern-4500b262047e>
- Fowler, M. (2012, Мај 1). *Test Pyramid*. Преузето Септембар 1, 2026 са Martin Fowler: <https://martinfowler.com/bliki/TestPyramid.html>
- Gandhi, R., Richards, M., & Ford, N. (2024). *Head First Software Architecture: A Learner's Guide to Architectural Thinking*. O'Reilly Media, Inc.
- Garrett, J. J. (2005). *Ajax: A new approach to web applications*. Washington.
- Geers, M. (2020). *Micro Frontends in Action*. Manning Publications.
- Glazkov, D. (2011, Јануар 14). *What the Heck is Shadow DOM?* Преузето Мај 11, 2025 са Glazkov: <https://glazkov.com/2011/01/14/what-the-heck-is-shadow-dom/>
- Maruthi. (2022, Мај 27). *Planning for Blue-Green Deployments*. Преузето Септембар 1, 2026 са Medium: <https://medium.com/@maruthijr/planning-for-blue-green-deployments-3a04f28c4a6d>

- Mezzalira, L. (2020, Maj). *Micro-frontends in context*. Retrieved Септембар 1, 2026, from Increment: <https://increment.com/frontend/micro-frontends-in-context/>
- Mezzalira, L. (2025). *Building Micro-Frontends* (Друго изд.). O'Reilly Media, Inc.
- Mikowski, M. S., & Powell, J. C. (2013). *Single Page Web Applications*. Manning.
- Mishra, M., Kunde, S., & Nambiar, M. (2018). *Cracking the monolith: Challenges in data transitioning to cloud native architectures*. New York, NY, USA: Association for Computing Machinery.
- Monteiro, F. (2014). *Learning single-page web application development*. Packt Publishing Ltd.
- Mosyan, D. (2023, Септембар 23). *Claim-Check Design Pattern*. Преузето Септембар 1, 2026 са Medium: <https://medium.com/@dmosyan/claim-check-design-pattern-603dc1f3796d>
- Mukhadin, B. (2026, Maj 11). *GraphQL protocol*. Преузето Септембар 1, 2026 са Wallarm: <https://www.wallarm.com/what/what-is-graphql-definition-with-example>
- Münster, S. A. (2024). *Basics and Definitions*. In *Handbook of Digital 3D Reconstruction of Historical Architecture*. Cham.
- Neill, C. J., Laplante, P. A., & DeFranco, J. F. (2011). *Antipatterns*.
- Newman, S. (2021). *Building Microservices, 2nd Edition* (Друго изд.). O'Reilly Media Inc.
- Nwamba, C. (2024, Новембар 26). *The Ultimate Guide to the Broadcast Channel API*. Преузето Септембар 1, 2026 са Telerik & Kendo UI: <https://www.telerik.com/blogs/ultimate-guide-broadcast-channel-api>
- Ottaviani Aragão, E. (2024, Октобар 27). *Web Components — A Practical Perspective Using Custom Elements*. Преузето Септембар 1, 2026 са Medium: <https://eduardo-ottaviani.medium.com/web-components-a-practical-perspective-using-custom-elements-7523a6462387>
- Ozkaya, M. (2021, Септембар 7). *API Gateway Pattern*. Преузето Септембар 1, 2026 са Medium: <https://medium.com/design-microservices-architecture-with-patterns/api-gateway-pattern-8ed0ddfce9df>
- Rappl, F. (2024). *The Art of Micro Frontends*.

- Rappl, F. (2024, Март 20). *Top 10 Micro Frontend Anti-Patterns*. Преузето Септембар 1, 2026 са DEV Community: <https://dev.to/florianrappl/top-10-micro-frontend-anti-patterns-3809>
- Rastogi, A. (2022, Фебруар 10). *Canary Deployment Strategy: Benefits, Constraints And How It Can Be Used For Application Traffic Management*. Преузето Септембар 1, 2026 са AppViewX: <https://www.appviewx.com/blogs/canary-deployment-strategy-benefits-constraints-and-how-it-can-be-used-for-application-traffic-management/>
- Richards, M., & Ford, N. (2025). *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media, Inc.
- Rufus, V. J. (2023). *Building Micro Frontends with React 18*.
- Vaishnavi, V. K., & Kuechler, W. (2015). *Design Science Research Methods and Patterns*.
- Villavicencio, N. (2023, Август 29). *Strangler Fig Pattern: How to migrate from an old to a better architecture*. Преузето Септембар 1, 2026 са Samurai Developer: <https://samurai-developer.com/strangler-fig-pattern/>
- Wright, G. (2022, Април 8). *inline frame (iframe)*. Преузето Мај 5, 2025 са TechTarget: [https://www.techtarget.com/whatis/definition/IFrame-Inline-Frame#:~:text=An%20inline%20frame%20\(iframe\)%20is,webpage%20within%20the%20parent%20page.](https://www.techtarget.com/whatis/definition/IFrame-Inline-Frame#:~:text=An%20inline%20frame%20(iframe)%20is,webpage%20within%20the%20parent%20page.)
- Yadav, P. (2025, Август 5). *What are Single Page Apps?* Преузето Септембар 4, 2025 са GeeksforGeeks: <https://www.geeksforgeeks.org/what-are-single-page-apps/>
- Влајић, С. (2020). *ПРОЈЕКТОВАЊЕ СОФТВЕРА*. Београд: Златни пресек.